

# 내가 한 것들

## Animation

- IK Retargeting을 통해 Mannequin에서 사용 가능한 Animation을 BountyHunter Character에서도 사용할 수 있도록 구성했다
- `CharacterState`와 `Locomotion`, `Gesture`, `Look Tracking`을 서로 다른 역할로 분리해서 관리했다
- `Idle`, `Talk`, `Dance`는 Character가 유지하는 지속 State로 StateMachine에서 관리하고, 이동이나 Gesture가 발생해도 CharacterState 자체는 필요한 경우 유지 되도록 했다
- `Walk`, `Turn`, `Fall`, `Land`는 CharacterState와 분리된 Locomotion Animation으로 처리했다
- Gesture는 CharacterState를 변경하지 않는 일회성 Action으로 구성하고 `GestureSlot`을 이용한 Animation Montage로 재생했다
- Mouse Look은 Aim Offset으로 구현하고, Yaw / Pitch를 보간해서 Character의 상체가 Mouse를 부드럽게 따라가도록 했다
- 이동 시작과 종료 시 Character가 즉시 Walk / Idle로 전환되면서 회전하는 것이 어색해서 `MovementPhase`를 기준으로 Walk In Place를 추가했다
- Dance Animation의 특정 시점에는 Anim Notify를 사용해 Sound 재생과 정지를 연결했다

## IK Retargeting

- `IK_Mannequin`과 `IK_BountyHunter`에 대응되는 Retarget Chain을 구성했다
- `RTG_Mannequin_BountyHunter`에서 Mannequin을 Source, BountyHunter를 Target으로 지정해 Animation을 Retarget했다
- Spine, Arm, Leg, Finger, Neck, Head, Root Chain을 대응시켜 전신 Animation을 사용할 수 있도록 했다
- Preview 단계에서 Source와 Target Mesh를 함께 확인하고 Target Mesh Offset과 Scale을 조정했다
- Retarget 결과는 단순히 Animation이 재생되는지만 확인하지 않고 손, 발, Spine, Head 등의 변형과 Character Asset의 추가 Bone 영향을 함께 확인했다

```
SKM_Quinn_Simple
→ IK_Mannequin
→ RTG_Mannequin_BountyHunter
→ IK_BountyHunter
→ SK_StylizedBountyHunter
```

## CharacterState와 Locomotion 분리

- `CharacterState`: Character가 현재 유지해야 하는 기본 Animation State
  - `Idle`
  - `Talk`
  - `Dance`
- `Locomotion`: Character의 실제 이동 상황에 따라 일시적으로 재생되는 Animation
  - `Turn`
  - `Walk`
  - `Fall`
  - `Land`
- 이동을 시작했다고 CharacterState를 `Walk`로 변경하지 않도록 했다
- 예를 들어 `CharacterState = Talk`인 상태에서 Position 이동을 시작하면 이동 중에는 Walk Animation을 사용하지만 CharacterState 자체는 Talk로 유지하고, 이동 종료 후 다시 Talk Animation으로 복귀할 수 있도록 구성했다
- Character의 Presentation 상태와 실제 이동 상태를 분리해 서로의 역할이 섞이지 않도록 했다

```
CharacterState = Talk
→ Position 이동 시작
→ Locomotion = Walk
→ 목적지 도착
→ Talk Animation 복귀
```

## StateMachine

- `Idle`, `Talk`, `Dance`를 CharacterState 기반 StateMachine으로 구성했다
- `StudioManager.CurrentCharacterState` 값을 가져와 현재 CharacterState에 맞는 Animation State로 전환했다
- CharacterState 전환과 Locomotion 전환을 하나의 조건으로 섞지 않고 각각 필요한 상태값을 사용하도록 했다

## CharacterState 전환

```
Idle ↔ Talk
Idle ↔ Dance
Talk ↔ Dance
```

- 각 전환은 `CurrentCharacterState` 값을 기준으로 처리했다

## MovementPhase 기반 Locomotion

- Animation Blueprint가 Character 이동 과정을 자체적으로 추측하지 않도록 `BP_BountyHunter` 가 관리하는 `MovementPhase` 값을 전달받아 Animation을 결정했다
- Movement Logic과 Animation이 같은 상태값을 사용하도록 해서 실제 Actor 이동과 Animation 전환이 어긋나는 것을 줄였다

```
Idle
→ TurningToPath
→ Moving
→ TurningToCamera
→ Idle
```

### TurningToPath

- Character가 이동하기 전에 실제 이동 방향으로 먼저 회전하는 단계
- 바로 Walk Animation으로 전환하면 Character가 제자리에서 빠르게 미끄러지듯 회전하는 느낌이 발생해서 Walk In Place Animation을 사용했다

```
MovementPhase == TurningToPath
→ MM_Walk_InPlace
```

### Moving

- 이동을 시작하면 `MovementPhase = Moving`
- 실제 Character Velocity의 크기를 `Speed` 로 계산하고 1D Blend Space에서 이동 Animation 선택

```
MovementPhase == Moving
→ BS_Walk
```

### TurningToCamera

- 목적지 도착 후 Character가 Stage 정면을 다시 바라보는 동안에도 바로 Idle로 전환하지 않고 Walk In Place를 사용했다

```
BS_Walk
→ MM_Walk_InPlace
Condition:
MovementPhase == TurningToCamera
```

- 이동 시작과 종료의 회전 구간엔 같은 Walk In Place Animation을 사용해 Actor Rotation과 Animation이 시각적으로 연결되도록 했다

## Blend Space 1D

- Character의 이동 Animation은 이동 방향별 Animation이 아니라 정면 이동 Animation만 사용할 계획이었기 때문에 2D가 아닌 1D Blend Space를 사용했다
- Character Velocity의 크기를 `Speed` 로 계산해 Blend Space 입력값으로 사용했다

```
Speed = 0
→ IdleToWalk

Speed = 250
→ Walk

Speed = 500
→ Jog
```

- Character의 실제 이동 속도에 맞춰 Idle에서 Walk, Jog까지 자연스럽게 Blend되도록 구성했다

## Falling과 Landing Animation

- Navigation 이동 중 Character가 경사나 높이 차로 Falling 상태가 될 수 있기 때문에 `CharacterMovement.IsFalling()` 값을 Animation Blueprint에서 확인했다
- Falling 상태가 되면 `MM_Fall_Loop` 으로 전환하고 지면에 닿으면 `MM_Land` 를 재생했다

```
BS_Walk
→ MM_Fall_Loop
→ MM_Land
→ BS_Walk
```

- Landing Animation이 끝나기 직전에 다시 Walk 상태로 돌아가 이동을 이어가도록 했다
- Character Movement 쪽에서도 착지 순간 Navigation 이동을 잠시 정지한 뒤 Landing Animation 재생 시간을 확보하고 다시 이동하도록 구성해 실제 Movement 와 Animation이 함께 맞도록 처리했다

## Gesture

- Gesture는 `Idle`, `Talk`, `Dance` 와 같은 지속 State가 아니라 Operator가 실행하는 짧은 일회성 Action으로 분리했다
- Gesture를 실행해도 `CurrentCharacterState` 자체는 변경하지 않도록 했다

```
CharacterState = Talk
→ Wave 실행
→ Wave Montage 1회 재생
→ Montage 종료
→ Talk 유지
```

- **GestureSlot**을 별도로 만들어 Gesture Montage가 신호를 받을 때만 한 번 재생되도록 했다
- Montage Loop는 사용하지 않았다

## Gesture Montage

- Clap
- CrossHands
- Laugh
- Point
- Pose
- Wave

```
DefaultGroup.GestureSlot
→ Gesture Montage
→ 1회 재생
→ 기존 CharacterState Pose로 복귀
```

- Gesture와 Base CharacterState를 분리해서 일회성 동작 때문에 지속 State가 변경되지 않도록 했다

## Dance Sequence

- Dance는 하나의 Animation만 반복하지 않고 여러 Dance Animation을 이어서 재생하도록 구성했다
- 첫 번째 Dance Animation의 남은 시간이 일정 이하가 되면 다음 Dance Animation으로 전환하도록 했다
- 두 Dance Animation이 반복될 수 있도록 State를 연결했다

```
CharacterState_Dance
→ CharacterState_Dance_1
```

- Dance 중 **CurrentCharacterState**가 **Idle** 또는 **Talk**로 변경되면 State Alias를 사용해 해당 CharacterState로 빠져나갈 수 있도록 했다

## Anim Notify와 Sound

- Dance Animation과 음악 재생 시점을 연결하기 위해 Anim Notify를 사용했다
- Animation 재생 시작 자체를 기준으로 Sound를 틀지 않고 Animation 내부의 Notify 시점을 Event로 사용했다

```
Dance Notify
→ StudioManager.PlayMusic

DanceEnd Notify
→ StudioManager.StopMusic
```

- Animation Timeline과 Sound 재생 시점을 직접 연결해 Dance 상태와 배경음이 함께 동작하도록 했다

## AnimGraph Layer 구조

Animation Graph는 다음 순서로 구성했다

```
Locomotion StateMachine
→ Cached Locomotion

Cached Locomotion
→ GestureSlot
→ Cached Gesture

Cached Gesture
→ Aim Offset
→ Final Pose
```

- 먼저 Locomotion과 CharacterState 결과를 **Cached Locomotion**으로 저장했다
- 그 Pose 위에 **GestureSlot**을 적용한 뒤 다시 **Cached Gesture**로 저장했다
- 최종 단계에서 Aim Offset을 적용해 Look Tracking을 추가했다
- 같은 Locomotion Pose를 여러 단계에서 다시 계산하지 않고 Cached Pose를 재사용하도록 했다

## Aim Offset과 Mouse Look Tracking

- Mouse Cursor 방향을 Character가 따라보도록 Aim Offset을 사용했다
- 사용할 수 있는 Aim Offset용 Animation이 따로 없어서 Idle Animation을 기준으로 직접 Pose를 제작했다

- Level Sequence에서 기존 Animation Pose를 불러온 뒤 상체 방향을 직접 조정해 Aim Offset에 사용할 5개 Pose Animation을 만들었다

## 직접 제작한 Pose

- **AO\_CC** : Center / Center
- **AO\_CD** : Center / Down
- **AO\_CU** : Center / Up
- **AO\_LC** : Left / Center
- **AO\_RC** : Right / Center

```
Yaw    -90 ~ 90
Pitch  -45 ~ 45

→ 5개 Pose 사이 Blend
→ Character Look 방향 변경
```

## Runtime Yaw / Pitch

- Mouse 위치를 기준으로 계산된 Yaw, Pitch 값은 **BP\_PlayerController**에서 가져왔다
- 값을 그대로 Aim Offset에 입력하지 않고 **FInterpTo**를 거쳐 부드럽게 변화하도록 했다

```
PlayerController AimOffsetYaw
→ FInterpTo
→ Yaw

PlayerController AimOffsetPitch
→ Camera Pitch Offset
→ FInterpTo
→ Pitch
```

## Camera State별 Aim Offset 보정

- Full, Upper, Close Camera는 각각 Character의 화면상 머리 위치가 달라 같은 Pitch 값을 사용하면 Mouse를 바라보는 방향이 시각적으로 어색해지는 문제가 발생했다
- 이를 보정하기 위해 **CurrentCameraState**에 따라 다른 Pitch Offset을 추가했다

```
Full   = +12
Upper  = -10
Close  = -3
```

- Camera 구도에 따라 Character가 Mouse Cursor를 바라보는 방향이 자연스럽게 보이도록 보정했다
- Offset은 항상 적용하지 않고 다음 조건에서만 사용했다

```
LookTrackingActive
&& !StageViewActive
```

- Look Tracking을 사용하지 않거나 Stage View가 활성화된 경우에는 Pitch Offset을 0으로 처리했다
- Stage View의 고정 Camera 구도와 Mouse Look이 충돌하지 않도록 한 처리

## Look Tracking과 Runtime State

- Look Tracking 설정값과 실제 Tracking 동작은 분리해서 처리했다
- Character가 이동하거나 Gesture를 실행하는 동안에는 Mouse Tracking을 계속 적용하지 않고 Look을 Neutral 방향으로 복귀시키도록 구성했다
- 이동 또는 Gesture가 종료된 뒤 Look Tracking 설정이 ON이면 다시 Mouse Tracking을 시작하고, OFF이면 Neutral 상태를 유지하도록 했다
- Look Tracking Button 설정 자체는 유지하면서 실제 Animation 적용 시점만 Runtime 상태에 따라 제어할 수 있도록 했다

```
Look Tracking ON

일반 상태
→ Mouse Tracking

이동 / Gesture
→ Tracking 일시 중지
→ Neutral

동작 종료
→ Look Tracking ON이면 Tracking 재개
→ OFF이면 Neutral 유지
```

- 이동, Gesture, Aim Offset이 동시에 적용되어 상체 Animation이 지나치게 복잡하게 섞이는 것을 줄였다

## Animation 구조 정리

```
Animation System
|
```

```

├─ CharacterState
│  ├─ Idle
│  ├─ Talk
│  └─ Dance
├─ Locomotion
│  ├─ TurningToPath
│  ├─ Walk
│  ├─ Falling
│  ├─ Landing
│  └─ TurningToCamera
├─ Gesture
│  └─ GestureSlot Montage
├─ Dance
│  ├─ Dance Sequence
│  └─ Anim Notify → Sound
└─ Look Tracking
   ├─ Aim Offset
   ├─ Mouse Yaw / Pitch
   ├─ Camera State Pitch 보정
   └─ Runtime Suspend / Resume

```

## ABP\_BountyHunter

### EventGraph

```

ABP_BountyHunter / EventGraph

Initialize:
InitializeAnimation
→ GetRef(self)

AnimNotify:
Dance → StudioManager.PlayMusic
DanceEnd → StudioManager.StopMusic

UpdateAnimation:
Branch(
    IsValid(BountyHunterRef)
    && IsValid(StudioManager)
    && IsValid(PlayerControllerRef)
)
→ Sequence

Then0:
Speed = VectorLength(BountyHunterRef.GetVelocity())
IsMoving = Speed > 0.0

Then1:
CurrentCharacterState = StudioManager.CurrentCharacterState
IsTurning = BountyHunterRef.IsTurning
IsFalling = BountyHunterRef.CharacterMovement.IsFalling()
MovementPhase = BountyHunterRef.MovementPhase
CurrentCameraState = StudioManager.CurrentCameraState

Then2:
Yaw = FInterpTo(
    Current = Yaw,
    Target = PlayerControllerRef.AimOffsetYaw,
    DeltaTime = GetWorldDeltaSeconds(),
    InterpSpeed = 10.0
)

CameraPitchOffset = Select(CurrentCameraState)
    Full = 12.0
    Upper = -10.0
    Close = -3.0

PitchOffset =
    PlayerControllerRef.LookTrackingActive
    && !StudioManager.StageViewActive
    ? CameraPitchOffset
    : 0.0

Pitch = FInterpTo(
    Current = Pitch,
    Target = PlayerControllerRef.AimOffsetPitch + PitchOffset,
    DeltaTime = GetWorldDeltaSeconds(),
    InterpSpeed = 10.0
)

```

### GetRef

```

ABP_BountyHunter / GetRef

GetRef:
GetPlayerPawn(PlayerIndex = 0)
→ CastTo BP_BountyHunter
→ BountyHunterRef = CastResult

GetActorOfClass(
    ActorClass = BP_StudioManager
)
→ StudioManager = ReturnValue

```

```

GetPlayerController(PlayerIndex = 0)
→ CastTo BP_PlayerController
→ PlayerControllerRef = CastResult

```

## AnimGraph

```

ABP_BountyHunter / AnimGraph

Locomotion:
Locomotion StateMachine
→ SaveCachedPose("Cached Locomotion")

Gesture:
UseCachedPose("Cached Locomotion")
→ Slot("GestureSlot")
→ SaveCachedPose("Cached Gesture")

AimOffset:
A0_MouseTracking
    BasePose = UseCachedPose("Cached Gesture")
    Yaw = Yaw
    Pitch = Pitch
    Alpha = 1.0

FinalBlend:
LayeredBlendPerBone(
    BasePose = UseCachedPose("Cached Gesture"),
    BlendPose0 = A0_MouseTracking,
    BlendWeight0 = 1.0
)
→ OutputPose

```

## Locomotion

```

ABP_BountyHunter / AnimGraph / Locomotion StateMachine

Entry:
Entry
→ CharacterState_Idle

CharacterState:
CharacterState_Idle
↔ CharacterState_Talk

CharacterState_Idle
↔ CharacterState_Dance

CharacterState_Talk
↔ CharacterState_Dance

Walk / Turn:
CharacterState_Idle
↔ MM_Walk_InPlace

MM_Walk_InPlace
↔ BS_Walk

CharacterState_Idle
↔ BS_Walk

Falling / Landing:
BS_Walk
→ MM_Fall_Loop
→ MM_Land
→ BS_Walk

Dance:
CharacterState_Dance
↔ CharacterState_Dance_1

CharacterState_Dance_1
→ StateAlias("Idle")

CharacterState_Dance_1
→ StateAlias("Talk")

```

## Transition Rules

```

ABP_BountyHunter / AnimGraph / Locomotion StateMachine / Transition Rules

Movement:
CharacterState_Idle
→ MM_Walk_InPlace
Condition:
MovementPhase == TurningToPath

MM_Walk_InPlace
→ CharacterState_Idle
Condition:
MovementPhase == Idle

```

```

MM_Walk_InPlace
→ BS_Walk
Condition:
MovementPhase == Moving

BS_Walk
→ MM_Walk_InPlace
Condition:
MovementPhase == TurningToCamera

BS_Walk
→ CharacterState_Idle
Condition:
MovementPhase == Idle

CharacterState_Idle
→ BS_Walk
Condition:
MovementPhase == Moving

Falling:
BS_Walk
→ MM_Fall_Loop
Condition:
IsFalling

MM_Fall_Loop
→ MM_Land
Condition:
!IsFalling

MM_Land
→ BS_Walk
Condition:
TimeRemainingRatio(MM_Land) < 0.1

CharacterState:
CharacterState_Idle
→ CharacterState_Dance
Condition:
CurrentCharacterState == Dance

CharacterState_Dance
→ CharacterState_Idle
Condition:
CurrentCharacterState == Idle

CharacterState_Dance
→ CharacterState_Talk
Condition:
CurrentCharacterState == Talk

CharacterState_Talk
→ CharacterState_Dance
Condition:
CurrentCharacterState == Dance

CharacterState_Talk
→ CharacterState_Idle
Condition:
CurrentCharacterState == Idle

CharacterState_Idle
→ CharacterState_Talk
Condition:
CurrentCharacterState == Talk

DanceSequence:
CharacterState_Dance
→ CharacterState_Dance_1
Condition:
TimeRemaining(pop-dance-shutdown) < 1.0

CharacterState_Dance_1
→ CharacterState_Dance
Condition:
TimeRemaining(pop-dance-shutdown) < 1.0

CharacterState_Dance_1
→ StateAlias("Talk")
Condition:
CurrentCharacterState == Talk

CharacterState_Dance_1
→ StateAlias("Idle")

```

```
Condition:  
CurrentCharacterState == Idle
```

## Blend Space 1D

```
ABP_BountyHunter / AnimGraph / Locomotion / BS_Walk  
  
BS_Walk State:  
Speed  
→ BS_Walk.Speed  
→ OutputAnimationPose  
  
BS_Walk / BlendSpace  
  
Axis:  
HorizontalAxis = Speed  
Min = 0.0  
Max = 500.0  
GridDivisions = 2  
  
BlendSamples:  
Speed = 0.0  
→ IdleToWalk  
  
Speed = 250.0  
→ MF_Unarmed_Walk_Fwd  
  
Speed = 500.0  
→ MF_Unarmed_Jog_Fwd
```

## Animation Montage

```
GestureMontages / DefaultGroup.GestureSlot  
  
AM_Clap:  
MontageSection = Clap  
Slot = DefaultGroup.GestureSlot  
Loop = false  
  
AM_CrossHands:  
MontageSection = CrossHands  
Slot = DefaultGroup.GestureSlot  
Loop = false  
  
AM_Laugh:  
MontageSection = Laugh  
Slot = DefaultGroup.GestureSlot  
Loop = false  
  
AM_Point:  
MontageSection = Point  
Slot = DefaultGroup.GestureSlot  
Loop = false  
  
AM_Pose:  
MontageSection = Pose  
Slot = DefaultGroup.GestureSlot  
Loop = false  
  
AM_Wave:  
MontageSection = Wave  
Slot = DefaultGroup.GestureSlot  
Loop = false
```

## Aim Offset

```
AimOffset Assets / A0_MouseTracking  
  
SourceAnimation:  
Base = Idle Animation  
  
Creation:  
Idle Animation  
→ Level Sequence에서 기준 Pose로 사용  
→ 상체 각도를 방향별로 직접 조정  
→ 각 방향 Pose Animation 생성  
  
CreatedPoses:  
A0_CC = Center / Center  
A0_CD = Center / Down  
A0_CU = Center / Up  
A0_LC = Left / Center  
A0_RC = Right / Center  
  
A0_MouseTracking / AimOffset  
  
HorizontalAxis:  
Name = Yaw
```

```

Min = -90.0
Max = 90.0
GridDivisions = 4

VerticalAxis:
Name = Pitch
Min = -45.0
Max = 45.0
GridDivisions = 2

BlendSamples:
Yaw = 0.0, Pitch = 0.0
→ A0_CC

Yaw = 0.0, Pitch = -45.0
→ A0_CD

Yaw = 0.0, Pitch = 45.0
→ A0_CU

Yaw = -90.0, Pitch = 0.0
→ A0_LC

Yaw = 90.0, Pitch = 0.0
→ A0_RC

Runtime:
Yaw + Pitch
→ A0_MouseTracking
→ 5개 Pose 사이를 Blend
→ 캐릭터 상체 Look 방향 변경

```

## IK Retargeter

```

IK Retargeting Setup

IK_Mannequin:

PreviewMesh = SKM_Quinn_Simple

RetargetChains:
Root:
    StartBone = root
    EndBone = root

Spine:
    StartBone = spine_01
    EndBone = spine_05

LeftClavicle:
    StartBone = clavicle_l
    EndBone = clavicle_l

LeftArm:
    StartBone = upperarm_l
    EndBone = hand_l

LeftMiddle:
    StartBone = middle_metacarpal_l
    EndBone = middle_03_l

LeftPinky:
    StartBone = pinky_metacarpal_l
    EndBone = pinky_03_l

LeftRing:
    StartBone = ring_metacarpal_l
    EndBone = ring_03_l

LeftThumb:
    StartBone = thumb_01_l
    EndBone = thumb_03_l

LeftIndex:
    StartBone = index_metacarpal_l
    EndBone = index_03_l

RightClavicle:
    StartBone = clavicle_r
    EndBone = clavicle_r

RightArm:
    StartBone = upperarm_r
    EndBone = hand_r

RightMiddle:
    StartBone = middle_metacarpal_r
    EndBone = middle_03_r

RightPinky:
    StartBone = pinky_metacarpal_r
    EndBone = pinky_03_r

RightRing:
    StartBone = ring_metacarpal_r
    EndBone = ring_03_r

RightThumb:
    StartBone = thumb_01_r

```

```

        EndBone = thumb_03_r

    RightIndex:
        StartBone = index_metacarpal_r
        EndBone = index_03_r

    RightLeg:
        StartBone = thigh_r
        EndBone = ball_r

    LeftLeg:
        StartBone = thigh_l
        EndBone = ball_l

    Head:
        StartBone = head
        EndBone = head

    Neck:
        StartBone = neck_01
        EndBone = neck_02

    IK_BountyHunter:

    PreviewMesh = SK_StylizedBountyHunter

    RetargetChains:
    Spine:
        StartBone = spine_01
        EndBone = spine_05

    Neck:
        StartBone = neck_01
        EndBone = neck_02

    Head:
        StartBone = head
        EndBone = head

    LeftClavicle:
        StartBone = clavicle_l
        EndBone = clavicle_l

    LeftArm:
        StartBone = upperarm_l
        EndBone = hand_l

    LeftIndex:
        StartBone = index_metacarpal_l
        EndBone = index_03_l

    LeftMiddle:
        StartBone = middle_metacarpal_l
        EndBone = middle_03_l

    LeftThumb:
        StartBone = thumb_01_l
        EndBone = thumb_03_l

    LeftPinky:
        StartBone = pinky_metacarpal_l
        EndBone = pinky_03_l

    LeftRing:
        StartBone = ring_metacarpal_l
        EndBone = ring_03_l

    RightClavicle:
        StartBone = clavicle_r
        EndBone = clavicle_r

    RightArm:
        StartBone = upperarm_r
        EndBone = hand_r

    RightPinky:
        StartBone = pinky_metacarpal_r
        EndBone = pinky_03_r

    RightMiddle:
        StartBone = middle_metacarpal_r
        EndBone = middle_03_r

    RightIndex:
        StartBone = index_metacarpal_r
        EndBone = index_03_r

    RightThumb:
        StartBone = thumb_01_r
        EndBone = thumb_03_r

    RightLeg:
        StartBone = thigh_r
        EndBone = ball_r

    LeftLeg:
        StartBone = thigh_l
        EndBone = ball_l

    Root:
        StartBone = root
        EndBone = root

```

```
RTG_Mannequin_BountyHunter / IK Retargeter

Source:
SourceIKRigAsset = IK_Mannequin
SourcePreviewMesh = SKM_Quinn_Simple

Target:
TargetIKRigAsset = IK_BountyHunter
TargetPreviewMesh = SK_StylizedBountyHunter

Retarget:
SKM_Quinn_Simple
→ IK_Mannequin
→ RTG_Mannequin_BountyHunter
→ IK_BountyHunter
→ SK_StylizedBountyHunter

Preview:
SourceMesh = SKM_Quinn_Simple
TargetMesh = SK_StylizedBountyHunter

TargetMeshOffset:
X = 113.56793
Y = 0.0
Z = 0.0

TargetMeshScale = 1.015984
```

## Stage

- Virtual Performer의 외형과 Lighting 변화가 잘 보이고, Presentation / Performance 상태에 따라 분위기를 바꿀 수 있는 작은 Virtual Studio 형태의 Stage를 목표로 했다
- 원하는 형태의 완성 Stage Asset을 찾기 어려워 Maya에서 직접 모델링했다
- 무대는 Character보다 시각적으로 강하게 튀지 않도록 전체적으로 Dark / Matte 계열로 구성하고, Metal과 Emissive 영역을 Accent로 사용했다
- Stage 전체를 여러 개의 개별 Material로 나누기보다 **RGB Mask Texture 하나로 Material 영역을 구분하는 Master Material 구조**를 만들었다
- Floor, Wall, Platform, Ceiling, Logo는 같은 **M\_Stage\_Master** 구조를 사용하고 각 Mesh의 UV와 Mask Texture만 다르게 적용했다
- Lighting State 변경 시 실제 Light만 바뀌는 것이 아니라 Stage의 Emissive Color도 함께 변경되도록 해서 무대 분위기가 하나의 State처럼 바뀌도록 구성했다

## Stage Modeling

- 원하는 형태의 Virtual Stage Asset을 찾지 못해 Maya에서 직접 제작했다
- Stage는 Character 운용과 Camera 구도가 중심이기 때문에 복잡한 배경보다는 Character를 방해하지 않는 구조로 제작했다
- Floor, Wall, Platform, Ceiling, Logo 영역을 분리해서 Unreal에서 개별 Static Mesh Component로 제어할 수 있도록 구성했다
- Stage 중앙과 후면 구조가 Character의 기본 Presentation 공간이 되도록 만들고, Character Position 이동과 Camera Full / Upper / Close 구도가 모두 보이도록 크기와 형태를 조정했다
- Logo는 큰 Text 형태가 아니라 후면 중앙의 작은 Branding 요소로 사용해 Character보다 시각적으로 앞에 나오지 않도록 했다

## UV와 Mask Texture

- Material 종류가 많지 않았기 때문에 각 Material 영역마다 별도 Material Slot을 만드는 대신 **UV와 Mask Texture를 이용해 하나의 Master Material에서 영역을 분리**했다
- Maya에서 UV를 펼칠 때 같은 재질을 사용할 Face를 같은 영역에 겹쳐 배치했다
- 이 UV Layout을 기준으로 Mask Texture를 제작했다

## Mask Channel

```
R → Emissive
G → Metal
B → Black Matte
RGB가 모두 0인 영역 → Dark Matte
```

- 하나의 Texture에서 RGB Channel을 각각 Material 영역 구분용 Mask로 사용했다
- 동일한 Mask를 Base Color, Roughness, Emissive, Normal 계산에 공통으로 사용했다
- 여러 종류의 Mask Texture를 따로 관리하지 않고 하나의 Mask Texture로 Stage Material 영역을 제어하도록 했다

## Master Material

**M\_Stage\_Master** 하나에서 Stage에 필요한 네 가지 재질 표현을 조합했다

```
Dark Matte
Metal
Emissive
Black Matte
```

## Base Color

- 기본 Dark Base Color를 기준으로 Metal 영역을 `MaskTexture.G` 로 Blend
- Emissive 영역을 `MaskTexture.R` 로 Blend
- 더 어두운 Black Matte 영역을 `MaskTexture.B` 로 Blend

```
Dark
→ G Mask로 Metal Blend
→ R Mask로 Emissive Blend
→ B Mask로 Black Blend
→ Base Color
```

## Roughness

- Dark Matte와 Metal의 Roughness를 다르게 설정
- `MaskTexture.G` 를 사용해 Metal 영역의 Roughness를 별도로 적용
- Emissive 영역은 `MaskTexture.R` 을 이용해 별도의 Roughness 적용

```
Dark Roughness
→ Metal Roughness
    ↑ G

Previous Roughness
→ Emissive Roughness
    ↑ R
```

## Metallic

- Metal 영역이 다른 영역보다 강하게 Metallic 특성을 가지도록 값을 설정했다
- Stage 전체를 금속으로 보이게 하지 않고 Mask로 지정된 부분만 Metal 느낌이 나도록 구성했다

## Emissive

- `MaskTexture.R` 영역에만 Emissive 적용
- `EmissiveColor × EmissiveIntensity × MaskTexture.R` 구조로 구성
- Runtime에서 `EmissiveColor` Parameter를 변경할 수 있도록 만들어 Lighting State와 연결했다

```
EmissiveColor
× EmissiveIntensity
× MaskTexture.R
→ Emissive Color
```

## Normal

- Dark와 Metal 영역의 Normal을 Mask 기준으로 Blend
- Emissive 영역은 표면이 지나치게 요철져 보이지 않도록 Flat Normal 방향으로 Blend
- `FlattenNormal` 을 사용해 전체 Normal 강도를 조정할 수 있도록 했다

## Material Instance 구성

- `M_Stage_Master` 를 직접 각 Mesh에 사용하는 대신 Mesh별 Material Instance를 생성했다
- 동일한 Material Logic을 공유하면서 각 Mesh에 맞는 Mask Texture만 다르게 넣었다

```
M_Stage_Master
├─ MI_Floor
├─ MI_Wall
├─ MI_Platform
├─ MI_Ceiling
└─ MI_Logo
```

- 각 Material Instance에 해당 Mesh UV Layout에 맞는 Mask Texture 적용
- Material 구조는 하나로 유지하면서 Mesh별 Material 영역만 변경 가능
- Stage 전체의 Material 수정이 필요할 경우 Master Material 하나를 수정하면 모든 Stage Mesh에 공통 반영되는 구조

### BP\_Stage

- Stage를 단순 Static Mesh 모음으로 두지 않고 `BP_Stage` 로 묶어서 Runtime에서 무대 전체의 Emissive 상태를 제어할 수 있도록 했다
- Floor, Platform, Wall, Logo, Ceiling을 하나의 Stage Actor에서 관리
- Lighting State가 변경되면 각 Mesh의 Emissive Parameter를 한 번에 변경하도록 구성했다

### SetStageEmissiveColor

- Stage Mesh마다 같은 Node를 반복해서 직접 작성하지 않도록 공통 함수로 분리했다

- 입력받은 Static Mesh Component에 `EmissiveBaseColor`, `EmissiveColor` Parameter를 적용한다

```
StaticMesh
+ Color
→ SetStageEmissiveColor
→ EmissiveBaseColor 변경
→ EmissiveColor 변경
```

- Material 내부에서 실제 Emissive 영역은 `MaskTexture.R`로 제한되어 있기 때문에 Parameter 값을 변경해도 지정된 Emissive 부분만 색이 바뀐다

## Lighting State와 Stage Emissive 연동

- Character Lighting만 변경하면 Stage와 Character의 분위기가 따로 노는 느낌이 생기기 때문에 Stage Emissive도 `E_LightingState`와 연결했다
- `ChangeStageLight`에서 현재 Lighting State에 따라 Stage 전체의 Emissive Color를 변경한다

```
E_LightingState
→ Neutral / Focus
→ DefaultLightColor

→ Stage
→ StageLightColor
```

- `Neutral`, `Focus` 상태에서는 기본 Emissive Color 유지
- `Stage` 상태에서는 Performance용 `StageLightColor` 적용
- Floor, Platform, Wall, Logo, Ceiling에 동일한 State Color를 전달해서 무대 전체가 하나의 Lighting Theme처럼 전환되도록 했다

```
Lighting State 변경
→ Stage Emissive Color 결정
→ Floor
→ Platform
→ Wall
→ Logo
→ Ceiling
```

## Stage Lighting 연출 의도

- Stage Material 자체는 Dark / Metal 계열로 최대한 중립적으로 유지
- Character의 Hair, Clothes, Skin Color가 변경되어도 배경색 때문에 색이 묻히지 않도록 했다
- 기본 상태에서는 어두운 무채색 Stage를 유지하고, Performance 상태에서 Emissive Accent를 변경해 분위기 차이를 만들었다
- Material 전체 Color를 바꾸는 것이 아니라 Emissive 영역만 변경해서 Stage 구조와 재질감은 유지하면서 Lighting 분위기만 바뀌도록 했다
- Character가 중심인 프로젝트이기 때문에 Stage 자체를 지나치게 화려하게 만들기보다 **Character Runtime Control**을 보여주는 배경 역할에 맞게 제한했다

## Stage와 다른 Runtime System의 관계

Stage 자체는 독립적인 배경 Asset이지만 Character Runtime System과 함께 작동하도록 구성했다

```
Lighting State
├─ Character Lighting
│   └─ SpotLight Color / Intensity
└─ Stage
    └─ Emissive Color
```

- 같은 `LightingState`를 기준으로 Character Lighting과 Stage Emissive가 함께 변경
- Character Lighting과 Stage Material이 서로 다른 분위기를 보여주는 문제 감소
- Operator 입장에서는 하나의 Lighting 명령으로 Character와 Stage 분위기를 동시에 변경 가능

## 직접 제작한 부분

```
Stage
├─ Maya Modeling
├─ UV Layout
├─ Mask Texture
├─ Master Material
├─ Material Instance
├─ BP_Stage
└─ Runtime Emissive Control
```

- 외부 Stage Asset을 그대로 사용하지 않고 Modeling부터 Runtime Material Control까지 직접 구성
- Stage 제작 자체가 프로젝트의 중심 기능은 아니지만, Virtual Performer를 보여줄 공간을 프로젝트 목적에 맞게 직접 설계하고 Unreal Runtime System과 연결했다

## 구현하면서 선택한 방식

- Material마다 Mesh를 분리하거나 Material Slot을 계속 추가하는 방식 X
- **RGB Mask + Master Material + Material Instance** 방식 사용
- Lighting State마다 별도의 Material을 교체하는 방식 X
- 동일 Material을 유지하고 Runtime에서 Emissive Parameter만 변경
- Stage마다 별도의 복잡한 Lighting Tool을 만드는 방식 X
- 기존 **E\_LightingState**를 재사용해 Character Lighting과 Stage Emissive를 같은 State에서 제어

⇒ Stage를 별도 시각 Asset으로 끝내지 않고 Runtime Lighting System의 일부로 연결한 구조

## M\_Stage\_Master

```
M_Stage_Master / Material Graph

Material:
Domain = Surface
BlendMode = Opaque
ShadingModel = Default Lit

BaseColor:
T_Dark_BaseColor
+ T_Metal_BaseColor
→ Lerp

MaskTexture.G
→ Lerp.Alpha

LerpResult
+ EmissiveBaseColor
→ Lerp

MaskTexture.R
→ Lerp.Alpha

LerpResult
+ T_Black_BaseColor
→ Lerp

MaskTexture.B
→ Lerp.Alpha

→ BaseColor

Metallic:
Lerp(
    A = 0.0,
    B = MetallicValue,
    Alpha = 0.5
)

MetallicValue = 0.8

→ Metallic

Roughness:
Lerp(
    A = DarkRoughness,
    B = MetalRoughness,
    Alpha = MaskTexture.G
)

DarkRoughness = 0.6
MetalRoughness = 0.2

→ Lerp(
    A = PreviousRoughness,
    B = EmissiveRoughness,
    Alpha = MaskTexture.R
)

EmissiveRoughness = 0.2

→ Roughness

Emissive:
EmissiveColor
* EmissiveIntensity
* MaskTexture.R

EmissiveIntensity = 0.1

→ EmissiveColor

Normal:
T_Dark_Normal
+ T_Metal_Normal
→ Lerp(
    Alpha = MaskTexture.G
)

→ Lerp(
    A = PreviousNormal,
    B = Constant3Vector(0.5, 0.5, 1.0),
```

```

    Alpha = MaskTexture.R
)

→ FlattenNormal(
    Normal = LerpResult,
    Flatness = NormalFlatness
)

NormalFlatness = 0.95

→ Normal

MainMask:
MaskTexture
→ RGB Channel별로
    BaseColor 영역 분리
    Roughness 영역 분리
    Emissive 영역 분리
    Normal 영역 분리

```

## BP\_Stage

### SetStageEmissive

```

BP_Stage / SetStageEmissiveColor

Input:
StaticMesh = MeshComponent
Color = LinearColor

SetStageEmissiveColor:

StaticMesh
→ SetVectorParameterValueOnMaterials(
    ParameterName = "EmissiveBaseColor",
    ParameterValue = Color
)

→ SetVectorParameterValueOnMaterials(
    Target = StaticMesh,
    ParameterName = "EmissiveColor",
    ParameterValue = Color
)

```

### ChangeStageLight

```

BP_Stage / ChangeStageLight

Input:
NewParam = E_LightingState

ChangeStageLight:
SwitchOn E_LightingState(NewParam)

Neutral, Focus:
DefaultLightColor
→ SetStageEmissiveColor(
    StaticMesh = Floor,
    Color = DefaultLightColor
)
→ SetStageEmissiveColor(
    StaticMesh = Platform,
    Color = DefaultLightColor
)
→ SetStageEmissiveColor(
    StaticMesh = Wall,
    Color = DefaultLightColor
)
→ SetStageEmissiveColor(
    StaticMesh = Logo,
    Color = DefaultLightColor
)
→ SetStageEmissiveColor(
    StaticMesh = Ceiling,
    Color = DefaultLightColor
)

Stage:
StageLightColor
→ SetStageEmissiveColor(
    StaticMesh = Floor,
    Color = StageLightColor
)
→ SetStageEmissiveColor(
    StaticMesh = Platform,
    Color = StageLightColor
)
→ SetStageEmissiveColor(
    StaticMesh = Wall,
    Color = StageLightColor
)
→ SetStageEmissiveColor(
    StaticMesh = Logo,
    Color = StageLightColor
)

```

```

)
→ SetStageEmissiveColor(
    StaticMesh = Ceiling,
    Color = StageLightColor
)

```

## Character

### BP\_BountyHunter 역할

- **BP\_BountyHunter** : Virtual Performer의 실제 Character Actor
- Character 이동, Camera Tracking, Runtime Appearance, Morph Target, Character Lighting을 담당
- Animation Blueprint가 Animation 상태를 표현하는 쪽이라면 **BP\_BountyHunter** 는 **Character의 실제 위치, 회전, Camera 기준점, Material, Light 같은 Runtime 상태를 처리**
- **BeginPlay** 에서 필요한 Reference와 Runtime 데이터를 초기화한 뒤 시스템 사용 가능 상태로 전환

```

BP_BountyHunter
├─ Movement
│  ├─ 이동 전 Path 방향 회전
│  ├─ Navigation 이동
│  ├─ Landing 처리
│  ├─ 정확한 도착 위치 보정
│  └─ 도착 후 정면 회전
├─ Camera
│  ├─ Camera State 적용
│  ├─ DataTable 기반 Camera 설정
│  ├─ Camera Blend
│  ├─ Orbit Rotation
│  └─ Bone Tracking
├─ Appearance
│  ├─ Hair / Eyebrow
│  ├─ Eye
│  ├─ Lip
│  ├─ Skin
│  └─ Clothes
├─ Facial
│  ├─ Smile Morph Target
│  └─ Eye Blink
└─ Lighting
   ├─ Character Light 설정
   └─ Head Bone Tracking

```

### Runtime 초기화

- **BeginPlay** 에서 **BP\_StudioManager** Reference 획득
- Morph Target 초기값 적용
- Runtime Appearance 변경에 사용할 Dynamic Material Instance 생성 및 Cache
- Camera Tracking과 Light Tracking에 필요한 초기 Bone Offset 계산
- 초기화 완료 후 **IsRuntimeInitialized = true**
- **OnRuntimeInitialized** 호출로 외부 System에 Character 초기화 완료 전달

```

BeginPlay
→ StudioManager Reference 획득
→ Morph Target 초기화
→ Dynamic Material Instance Cache
→ Camera Tracking 기준값 계산
→ Light Tracking 기준값 계산
→ Runtime Initialized

```

### Camera State와 DataTable

- Camera State는 **Full**, **Upper**, **Close** 3종으로 구성
- Camera 값을 Blueprint 내부에 직접 하드코딩하지 않고 **DT\_CameraData** 에서 State별 설정 관리
- Camera State 하나에서 다음 정보를 함께 관리
  - **TargetArmLength**
  - **SocketOffsetZ**
  - **FieldOfView**
  - **Pitch**
  - **MinClamp**
  - **MaxClamp**
  - **FocusSocketName**

- Camera State 변경 시 같은 Data Row를 기준으로 **Camera 구도, Tracking Bone, Camera 회전 제한을 함께 변경**
- Camera 관련 값과 Tracking 기준을 따로 관리할 때 발생할 수 있는 불일치 감소

```
CurrentCameraState
→ DT_CameraData Row 조회
→ Camera Parameter 적용
→ Focus Socket 결정
→ Camera Blend
→ Bone Tracking
```

## Camera 전환

- Camera State 변경 순간 현재 Camera 값을 시작값으로 저장
- 목표값은 `DT_CameraData` 에서 조회
- Timeline과 `Lerp` 를 사용해 다음 항목을 부드럽게 전환
  - SpringArm Length
  - Socket Offset
  - FOV
  - Camera Pitch
- 순간적으로 Camera가 끊겨 이동하지 않고 기존 구도에서 새로운 구도로 자연스럽게 전환

## Camera Orbit

- Mouse Drag로 Camera를 Character 중심으로 회전 가능
- Camera State마다 허용해야 하는 Pitch 범위가 다르므로 `DT_CameraData` 의 `MinClamp`, `MaxClamp` 사용
- `Full`, `Upper`, `Close` 상태마다 서로 다른 회전 범위를 적용해 Character가 화면 밖으로 벗어나거나 부자연스러운 각도까지 회전하는 것을 제한

## Camera Bone Tracking

### 문제

- Character Actor 위치만 기준으로 Camera를 따라가게 하면 Animation에 따라 골반, 상체, 머리 위치가 변해 원하는 구도가 유지되지 않는 문제 발생
- Bone 위치를 Camera Pivot 위치로 그대로 사용하면 Character Mesh와 Pivot의 초기 위치 차이가 사라져 Camera가 좌우나 상하로 치우치는 문제 발생
- Character가 이동하거나 Mesh Transform이 달라질 때 World Space 기준값을 그대로 사용하면 Tracking 기준이 쉽게 어긋남

### 해결

- Camera State마다 추적할 Bone을 지정
  - `Full` → `pelvis`
  - `Upper` → `spine_04`
  - `Close` → `head`
- `BeginPlay` 에서 각 Bone과 `CameraTrackingPivot` 사이의 초기 Offset을 **Mesh Component Space** 기준으로 저장
- Runtime에서는 현재 Bone 위치에 저장한 Offset을 다시 적용
- 마지막에 Mesh Transform을 이용해 Component Space에서 World Space로 변환
- `VInterpTo` 를 사용해 Bone의 작은 Animation 변화에 Camera가 즉각적으로 흔들리지 않도록 보간

```
CurrentCameraState
→ FocusSocketName 조회
→ 현재 Socket 위치 확인
→ 초기 Tracking Offset 적용
→ Component Space 위치 계산
→ World Space 변환
→ VInterpTo
→ CameraTrackingPivot 갱신
```

## 기준 좌표계 통일

- Camera Tracking 계산 과정은 가능한 한 Mesh Component Space 안에서 처리
- Character 이동과 회전이 발생한 뒤 마지막에 World Space로 변환
- Actor 위치나 회전 변화와 Animation Bone 변화가 섞이면서 발생하는 Offset 문제 감소
- 초기 **Camera Composition**은 유지하면서 **Animation에 따른 Bone 변화만 추적하는 구조**

## Stage View

- `Stage View` : `Full`, `Upper`, `Close` 와 별개의 Camera Override
- Stage View를 활성화해도 `CurrentCameraState` 는 변경하지 않음
- Stage View 해제 시 마지막으로 선택되어 있던 Camera State로 복귀
- 별도 Camera State를 추가하지 않아 일반 Camera 상태 관리와 고정 Stage View 역할을 분리

- 기존 Camera Blend가 실행 중인 상태에서 Stage View가 실행될 경우 Timeline이 동시에 Camera 값을 수정하지 않도록 기존 Camera Blend 중단 처리
- Stage View 중에는 Mouse Look에 따른 Camera와 Character 구도가 어색해질 수 있어 Look Tracking 적용 조건에서 제외

## Character Movement

Character 이동은 단순 `SimpleMoveToLocation` 호출 한 번으로 처리하지 않고 다음 단계로 분리

```
Idle
→ TurningToPath
→ Moving
→ TurningToCamera
→ Idle
```

`MovementPhase`를 Character에서 관리하고 Animation Blueprint에서도 같은 값을 사용해 **Movement Logic**과 **Animation 상태**를 연결

### 1 실제 Navigation Path 방향 확인

- 단순히 `GoalLocation - ActorLocation` 방향을 보고 Character를 회전시키지 않음
- `FindPathToLocationSynchronously`로 실제 Navigation Path를 먼저 생성
- `PathPoints[1] - PathPoints[0]`을 사용해 Character가 실제로 처음 이동하게 될 방향 계산

```
현재 위치
→ Navigation Path 생성
→ PathPoints[0]
→ PathPoints[1]
→ 첫 이동 방향 계산
```

- 장애물이나 경사 때문에 실제 Path의 첫 방향이 목적지를 향한 직선 방향과 다를 수 있기 때문
- Character가 목적지 방향을 바라본 뒤 다른 방향으로 이동하기 시작하는 현상 방지

### 2 이동 전 회전

- 현재 Actor Rotation과 첫 Path 방향의 Rotation 차이 계산
- Timeline과 `LerpRotator`를 사용해 이동 방향으로 먼저 회전
- 작은 각도 차이에서는 불필요한 Turning을 실행하지 않도록  $3^\circ$  이하 회전 생략
- 회전 각도에 따라 Timeline 속도를 조정해 큰 각도와 작은 각도의 회전 시간이 동일하게 보이지 않도록 처리

```
TurnAngle
→ TurnAngle / 90
→ Timeline Play Rate
```

### Mesh 회전 방식에서 Actor 회전 방식으로 변경

- 초기에는 Character Mesh를 따로 회전시키는 방식도 시도
- Mesh가 Capsule 중심에서 Offset되어 있어 Mesh Rotation만 변경하면 Character가 제자리에서 회전하지 않고 원을 그리듯 움직이는 문제 발생
- 최종적으로 **Mesh가 아닌 Character Actor Rotation을 기준으로 통일**
- 이동 방향, Collision, Character Transform의 기준을 동일하게 유지

### 3 Navigation 이동

- 이동 전 회전 완료 후 `SimpleMoveToLocation` 실행
- `MovementPhase = Moving`
- Animation Blueprint는 `MovementPhase`를 확인해 Walk Animation으로 전환
- 이동 중에는 현재 Velocity 방향을 기준으로 `RInterpTo`를 사용해 Actor Rotation을 지속적으로 보정

```
TurningToPath
→ SimpleMoveToLocation
→ MovementPhase = Moving
→ Velocity 방향 확인
→ RInterpTo
→ Actor Rotation 보정
```

### 4 도착 판정

- Actor Origin이 아니라 Character의 실제 발 위치를 나타내는 `FootPivot` 기준으로 목적지까지 거리 계산
- `ArrivalTolerance` 이하가 되면 목적지에 도착한 것으로 판단

```
GoalPosition
- FootPivot WorldLocation
→ Distance
→ Distance < ArrivalTolerance
```

### 5 정확한 도착 위치 보정

- Navigation 이동은 Goal Position과 완전히 동일한 좌표에서 멈추는 것이 아니라 일정 거리 안에서 이동을 종료할 수 있음
- Camera와 Stage Position 연출에서는 Character가 미세하게 다른 위치에 서는 것도 눈에 띄기 때문에 별도 보정 단계 추가
- Navigation 도착 후 `MoveToExactGoal` 실행
- Timeline과 `LerpVector` 를 사용해 현재 위치에서 정확한 목표 위치로 이동
- Character Actor 위치는 Capsule 중심이므로 Goal의 바닥 위치에 Capsule Half Height를 더해 Z 위치 보정
- `Sweep = true` 를 사용해 보정 중 Collision을 완전히 무시하지 않도록 처리

```
Navigation 도착
→ 현재 위치 저장
→ Exact Goal 계산
→ Timeline
→ LerpVector
→ SetActorLocation
```

## 6 도착 후 정면 회전

- 이동 종료 후 Character가 마지막 이동 방향을 계속 바라보지 않도록 정면 방향으로 회전
- Timeline으로 부드럽게 회전
- 완료 후

```
MovementPhase = Idle
IsMoving = false
```

로 복귀

## 이동 중 목적지 변경

- 이동 중에도 새로운 Position이나 Click-to-Move 목적지를 지정 가능
- 기존 목적지를 끝까지 이동한 뒤 새 목적지로 가는 방식이 아니라 새로운 Goal을 기준으로 이동 과정 갱신
- Operator가 Character 위치를 Runtime에서 즉시 수정 가능

## Fixed Position과 Click-to-Move 공통화

- `Left`, `Center`, `Right` Position 이동과 Stage Floor Click-to-Move는 입력 방식만 다르게 구성
- 목적지가 결정된 이후에는 동일한 Character Movement System 사용

```
Fixed Position Button
├── Goal Position
└── Floor Click
    └── Goal Position

Goal Position
→ TurningToPath
→ Navigation
→ Exact Goal 보정
→ TurningToCamera
```

- Position 방식별 이동 로직을 따로 만들지 않고 목적지 입력 이후 흐름을 재사용

## Falling과 Landing 처리

- 이동 중 `CharacterMovement.IsFalling()` 을 확인
- 이전 Frame의 `WasFalling` 과 현재 Falling 상태를 비교해 착지 순간 감지
- Falling에서 지면 상태로 바뀌는 순간 Navigation Movement를 잠시 중단
- Landing Animation이 보일 수 있도록 `0.6초` 대기
- 이후 기존 Goal을 향한 이동 재개

```
Moving
→ Falling
→ Landing 감지
→ StopMovementImmediately
→ Landing Animation
→ 0.6초
→ MoveToGoal
```

- Navigation 이동 때문에 Landing Animation이 즉시 Walk로 덮이는 문제 방지

## Runtime Appearance Control

### 외부 Character Asset 구조 분석

- Bounty Hunter는 Hair, Face, Eye, Skin, Clothes 등이 여러 Material Slot과 서로 다른 Material Parameter로 구성

- 단일 Material의 Color 하나만 변경하는 방식으로 Appearance 구현 불가능
- Asset의 Material Slot과 Material Parameter를 확인해 Runtime에서 변경할 항목을 분류

## Dynamic Material Instance Cache

- Runtime에서 Appearance를 반복 변경할 때마다 Material Instance를 새로 생성하지 않도록 `BeginPlay` 에서 필요한 Dynamic Material Instance를 생성하고 Reference 저장
- 대상
  - Hair
  - Face
  - Goggles Lens
  - Shoulder Armor
  - Dress
  - Shoelaces
  - Coat
  - Left Eye
  - Right Eye
  - Upper Body
  - Lower Body
  - Skin

## Hair / Eyebrow

- Hair Material의 `Hair_Color` 변경
- Face Material의 `Eyebrows_Color` 도 함께 변경
- 하나의 Hair Appearance 명령으로 Hair와 Eyebrow를 같이 제어

## Eye

- Left Eye와 Right Eye Material의 `IrisColor`를 동일하게 변경

## Lip

- Face Material의 `Lips_Color01`, `Lips_Color02` 변경

## Skin

- `SkinHue_Global`, `SkinDepth_Global` 사용
- Skin, Upper Body, Lower Body Material에 동일한 값을 적용
- 몸의 일부만 다른 피부색으로 보이는 현상 방지

## Clothes

- Clothes는 하나의 Material이 아니라 여러 Material로 분리되어 있으므로 각 Parameter를 개별적으로 변경
- Goggles
- Shoulder Armor
- Dress
- Shoelaces
- Coat
- 각각의 Material 구조에 맞는 Hue / Depth Parameter 사용

⇒ 외부 Character Asset의 기존 Material 구조를 분석하고 여러 Material Slot과 Parameter를 하나의 Runtime Appearance 기능으로 통합

## Morph Target을 이용한 Character 표현

- Bounty Hunter Asset에 포함된 Morph Target을 Runtime Character 표현에 활용
- `mouthSmileLeft`, `mouthSmileRight` 등을 조정해 기본 표정 설정
- Eye Blink는 `EyeBlinkLeft`, `EyeBlinkRight` Morph Target 사용
- Timeline을 이용해 Blink 값을 변화시켜 눈 깜빡임 구현

```
Timeline
→ Blink Value
→ EyeBlinkLeft
→ EyeBlinkRight
```

- Animation Asset만 사용하는 것이 아니라 Character Mesh에서 제공하는 Facial Morph Target도 함께 활용

## Character Lighting

- Character에 `SpotLight1`, `SpotLight2` 를 배치
- `ApplyCharacterLighting` 을 통해 두 Light의 Intensity와 Color를 한 번에 변경
- Lighting State 변경 시 외부에서 전달받은 값을 Character Lighting에 적용

## Light Bone Tracking

- Spot Light가 단순히 Character Actor 위치를 바라보는 것이 아니라 실제 Animation 중 움직이는 Character의 머리를 추적하도록 구성
- `head` Socket을 Tracking 기준으로 사용
- `BeginPlay` 에서 `LightTrackingPivot` 과 `head` Socket 사이의 초기 Offset을 Component Space로 저장
- Runtime에서 현재 Head 위치에 Offset을 적용
- World Space로 변환한 뒤 `VInterpTo` 로 부드럽게 `LightTrackingPivot` 이동

```
Head Socket
→ 초기 Tracking Offset
→ Component Space
→ World Space
→ VInterpTo
→ LightTrackingPivot
```

- Character 이동과 Animation이 동시에 발생해도 Spotlight의 Tracking Target이 실제 머리 위치를 따라가도록 구성

## 구현 과정에서 확인한 개선점

- 현재 `BP_BountyHunter` 가 Movement, Camera, Appearance, Morph, Lighting까지 여러 Runtime 기능을 담당
- 첫 프로젝트에서는 Character와 직접 연결되는 기능을 빠르게 구현하고 통합하기 위해 Character Blueprint 중심으로 구성
- 기능이 확장되면서 `BP_BountyHunter` 의 책임이 커진 것을 확인
- 후속 구조에서는 다음 기능을 ActorComponent 또는 독립 System으로 분리하는 방향 고려
  - Movement
  - Camera Tracking
  - Appearance
  - Character Lighting

⇒ 현재 구현을 그대로 최종 구조라고 보기보다 **Runtime 기능 구현 후 책임 분리가 필요한 지점을 확인한 첫 구조**로 정리 가능

## BP\_BountyHunter

### EventGraph

```
BP_BountyHunter / EventGraph

BeginPlay:
EventBeginPlay
→ Sequence

Then0:
GetActorOfClass(
    ActorClass = BP_StudioManager
)
→ StudioManagerRef = ReturnValue
→ SetMorphTargets()
→ CacheMID()

Then1: CameraTracking 초기화

DefaultPivotLocationCS =
InverseTransformLocation(
    Transform = Mesh.GetWorldTransform(),
    Location = CameraTrackingPivot.GetWorldLocation()
)

FullTrackingOffsetCS =
Mesh.GetSocketTransform(
    SocketName = "pelvis",
    TransformSpace = RTS_Component
).Location
- DefaultPivotLocationCS

UpperTrackingOffsetCS =
Mesh.GetSocketTransform(
    SocketName = "spine_04",
    TransformSpace = RTS_Component
).Location
- DefaultPivotLocationCS

CloseTrackingOffsetCS =
Mesh.GetSocketTransform(
    SocketName = "head",
    TransformSpace = RTS_Component
).Location
- DefaultPivotLocationCS
```

```

Then2: LightTracking 초기화

DefaultLightPivotLocationCS =
InverseTransformLocation(
    Transform = Mesh.GetWorldTransform(),
    Location = LightTrackingPivot.GetWorldLocation()
)

LightTrackingOffsetCS =
Mesh.GetSocketTransform(
    SocketName = "head",
    TransformSpace = RTS_Component
).Location
- DefaultLightPivotLocationCS

IsRuntimeInitialized = true
→ Call OnRuntimeInitialized

Tick:
EventTick
→ UpdateCameraBoneTracking()
→ UpdateLightBoneTracking()
→ Branch(IsMoving)

True:
→ Branch(IsReachedGoal)

    True:
    IsMoving = false
    → TurningToCamera()

    False:
    → RotateBody()

    → Branch(
        !CharacterMovement.IsFalling()
        && WasFalling
    )

    True:
    CharacterMovement.StopMovementImmediately()

    → SetTimerByEvent(
        Time = 0.6,
        Event = MoveToGoal(),
        Looping = false
    )

    False:
    → Continue

    WasFalling = CharacterMovement.IsFalling()

False:
→ No further execution shown

```

## TimelineGraph

```

BP_BountyHunter / TimelineGraph

PlayBlinkAnimation:
PlayBlinkAnimation
→ Timeline.PlayFromStart()

Timeline.Update:
BlinkValue = Lerp(
    A = 0.0,
    B = 1.0,
    Alpha = Timeline.Alpha
)

Mesh.SetMorphTarget(
    MorphTargetName = "EyeBlinkLeft",
    Value = BlinkValue
)

→ Mesh.SetMorphTarget(
    MorphTargetName = "EyeBlinkRight",
    Value = BlinkValue
)

ApplyCameraState:
ApplyCameraState(CurrentCameraState)
→ Sequence

Then0: CameraState 적용

CurrentCameraState = InputCurrentCameraState

RowName =
EnumToString(CurrentCameraState)

GetDataTableRow(
    DataTable = DT_CameraData,
    RowName = RowName
)

```

```

→ TargetArmLength = Row.TargetArmLength
→ TargetSocketOffsetZ = Row.SocketOffsetZ
→ TargetFieldOfView = Row.FieldOfView
→ TargetPitch = Row.Pitch
→ TargetMinClamp = Row.MinClamp
→ TargetMaxClamp = Row.MaxClamp
→ TargetFocusSocket = Row.FocusSocketName

Then1: Camera 전환 시작값 저장

StartArmLength =
SpringArm.TargetArmLength

StartSocketOffsetZ =
SpringArm.SocketOffset.Z

StartFieldOfView =
Camera.FieldOfView

StartPitch =
Camera.RelativeRotation.Pitch

CurrentSpringArmPitch =
SpringArm.GetWorldRotation().Pitch

MinClampedPitch =
SelectFloat(
    A = TargetMinClamp,
    B = CurrentSpringArmPitch,
    PickA = TargetMinClamp > CurrentSpringArmPitch
)

TargetPitchClamp =
SelectFloat(
    A = TargetMaxClamp,
    B = MinClampedPitch,
    PickA = TargetMaxClamp < MinClampedPitch
)

StartPitchClamp =
SpringArm.GetWorldRotation().Pitch

Then2:
TL_CameraBlend.PlayFromStart()

TL_CameraBlend / Update:

SpringArm.TargetArmLength =
Lerp(
    A = StartArmLength,
    B = TargetArmLength,
    Alpha = TL_CameraBlend.Alpha
)

SpringArm.SocketOffset =
(
    X = 0.0,
    Y = 0.0,
    Z = Lerp(
        StartSocketOffsetZ,
        TargetSocketOffsetZ,
        TL_CameraBlend.Alpha
    )
)

Camera.FieldOfView =
Lerp(
    StartFieldOfView,
    TargetFieldOfView,
    TL_CameraBlend.Alpha
)

Camera.SetRelativeRotation(
    Roll = 0.0,
    Pitch = Lerp(
        StartPitch,
        TargetPitch,
        TL_CameraBlend.Alpha
    ),
    Yaw = 0.0
)

SpringArm.SetWorldRotation(
    Roll = SpringArm.GetWorldRotation().Roll,
    Pitch = Lerp(
        StartPitchClamp,
        TargetPitchClamp,
        TL_CameraBlend.Alpha
    ),
    Yaw = SpringArm.GetWorldRotation().Yaw
)

StopCameraBlend:
StopCameraBlend
→ TL_CameraBlend.Stop()

SetDefaultCameraRotation:
SetDefaultCameraRotation
→ CurrentCameraRotation =

```

```

SpringArm.GetWorldRotation()

→ TL_BackToDefault.PlayFromStart()

StopSetDefaultCameraRotation:
StopSetDefaultCameraRotation
→ TL_BackToDefault.Stop()

TL_BackToDefault / Update:

NewRotation =
LerpRotator(
    A = CurrentCameraRotation,
    B = DefaultCameraRotation,
    Alpha = TL_BackToDefault.Alpha,
    ShortestPath = true
)

SpringArm.SetWorldRotation(
    Roll = 0.0,
    Pitch = NewRotation.Pitch,
    Yaw = NewRotation.Yaw
)

```

## MoveGraph

```

BP_BountyHunter / MoveGraph

TurningToMove:
TurningToMove
→ Sequence

Then0:
Branch(!IsReachedGoal)

True:
IsMoving = true

StartRotation =
GetActorRotation(self)

TargetRotation =
MakeRotator(
    Roll = 0.0,
    Pitch = 0.0,
    Yaw = MakeRotFromX(
        GetMoveStartDirection()
    ).Yaw
)

MovementPhase = TurningToPath
→ Call OnChangeMovePhase

Then1:
Branch(
    MovementPhase == TurningToPath
)

True:
TurnAngle =
Abs(
    DeltaRotator(
        A = TargetRotation,
        B = StartRotation
    ).Yaw
)

→ Branch(TurnAngle >= 3.0)

True:
TL_TurningToMove.PlayRate =
TurnAngle / 90.0

→ TL_TurningToMove.PlayFromStart()

False:
→ MoveToGoal()

TL_TurningToMove.Update:
NewRotation =
LerpRotator(
    A = StartRotation,
    B = TargetRotation,
    Alpha = TL_TurningToMove.Alpha,
    ShortestPath = true
)

→ SetActorRotation(
    Target = self,
    NewRotation = NewRotation
)

TL_TurningToMove.Finished:
→ MoveToGoal()

```

```

StopTurningToMove:
→ TL_TurningToMove.Stop()

MoveToGoal:
MoveToGoal
→ StopTurningToMove()
→ StopTurningToCamera()
→ StopMoveToExactGoal()

IsMoving = true

MovementPhase = Moving
→ Call OnChangeMovePhase

→ SimpleMoveToLocation(
    Controller = GetPlayerController(0),
    Goal = GoalPosition.Location
)

TurningToCamera:
TurningToCamera
→ Sequence

Then0:
StartRotation =
GetActorRotation(self)

TargetRotation =
Rotator(
    Roll = 0.0,
    Pitch = 0.0,
    Yaw = 0.0
)

MovementPhase = TurningToCamera

StartLocation =
GetActorLocation(self)

Then1:
→ MoveToExactGoal()

TurnAngle =
Abs(
    DeltaRotator(
        A = TargetRotation,
        B = StartRotation
    ).Yaw
)

→ Branch(TurnAngle >= 3.0)

True:
→ Call OnChangeMovePhase

TL_TurningToCamera.PlayRate =
TurnAngle / 90.0

→ TL_TurningToCamera.PlayFromStart()

False:
→ MovementPhase = Idle
→ IsMoving = false
→ Call OnChangeMovePhase

TL_TurningToCamera.Update:
NewRotation =
LerpRotator(
    A = StartRotation,
    B = TargetRotation,
    Alpha = TL_TurningToCamera.Alpha,
    ShortestPath = true
)

→ SetActorRotation(
    Target = self,
    NewRotation = NewRotation
)

TL_TurningToCamera.Finished:
MovementPhase = Idle
→ IsMoving = false
→ Call OnChangeMovePhase

StopTurningToCamera:
→ TL_TurningToCamera.Stop()

MoveToExactGoal:
MoveToExactGoal
→ TL_MoveToExactGoal.PlayFromStart()

StopMoveToExactGoal:

```

```

→ TL_MoveToExactGoal.Stop()

TL_MoveToExactGoal.Update:

ExactGoalLocation = (
    X = GoalPosition.Location.X,
    Y = GoalPosition.Location.Y,
    Z = GoalPosition.Location.Z
    + CapsuleComponent.GetScaledCapsuleHalfHeight()
)

NewLocation =
LerpVector(
    A = StartLocation,
    B = ExactGoalLocation,
    Alpha = TL_MoveToExactGoal.Alpha
)

→ SetActorLocation(
    Target = self,
    NewLocation = NewLocation,
    Sweep = true,
    Teleport = false
)

```

## MakeSmile

```

BP_BountyHunter / MakeSmile

MakeSmile:
→ Sequence

Then0:
Mesh.SetMorphTarget(
    MorphTargetName = "mouthSmileLeft",
    Value = 0.4
)

→ Mesh.SetMorphTarget(
    MorphTargetName = "mouthSmileRight",
    Value = 0.4
)

→ Mesh.SetMorphTarget(
    MorphTargetName = "breasts",
    Value = -1.0
)

→ Mesh.SetMorphTarget(
    MorphTargetName = "jawForward",
    Value = 0.249097
)

Then1:
Mesh.SetMorphTarget(
    MorphTargetName = "mouthFunnel",
    Value = -0.106115
)

→ Mesh.SetMorphTarget(
    MorphTargetName = "mouthPucker",
    Value = -0.255929
)

→ Mesh.SetMorphTarget(
    MorphTargetName = "mouthRollUpper",
    Value = 0.109863
)

→ Mesh.SetMorphTarget(
    MorphTargetName = "mouthRollLower",
    Value = 0.096129
)

```

## SetMorphTargets

```

BP_BountyHunter / SetMorphTargets

SetMorphTargets:
→ PlayBlinkAnimation()
→ MakeSmile()

```

## Move

```

BP_BountyHunter / Move

Input:
IsForward = Boolean
AxisValue = Float

Move:
Branch(IsForward)

```

```

True:
YawOnlyRotation =
GetYawOnlyRotation(self)

WorldDirection =
GetForwardVector(YawOnlyRotation)

→ AddMovementInput(
    Target = self,
    WorldDirection = WorldDirection,
    ScaleValue = AxisValue
)

False:
YawOnlyRotation =
GetYawOnlyRotation(self)

WorldDirection =
GetRightVector(YawOnlyRotation)

→ AddMovementInput(
    Target = self,
    WorldDirection = WorldDirection,
    ScaleValue = AxisValue
)

```

## SetColor

```

BP_BountyHunter / SetHairColor

Input:
HairColor = LinearColor
EyebrowColor = LinearColor

SetHairColor:
MID_Hair.SetVectorParameterValue(
    ParameterName = Hair_Color,
    Value = HairColor
)

→ MID_Face.SetVectorParameterValue(
    ParameterName = Eyebrows_Color,
    Value = EyebrowColor
)

BP_BountyHunter / SetEyeColor

Input:
EyeColor = LinearColor

SetEyeColor:
MID_Eye_L.SetVectorParameterValue(
    ParameterName = IrisColor,
    Value = EyeColor
)

→ MID_Eye_R.SetVectorParameterValue(
    ParameterName = IrisColor,
    Value = EyeColor
)

BP_BountyHunter / SetLipColor

Input:
LipColor01 = LinearColor
LipColor02 = LinearColor

SetLipColor:
MID_Face.SetVectorParameterValue(
    ParameterName = Lips_Color01,
    Value = LipColor01
)

→ MID_Face.SetVectorParameterValue(
    ParameterName = Lips_Color02,
    Value = LipColor02
)

BP_BountyHunter / SetSkinColor

Input:
SkinHue = Float
SkinDepth = Float

SetSkinColor:
Sequence

Then0:
MID_Skin.SetScalarParameterValue(
    ParameterName = SkinHue_Global,
    Value = SkinHue
)

→ MID_UpperBody.SetScalarParameterValue(
    ParameterName = SkinHue_Global,
    Value = SkinHue
)

```

```

)

→ MID_LowerBody.SetScalarParameterValue(
    ParameterName = SkinHue_Global,
    Value = SkinHue
)

Then1:
MID_Skin.SetScalarParameterValue(
    ParameterName = SkinDepth_Global,
    Value = SkinDepth
)

→ MID_UpperBody.SetScalarParameterValue(
    ParameterName = SkinDepth_Global,
    Value = SkinDepth
)

→ MID_LowerBody.SetScalarParameterValue(
    ParameterName = SkinDepth_Global,
    Value = SkinDepth
)

BP_BountyHunter / SetClothesColor

Input:
GogglesHue = Float
ArmorHue = Float
DressHue = Float
DressDepth = Float
ShoelacesHue = Float
CoatHue = Float
CoatDepth = Float

SetClothesColor:
MID_GogglesLens.SetScalarParameterValue(
    ParameterName = ColorHue_Global,
    Value = GogglesHue
)

→ MID_ShoulderArmor.SetScalarParameterValue(
    ParameterName = Color02_Hue,
    Value = ArmorHue
)

→ MID_Dress.SetScalarParameterValue(
    ParameterName = Color02_Hue,
    Value = DressHue
)

→ MID_Dress.SetScalarParameterValue(
    ParameterName = Color02_Depth,
    Value = DressDepth
)

→ MID_Shoelaces.SetScalarParameterValue(
    ParameterName = ColorHue_Global,
    Value = ShoelacesHue
)

→ MID_Coat.SetScalarParameterValue(
    ParameterName = Color02_Hue,
    Value = CoatHue
)

→ MID_Coat.SetScalarParameterValue(
    ParameterName = Color02_Depth,
    Value = CoatDepth
)

```

## CachMID

```

BP_BountyHunter / CachMID

CachMID:
Sequence

Then0:
MID_Hair = Mesh.CreateDynamicMaterialInstance(
    ElementIndex = 11
)

→ MID_Face = Mesh.CreateDynamicMaterialInstance(
    ElementIndex = 0
)

→ MID_GogglesLens = Mesh.CreateDynamicMaterialInstance(
    ElementIndex = 6
)

Then1:
MID_ShoulderArmor = Mesh.CreateDynamicMaterialInstance(
    ElementIndex = 12
)

→ MID_Dress = Mesh.CreateDynamicMaterialInstance(
    ElementIndex = 15
)

→ MID_Shoelaces = Mesh.CreateDynamicMaterialInstance(

```

```

        ElementIndex = 24
    )

    Then2:
    MID_Coat = Mesh.CreateDynamicMaterialInstance(
        ElementIndex = 29
    )

    → MID_EyeL = Mesh.CreateDynamicMaterialInstance(
        ElementIndex = 2
    )

    → MID_EyeR = Mesh.CreateDynamicMaterialInstance(
        ElementIndex = 3
    )

    Then3:
    MID_UpperBody = Mesh.CreateDynamicMaterialInstance(
        ElementIndex = 5
    )

    → MID_LowerBody = Mesh.CreateDynamicMaterialInstance(
        ElementIndex = 27
    )

    → MID_Skin = Mesh.CreateDynamicMaterialInstance(
        ElementIndex = 0
    )

```

## UpdateCameraBoneTracking

```

BP_BountyHunter / UpdateCameraBoneTracking

UpdateCameraBoneTracking:
→ Sequence

Then0: Target Pivot 계산

RowName =
EnumToString(CurrentCameraState)

GetDataTableRow(
    DataTable = DT_CameraData,
    RowName = RowName
)

FocusSocketName =
Break ST_CameraData.FocusSocketName

SocketLocationCS =
Mesh.GetSocketTransform(
    SocketName = FocusSocketName,
    TransformSpace = RTS_Component
).Location

TrackingOffsetCS =
Select(CurrentCameraState)
    Full = FullTrackingOffsetCS
    Upper = UpperTrackingOffsetCS
    Close = CloseTrackingOffsetCS

TargetPivotLocationCS =
SocketLocationCS + TrackingOffsetCS

Then1: Component Space → World Space 변환 / 보간

TargetPivotWorldLocation =
TransformLocation(
    T = Mesh.GetWorldTransform(),
    Location = TargetPivotLocationCS
)

CurrentPivotWorldLocation =
CameraTrackingPivot.GetWorldLocation()

NewPivotWorldLocation =
VInterpTo(
    Current = CurrentPivotWorldLocation,
    Target = TargetPivotWorldLocation,
    DeltaTime = GetWorldDeltaSeconds(),
    InterpSpeed = 10.0
)

→ CameraTrackingPivot.SetWorldLocation(
    NewLocation = NewPivotWorldLocation,
    Sweep = false,
    Teleport = false
)

```

## GetMoveStartDirection

```

BP_BountyHunter / GetMoveStartDirection

GetMoveStartDirection:

```

```

StartLocation =
GetActorLocation(self)

Path =
FindPathToLocationSynchronously(
    PathStart = StartLocation,
    PathEnd = GoalPosition.Location
)

PathPoints =
Path.PathPoints

MoveStartDirection =
PathPoints[1] - PathPoints[0]

→ Return MoveStartDirection

```

## GetCameraDirection

```

BP_BountyHunter / GetCameraDirection

GetCameraDirection:

CameraWorldLocation =
Camera.GetWorldLocation()

ActorWorldLocation =
GetActorLocation(self)

CameraDirection =
CameraWorldLocation - ActorWorldLocation

→ Return CameraDirection

```

## IsReachedGoal

```

BP_BountyHunter / IsReachedGoal

IsReachedGoal:

DistanceToGoal =
VectorLength(
    GoalPosition.Location
    - FootPivot.GetWorldLocation()
)

IsReachedGoal =
DistanceToGoal < ArrivalTolerance

→ Return IsReachedGoal

```

## RotateBody

```

BP_BountyHunter / RotateBody

RotateBody:

Velocity =
GetVelocity(self)

SpeedXY =
VectorLengthXY(Velocity)

→ Branch(SpeedXY > 1.0)

True:
CurrentRotation =
GetActorRotation(self)

TargetRotation =
RotationFromXVector(
    InVec = Velocity
)

InterpolatedRotation =
RInterpTo(
    Current = CurrentRotation,
    Target = TargetRotation,
    DeltaTime = GetWorldDeltaSeconds(),
    InterpSpeed = 10.0
)

→ SetActorRotation(
    Target = self,
    Roll = 0.0,
    Pitch = 0.0,
    Yaw = InterpolatedRotation.Yaw,
    TeleportPhysics = false
)

False:
→ No further execution

```

## RotateCamera

```
BP_BountyHunter / RotateCamera

Input:
ActionValueX = Float
ActionValueY = Float

RotateCamera:
→ Sequence

Then0: Orbit Rotation 갱신

OrbitRoll =
OrbitRotation.Roll

OrbitPitch =
OrbitRotation.Pitch
+ (ActionValueY * CameraRotateSpeed)

OrbitYaw =
OrbitRotation.Yaw
+ (ActionValueX * CameraRotateSpeed)

Then1: CameraState 기준 Pitch Clamp 적용

CurrentCameraState =
StudioManagerRef.CurrentCameraState

RowName =
EnumToString(CurrentCameraState)

GetDataTableRow(
    DataTable = DT_CameraData,
    RowName = RowName
)

Break ST_CameraData
→ MinClamp
→ MaxClamp

ClampedPitch =
Clamp(
    Value = OrbitPitch,
    Min = MinClamp,
    Max = MaxClamp
)

OrbitRotation =
MakeRotator(
    Roll = OrbitRoll,
    Pitch = ClampedPitch,
    Yaw = OrbitYaw
)

→ SpringArm.SetWorldRotation(
    NewRotation = OrbitRotation
)
```

## UpdateLightBoneTracking

```
BP_BountyHunter / UpdateLightBoneTracking

UpdateLightBoneTracking:
→ Sequence

Then0: Light Tracking Target 계산

HeadSocketLocationCS =
Mesh.GetSocketTransform(
    SocketName = "head",
    TransformSpace = RTS_Component
).Location

TargetLightPivotLocationCS =
HeadSocketLocationCS
+ LightTrackingOffsetCS

Then1: Component Space → World Space 변환 / 보간

TargetLightPivotWorldLocation =
TransformLocation(
    T = Mesh.GetWorldTransform(),
    Location = TargetLightPivotLocationCS
)

CurrentLightPivotWorldLocation =
LightTrackingPivot.GetWorldLocation()

NewLightPivotWorldLocation =
VInterpTo(
    Current = CurrentLightPivotWorldLocation,
    Target = TargetLightPivotWorldLocation,
    DeltaTime = GetWorldDeltaSeconds(),
    InterpSpeed = 10.0
)
```

```

→ LightTrackingPivot.SetWorldLocation(
    NewLocation = NewLightPivotWorldLocation,
    Sweep = false,
    Teleport = false
)

```

## ApplyCharacterLighting

```

BP_BountyHunter / ApplyCharacterLighting

Input:
Intensity1 = Float
Intensity2 = Float
Color = LinearColor

ApplyCharacterLighting:

SpotLight1.SetIntensity(
    NewIntensity = Intensity1
)

→ SpotLight2.SetIntensity(
    NewIntensity = Intensity2
)

→ SpotLight1.SetLightColor(
    NewLightColor = Color,
    SRGB = true
)

→ SpotLight2.SetLightColor(
    NewLightColor = Color,
    SRGB = true
)

```

## Controller

### BP\_PlayerController 역할

- **BP\_PlayerController** : Operator의 Mouse / Camera / Click 입력을 받아 Runtime System에 전달하는 입력 제어 Actor
- Character를 직접 이동시키거나 Animation을 직접 재생하기보다 **입력 해석과 요청 생성**에 집중하도록 구성
- 주요 역할
  - Enhanced Input Mapping Context 등록
  - Click-to-Move 입력 처리
  - Mouse 기반 Look Tracking용 Yaw / Pitch 계산
  - Camera Drag Rotation 입력 처리
  - 이동 가능 여부 판단
- Character의 실제 Navigation 이동, 회전, 도착 처리는 **BP\_BountyHunter** 에서 수행하도록 분리했다

```

BP_PlayerController
├─ Enhanced Input
├─ Click-to-Move Input
├─ Mouse Look 계산
├─ Camera Rotation Input
└─ 이동 가능 여부 판단

BP_BountyHunter
├─ Navigation 이동
├─ 이동 전 / 후 회전
├─ 정확한 위치 보정
└─ Camera 실제 Transform 처리

```

## Enhanced Input 초기화

- **BeginPlay** 에서 현재 Player Pawn을 가져와 **BP\_BountyHunter** Reference를 저장했다
- **EnhancedInputLocalPlayerSubsystem** 이 유효한지 확인한 뒤 **IMC\_Default** 를 등록했다
- Mapping Context가 등록되지 않은 상태에서는 **IA\_ClickToMove** , Camera Rotate Input 등이 들어오지 않았기 때문에 시작 시 명시적으로 등록하도록 했다
- **BP\_StudioManager** Reference도 초기화해 CharacterState, Stage View, Gesture 상태를 확인할 수 있도록 했다

```

BeginPlay
→ BP_BountyHunter Reference 획득
→ EnhancedInputLocalPlayerSubsystem 확인
→ IMC_Default 등록
→ BP_StudioManager Reference 획득

```

## Click-to-Move

- Mouse Click 한 번으로 목적지를 지정하는 방식으로 구성했다
- Mouse를 누르고 있는 동안 계속 이동 목표를 갱신하는 방식은 사용하지 않았다
- `IA_ClickToMove`가 Trigger되면 먼저 Click 이동이 가능한 상태인지 확인한 뒤 `RequestViewportClick`을 호출했다

```
IA_ClickToMove
→ CanClickMove 확인
→ Camera Rotate 중인지 확인
→ RequestViewportClick
```

- Camera를 Drag해서 회전 중일 때 Mouse Click이 이동 명령으로 동시에 처리되지 않도록 `!CanCameraRotate` 조건을 추가했다
- 즉 Mouse 입력 하나가 Camera Rotate와 Character Move 두 기능에 동시에 사용되지 않도록 상태를 분리했다

## Click 위치 검증

`RequestViewportClick`에서는 단순히 Mouse 아래 World 위치를 바로 목적지로 사용하지 않고 여러 단계로 검증했다

```
Mouse Click
→ Hit Result Under Cursor
→ Blocking Hit 확인
→ 바닥 방향 Surface인지 확인
→ Character 이동 가능 상태 확인
→ NavMesh Projection
→ 실제 Ground Trace
→ GoalPosition 설정
→ Cursor FX 출력
→ 이동 요청
```

### 1. World Hit 확인

- `GetHitResultUnderCursorByChannel`을 사용해 Mouse Cursor 아래의 World Hit Result를 구했다
- `Visibility` Trace Channel을 사용하고 `TraceComplex = true`로 설정했다

### 2. 바닥 Surface 판정

- 벽이나 수직 구조물을 클릭했을 때 Character가 그 위치로 이동하려고 하지 않도록 `ImpactNormal.Z` 값을 확인했다
- `SurfaceNormalZ > 0.5` 조건을 통과한 Surface만 이동 후보로 사용했다

```
BlockingHit
&& SurfaceNormalZ > 0.5
```

- Floor 계열의 위쪽을 바라보는 Surface만 통과시키기 위한 처리

### 3. 이동 가능 상태 확인

- Character가 언제든지 이동할 수 있게 하지 않고 현재 Runtime State를 확인했다
- `Talk`, `Dance`, Gesture 실행 중에는 Click 이동을 막았다

```
CurrentCharacterState != Talk
&& CurrentCharacterState != Dance
&& !IsGesturing
```

- Presentation이나 Performance 중 Character가 이동하면서 Animation State가 충돌하는 것을 방지했다

## Navigation 가능 위치 확인

- Mouse Hit 위치를 바로 Goal로 사용하지 않고 `ProjectPointToNavigation`으로 NavMesh 위의 유효한 위치인지 확인했다
- Navigation Projection이 실패하면 이동하지 않도록 했다

```
HitLocation
→ ProjectPointToNavigation
→ 성공 여부 확인
```

- World에 존재하는 위치라고 해서 항상 Character가 이동 가능한 위치는 아니기 때문에 Navigation 가능 영역과 실제 Click 위치를 분리했다

## Ground Position 보정

- `ProjectPointToNavigation` 결과는 Navigation 기준 위치이기 때문에 실제 Stage Surface 위치와 차이가 생길 수 있었다
- Projection된 위치에서 아래 방향으로 `LineTraceForObjects`를 실행해 실제 `WorldStatic` 바닥의 Impact Point를 다시 구했다

```
ProjectedLocation
→ 아래 방향 Line Trace
→ WorldStatic Hit
→ GroundHit.ImpactPoint
→ GoalPosition
```

- 최종적으로 Character가 실제 Stage 바닥 위에 위치하도록 Goal Position을 보정했다

## Click Feedback

- 정상적인 Goal Position이 결정되면 해당 위치에 **FXCursor** Niagara System을 Spawn했다
- Operator가 어느 위치를 이동 대상으로 선택했는지 즉시 확인할 수 있도록 시각적 Feedback을 추가했다

```
GoalPosition
→ SpawnSystemAtLocation(FXCursor)
→ 이동 요청
```

- 이동 실패 가능성이 있는 Click에는 Cursor FX가 생성되지 않도록 유효한 Goal Position이 확정된 이후 실행했다

## Click-to-Move와 Position 이동 공통화

- Click-to-Move와 **Left / Center / Right** Position Button은 입력 방식만 다르게 구성했다
- 두 기능 모두 최종적으로 Character의 **GoalPosition**을 정한 뒤 동일한 Movement System을 사용한다
- PlayerController에서 별도의 **SimpleMoveToLocation**을 실행하지 않고 실제 이동은 Character 쪽 Movement Logic에 맡겼다

```
Position Button
├── GoalPosition
└── Mouse Click
    └── GoalPosition

GoalPosition
→ Character Movement System
→ TurningToPath
→ Navigation
→ Exact Goal
→ TurningToCamera
```

- 고정 Position 이동과 자유 Click 이동을 별도의 이동 시스템으로 중복 구현하지 않도록 했다

## Click-to-Move 구현 중 입력 문제

### Enhanced Input Mapping Context 누락

- Click Event가 전혀 들어오지 않는 문제가 발생했다
- 원인은 **IA\_ClickToMove** 자체가 아니라 **IMC\_Default**가 Local Player Subsystem에 등록되지 않은 상태였다
- **BeginPlay**에서 Mapping Context를 등록한 뒤 입력이 정상적으로 들어왔다

```
EnhancedInputLocalPlayerSubsystem
→ AddMappingContext(IMC_Default)
```

- Enhanced Input에서는 Input Action을 만들어두는 것만으로는 동작하지 않고 해당 Mapping Context가 활성화되어 있어야 한다는 점을 확인했다

## UI와 World Click 분리

- Operator UI가 화면 위를 덮고 있기 때문에 모든 Mouse Click을 World 이동으로 처리하면 UI Button을 누를 때 Character도 이동하는 문제가 발생할 수 있었다
- Click-to-Move는 World 이동용 입력이므로 UI 조작과 충돌하지 않도록 분리했다
- 실제 이동 요청 전에 Click 이동이 허용된 영역인지와 Runtime 상태를 확인하도록 구성했다
- 중앙 Preview / Stage 영역에서의 Click은 World 이동으로 사용하고 UI 조작 영역의 Click은 이동 입력으로 처리하지 않는 방향으로 설계했다
- 구현 과정에서 단순 Hover 판정만으로 UI와 World Click을 나누는 방식은 중앙 Preview Image와 다른 Widget 구조 때문에 안정적이지 않아, 입력 허용 상태를 별도로 관리하는 방식으로 수정했다

## Viewport 좌표 문제

- Click-to-Move 구현 과정에서 Mouse가 가리키는 위치와 실제 이동 위치가 어긋나는 문제가 있었다
- 원인은 일반 Game Viewport 기준 Mouse Hit Result와 실제 Operator UI 안에 표시되는 SceneCapture Preview 영역의 좌표 기준이 같지 않았기 때문이었다
- 화면 전체 기준 Cursor 좌표와 Preview Image 내부의 좌표를 구분할 필요가 있다는 점을 확인했다
- 최종 구조에서는 Operator UI / Viewport 구성에 맞게 Click 허용 영역과 Goal 계산을 조정했다

## Look Tracking

### Mouse 위치를 AimOffset 값으로 변환

- Character가 Mouse Cursor 방향을 바라볼 수 있도록 PlayerController에서 Mouse 위치를 Yaw / Pitch 값으로 변환했다
- 매 Frame **GetMousePosition**과 **GetViewportSize**를 사용해 현재 Mouse가 화면 중심에서 얼마나 떨어져 있는지 계산했다

```
Viewport Center
- Mouse Position
```

```
→ 화면 중심 기준 Offset  
→ Yaw / Pitch
```

## Yaw

```
((ViewportSizeX / 2) - MouseLocationX)  
× ((MaxYaw × -2) / ViewportSizeX)
```

## Pitch

```
((ViewportSizeY / 2) - MouseLocationY)  
× ((MaxPitch × 2) / ViewportSizeY)
```

- 화면 중앙은 Yaw / Pitch가 0
- 왼쪽 / 오른쪽, 위 / 아래로 Mouse가 이동할수록 설정된 Max 값까지 변화하도록 구성했다
- PlayerController는 Look 방향에 필요한 목표값만 계산하고 실제 Aim Offset 적용과 보간은 Animation Blueprint에서 처리했다

## Look Tracking 활성화 조건

- Mouse Look은 항상 강제로 적용하지 않고 Operator가 LookTrackingActive 를 컨 경우에만 동작하도록 했다
- StageViewActive 상태에서는 일반 Character Camera 기준 Mouse Tracking이 어색해지기 때문에 Tracking을 중지했다

```
LookTrackingActive  
&& !StageViewActive
```

- 조건을 만족하지 않으면

```
AimOffsetYaw = 0  
AimOffsetPitch = 0
```

으로 설정해 Character가 Neutral 방향으로 돌아갈 수 있도록 했다

- PlayerController에서는 목표 Yaw / Pitch를 0으로 만들고, Animation Blueprint의 FInterpTo 가 실제 Pose를 부드럽게 Neutral로 복귀시키는 구조

## Look Tracking과 Animation 역할 분리

- PlayerController
  - Mouse 위치 확인
  - Viewport 크기 확인
  - Yaw / Pitch 계산
- Animation Blueprint
  - Yaw / Pitch 보간
  - Camera State별 Pitch Offset 적용
  - Aim Offset Blend
  - 최종 Character Pose 적용

```
Mouse Position  
→ BP_PlayerController  
→ AimOffsetYaw / Pitch  
→ ABP_BountyHunter  
→ FInterpTo  
→ Camera State 보정  
→ Aim Offset
```

- 입력 계산과 Animation 표현을 분리해서 PlayerController가 Character Bone이나 Animation Asset을 직접 제어하지 않도록 했다

## Camera Rotation Input

- Mouse Drag 중에는 Look Tracking과 별개로 Camera를 Character 중심으로 회전할 수 있도록 했다
- IA\_MakeCanCameraRotate 를 이용해 Camera Rotate Mode의 시작과 종료를 관리했다

## Camera Rotate 시작

```
IA_MakeCanCameraRotate Started  
→ CanCameraRotate = true  
→ 기존 Default 복귀 Timeline 중지  
→ 현재 SpringArm Rotation 저장  
→ OrbitRotation 초기화  
→ OnCameraRotateStateChange(true)
```

- Camera가 Default 위치로 복귀 중일 때 다시 Drag를 시작하면 두 Rotation 로직이 동시에 SpringArm을 수정할 수 있기 때문에 `StopSetDefaultCameraRotation()` 을 먼저 호출했다
- 현재 Camera Rotation을 Orbit 시작값으로 저장해 Drag 시작 순간 Camera가 튀지 않도록 했다

## Camera Rotate 중

- `IA_RotateCamera` 값을 `BP_BountyHunter.RotateCamera` 로 전달했다
- 실제 SpringArm Rotation 변경과 Camera State별 Pitch Clamp는 Character 쪽에서 처리했다
- `StageViewActive` 상태에서는 Camera Drag를 막았다

```
CanCameraRotate
&& !StageViewActive
→ RotateCamera
```

## Camera Rotate 종료

```
IA_MakeCanCameraRotate Completed
→ CanCameraRotate = false
→ SetDefaultCameraRotation
→ OnCameraRotateStateChange(false)
```

- Drag가 끝나면 현재 Camera를 그대로 멈춰두지 않고 기본 Camera Rotation으로 부드럽게 복귀하도록 했다

## Camera Rotate와 Click-to-Move 충돌 방지

- Camera Drag와 Click-to-Move 모두 Mouse 입력을 사용하기 때문에 두 기능이 동시에 실행되지 않도록 상태값으로 분리했다

```
Click-to-Move
→ !CanCameraRotate일 때만 허용
```

```
Camera Rotation
→ CanCameraRotate일 때만 허용
```

- Camera를 회전하려고 Mouse를 조작했는데 Character 이동까지 발생하는 입력 충돌을 방지했다

## Stage View와 Camera Input

- Stage View는 일반 Character Camera가 아니라 고정 Stage View Override이므로 Character Orbit Camera 입력을 사용하지 않도록 했다
- Stage View가 활성화된 동안
  - Camera Drag Rotation X
  - 일반 Look Tracking 보정 X
- Stage View 종료 후 다시 기존 Camera State의 입력으로 돌아오도록 했다

```
StageViewActive
→ Character Camera Rotate 차단
→ Mouse Look 차단
```

```
StageView OFF
→ 기존 Camera State 입력 복구
```

## Controller 구조 정리

```
BP_PlayerController
├── Enhanced Input
│   └── IMC_Default
├── Click-to-Move
│   ├── Mouse Hit Test
│   ├── Surface Normal 검사
│   ├── CanMove
│   ├── NavMesh Projection
│   ├── Ground Trace
│   └── GoalPosition 전달
├── Look Tracking
│   ├── Mouse Position
│   ├── Viewport Center 기준 계산
│   └── AimOffsetYaw / Pitch
└── Camera Rotation
    ├── Rotate Mode
    ├── Drag Input
    ├── Stage View 제한
    └── Default Camera 복귀
```

## 구현하면서 정리된 역할 분리

```

BP_PlayerController
→ Input을 해석하고 요청 생성

BP_StudioManager
→ Runtime State와 명령 기능 여부 관리

BP_BountyHunter
→ 실제 Character Movement / Camera 동작

ABP_BountyHunter
→ 최종 Animation 표현

```

- PlayerController에서 직접 SimpleMoveToLocation, CharacterState 변경, Animation 재생까지 모두 처리하는 구조를 피했다
- 입력과 실제 Runtime 동작의 책임을 나눠 기능이 추가되어도 PlayerController가 지나치게 많은 Game Logic을 가지지 않도록 했다

## BP\_PlayerController

### EventGraph

```

BP_PlayerController / EventGraph

BeginPlay:
Event BeginPlay
→ GetPlayerPawn(PlayerIndex = 0)
→ CastToBP_BountyHunter
→ BountyHunterRef = CastResult

→ IsValid(EnhancedInputLocalPlayerSubsystem)

Valid:
EnhancedInputLocalPlayerSubsystem.AddMappingContext(
    MappingContext = IMC_Default,
    Priority = 0
)

→ StudioManagerRef =
GetActorOfClass(
    ActorClass = BP_StudioManager
)

ClickMove:
EnhancedInputAction IA_ClickToMove
Triggered
→ Branch(
    CanClickMove
    && !CanCameraRotate
)

True:
→ RequestViewportClick(self)

LookTracking:
Event Tick
→ Branch(
    LookTrackingActive
    && !StudioManagerRef.StageViewActive
)

False:
AimOffsetYaw = 0.0
→ AimOffsetPitch = 0.0

True:
→ Sequence

Then0:
GetMousePosition(
    Target = self
)

MouseLocationX = LocationX
MouseLocationY = LocationY

GetViewportSize()
ViewportSizeX = ReturnValueX
ViewportSizeY = ReturnValueY

Then1:
AimOffsetYaw =
(
    (ViewportSizeX / 2.0) - MouseLocationX
)
*
(
    (MaxYaw * -2.0) / ViewportSizeX
)

AimOffsetPitch =
(
    (ViewportSizeY / 2.0) - MouseLocationY
)
*
(

```

```

        (MaxPitch * 2.0) / ViewportSizeY
    )

    CameraRotateMode:
    EnhancedInputAction IA_MakeCanCameraRotate

    Started:
    CanCameraRotate = true

    → BountyHunterRef.StopSetDefaultCameraRotation()

    StartCameraRotation =
    BountyHunterRef.SpringArm.GetWorldRotation()

    OrbitRotation =
    StartCameraRotation

    → Call OnCameraRotateStateChange(
        CanRotate = true
    )

    Completed:
    CanCameraRotate = false

    → BountyHunterRef.SetDefaultCameraRotation()

    → Call OnCameraRotateStateChange(
        CanRotate = false
    )

    RotateCameraInput:
    EnhancedInputAction IA_RotateCamera
    Triggered
    → Branch(
        CanCameraRotate
        && !StudioManagerRef.StageViewActive
    )

    True:
    → BountyHunterRef.RotateCamera(
        ActionValue = IA_RotateCamera.ActionValue
    )

```

## CanMove

```

BP_PlayerController / CanMove

CanMove:

CurrentCharacterState =
StudioManagerRef.CurrentCharacterState

CanMove =
    CurrentCharacterState != Talk
    && CurrentCharacterState != Dance
    && !StudioManagerRef.IsGesturing

→ Return CanMove

```

## RequestViewportClick

```

BP_PlayerController / RequestViewportClick

RequestViewportClick:

HitResult = GetHitResultUnderCursorByChannel(
    TraceChannel = Visibility,
    TraceComplex = true
)

HitLocation = HitResult.Location
SurfaceNormalZ = HitResult.ImpactNormal.Z

Branch(
    HitResult.BlockingHit
    && SurfaceNormalZ > 0.5
    && CanMove()
)

True:
ProjectedLocation = ProjectPointToNavigation(
    Point = HitLocation
)

Branch(ProjectPointToNavigation.ReturnValue)

True:
TraceStart = ProjectedLocation
TraceEnd = ProjectedLocation + (0, 0, -1000)

GroundHit = LineTraceForObjects(
    Start = TraceStart,
    End = TraceEnd,
    ObjectTypes = [WorldStatic],
    IgnoreSelf = true
)

```

```

)

Branch(GroundHit.ReturnValue)

True:
GoalPosition.Location = GroundHit.ImpactPoint

SpawnSystemAtLocation(
    SystemTemplate = FXCursor,
    Location = GoalPosition.Location,
    Rotation = (0, 0, 0),
    Scale = (1, 1, 1)
)

→ OnMoveToClicked()

```

## StudioManager

### BP\_StudioManager 역할

- **BP\_StudioManager**: Operator UI와 실제 Runtime System 사이에서 상태와 명령을 관리하는 중앙 Runtime Manager
- UI Widget이 Character, Camera, Lighting, Material을 직접 수정하지 않고 **BP\_StudioManager** 를 통해 명령을 전달하도록 구성
- Character, Camera, Lighting, Appearance, Position, Gesture, Look Tracking 등 서로 다른 Runtime System의 현재 State를 한 곳에서 관리
- Preset 실행, CurrentPreset 판정, Stage View 전환, Gesture 실행, Position 이동 요청, Lighting 적용 등을 공통 진입점에서 처리
- 각 System의 실제 세부 동작은 해당 Actor나 Blueprint에 맡기고, **BP\_StudioManager** 는 어떤 State를 적용할지 결정하고 명령을 전달하는 역할에 집중

```

Operator UI
→ BP_StudioManager
├─ CharacterState
├─ CameraState
├─ LightingState
├─ PositionState
├─ GestureState
├─ AppearancePreset
├─ LookTracking
└─ PresetState

→ 각 Runtime System에 적용

```

이 구조의 목적은 UI가 **Animation**, **Camera**, **Lighting**, **Material** 을 직접 제어해서 기능끼리 얽히는 것을 줄이고, 상태 변경 경로를 한 곳으로 모으는 것

### Runtime 초기화

- **BeginPlay** 에서 Studio 전체 운용에 필요한 Reference를 먼저 획득
- Character, PlayerController, Operator UI, Stage Camera, Stage, Ceiling Light, Moving Head 등을 가져와 저장
- **WBP\_Operator** 를 생성하고 Viewport에 추가
- Character 초기화가 끝난 뒤 Studio 초기화를 진행하도록 **OnRuntimeInitialized** Event Dispatcher에 바인딩

```

BeginPlay
→ BountyHunterRef
→ StageCameraAnchorRef
→ OperatorRef 생성
→ PlayerControllerRef
→ MovingHeadRefs
→ CeilingLightRef
→ StageRef

```

### Character 초기화 순서 대응

- **BP\_BountyHunter** 가 Camera Tracking Offset, Material Instance 등 Runtime 초기화를 완료하기 전에 StudioManager가 State를 적용하면 잘못된 Reference나 초기값을 사용할 수 있음
- 이를 방지하기 위해 **OnRuntimeInitialized** Event Dispatcher를 사용
- Character가 이미 초기화된 상태라면 바로 **InitializeStudio**
- 아직 초기화되지 않았다면 Event를 기다렸다가 실행

```

BountyHunter BeginPlay
→ Runtime 초기화
→ IsRuntimeInitialized = true
→ OnRuntimeInitialized

BP_StudioManager
→ Event 수신
→ InitializeStudio

```

- Actor들의 **BeginPlay** 실행 순서를 임의로 가정하지 않고 초기화 완료 Event를 기준으로 연결한 구조

이건 발표에서 꽤 괜찮은 포인트임

### Studio 초기 상태 설정

`InitializeStudio` 에서 Operator UI와 입력 상태를 초기화

- `OperatorRef.ResetAllState()` 호출
- `SetInputModeGameAndUI` 적용
- Operator Widget에 Focus 지정
- Mouse Cursor 표시

```
InitializeStudio
→ Operator UI Reset
→ Input Mode = Game And UI
→ Widget Focus
→ Mouse Cursor ON
```

- UI 조작과 World Interaction을 모두 사용해야 하기 때문에 `Game Only` 나 `UI Only` 대신 `Game And UI` 사용

이전 Click-to-Move 작업에서 Input Mode와 UI가 World Click에 영향을 줬기 때문에 이 설정도 실제 구현상 중요한 부분

## Runtime State 관리

### State를 중앙에서 관리

`BP_StudioManager` 에서 다음 상태를 보관

- `CurrentPresetState`
- `CurrentCharacterState`
- `CurrentPositionState`
- `CurrentCameraState`
- `CurrentLightingState`
- `CurrentGestureState`
- `CurrentAppearancePresetState`
- `CurrentLookTrackingActive`

각 UI가 자신의 선택 상태를 따로 기억하는 방식이 아니라 **실제 Runtime State를 StudioManager 기준으로 관리**

```
UI Button
→ StudioManager Update Function
→ Current State 변경
→ Runtime System 적용
→ UI Selected State 갱신
```

### CharacterState 변경

- `UpdateCharacterState` 에서 `CurrentCharacterState` 를 갱신
- Animation Blueprint는 StudioManager의 `CurrentCharacterState` 를 읽어 StateMachine 전환에 사용
- CharacterState 자체는 `Idle`, `Talk`, `Dance` 와 같은 지속 상태만 담당
- 이동, Gesture는 CharacterState와 분리

```
Operator
→ UpdateCharacterState
→ CurrentCharacterState
→ ABP_BountyHunter
→ StateMachine 전환
```

- 이동을 시작했다고 CharacterState를 Walk로 바꾸지 않고, 이동 종료 후 기존 State로 돌아갈 수 있도록 구조 유지

## Preset System

### Preset 역할

- Preset은 하나의 기능이 아니라 여러 Runtime State를 한 번에 적용하는 상위 명령
- Built-in Preset은 CharacterState, CameraState, LightingState의 조합으로 구성
- Appearance, Position, Gesture, Look Tracking은 Preset 판정에 포함하지 않음

예시

```
Presentation
→ CharacterState = Talk
→ CameraState = Upper
→ LightingState = Neutral

Performance
→ CharacterState = Dance
→ CameraState = Full
→ LightingState = Stage
```

- Preset마다 별도의 실행 로직을 만들기보다 DataTable과 공통 Update Function을 통해 처리하는 방향으로 설계

## CurrentPreset 자동 판정

이 부분은 StudioManager에서 가장 강조할 만한 기능 중 하나

- **CurrentPresetState** 는 **마지막으로 누른 Preset 버튼을 기억하는 값이 아님**
- 실제 **CurrentCharacterState + CurrentCameraState + CurrentLightingState** 조합을 **DT\_PresetData** 와 비교해서 현재 Preset을 판정
- 모든 Built-in Preset Row를 순회하면서 세 State가 모두 일치하는 Row를 찾음
- 일치하는 Row가 있으면 해당 Preset
- 하나도 일치하지 않으면 **Custom**

```
CurrentCharacterState
+ CurrentCameraState
+ CurrentLightingState

→ DT_PresetData 전체 비교

일치
→ 해당 Built-in Preset

불일치
→ Custom
```

예시

```
Presentation
Talk + Upper + Neutral

Camera를 Close로 변경
→ Talk + Close + Neutral
→ Custom

Camera를 다시 Upper로 변경
→ Talk + Upper + Neutral
→ Presentation
```

- 사용자가 개별 State를 Override해도 현재 실제 상태를 기준으로 Preset 표시가 자동 갱신
- 다시 Built-in Preset 조합과 같아지면 해당 Preset으로 자동 복귀

이건 그냥 Preset 버튼 UI보다 훨씬 설명 가치가 높음

## Preset과 Appearance 분리

- Appearance는 Preset 판정에 포함하지 않음
- Presentation 상태에서 Hair, Skin, Clothes Color를 변경해도 **CurrentPresetState = Presentation** 유지
- Character의 연출 State와 외형 Customization을 별개 개념으로 분리

```
Presentation
→ Appearance 변경
→ Presentation 유지
```

- Appearance를 Preset에 포함하면 색상 하나만 바뀌도 Preset이 Custom으로 바뀌는 문제가 생기기 때문에 제외

## Position State와 이동 요청

### Position State

- Position은 **Center, Left, Right, Back, Front, Custom** 으로 관리
- Fixed Position은 Level에 배치한 Position Actor 위치를 사용
- Click-to-Move는 PlayerController에서 계산한 **GoalPositionLocation** 을 **Custom** Position으로 사용

```
Center → StagePositionCenterRef
Left → StagePositionLeftRef
Right → StagePositionRightRef
Back → StagePositionBackRef
Front → StagePositionFrontRef
Custom → PlayerController GoalPosition
```

- 고정 Position Button과 Click-to-Move가 결국 같은 이동 Request로 연결되도록 구성

### RequestMove

- StudioManager는 Character 이동 자체를 수행하지 않고 Goal 위치만 전달
- 실제 회전과 Navigation은 **BP\_BountyHunter** 가 처리

```
StudioManager
→ GoalPosition 설정
```

```
MovementPhase == Idle
→ TurningToMove

이미 이동 중
→ MoveToGoal
```

- Character가 정지 상태라면 이동 전 회전부터 시작
- 이미 이동 중이라면 새로운 Goal로 바로 이동 경로를 갱신

#### 즉 이동 중 Position 재지정이 가능

이전 작업에서 중요하게 다뤘던 부분이라 넣는 게 좋음

## Gesture 관리

### Gesture와 CharacterState 분리

- Gesture는 CharacterState와 별개의 일회성 Action
- PlayGesture에서 CurrentGestureState에 맞는 Montage를 선택해서 재생
- None이거나 이미 Gesture 실행 중이면 중복 실행 방지
- Gesture 실행 시작 시 IsGesturing = true
- 완료 또는 Interrupted 시 false로 복귀
- 실행 후 CurrentGestureState = None

```
Gesture 선택
→ Montage 선택
→ IsGesturing = true
→ PlayMontage
→ Completed / Interrupted
→ IsGesturing = false
→ GestureState = None
```

- Gesture가 끝나도 CharacterState 자체는 바뀌지 않기 때문에 기존 Idle / Talk / Dance 상태로 돌아갈 수 있음

### Gesture 중복 실행 방지

- IsGesturing 값을 사용해 Gesture가 재생 중일 때 다른 Gesture를 바로 중첩 실행하지 않도록 했다
- Montage 종료와 Interrupted를 모두 처리해 중간에 다른 State 명령으로 Montage가 끊겨도 Gesture 상태가 남지 않도록 했다

이것도 실제 Runtime State가 꼬이지 않게 한 부분

## Camera State

### Camera State 업데이트

- UpdateCameraState에서 CurrentCameraState를 먼저 갱신
- 일반 Camera 상태라면 BP\_BountyHunter.ApplyCameraState 호출
- Stage View가 활성화되어 있다면 State 값만 바꾸고 실제 Camera에는 즉시 적용하지 않음

```
CameraState 변경

StageView OFF
→ CurrentCameraState 갱신
→ ApplyCameraState

StageView ON
→ CurrentCameraState만 갱신
→ Stage Camera 유지
```

- Stage View 중에도 Full / Upper / Close 선택을 받을 수 있지만 실제 Stage View Camera를 깨지 않도록 함
- Stage View를 끄면 마지막으로 선택한 CurrentCameraState 기준으로 복귀 가능

이건 이전에 같이 많이 수정했던 핵심 규칙

## Stage View

### Stage View를 CameraState와 분리

- Stage View는 Full, Upper, Close와 같은 Camera State가 아니라 임시 Camera Override
- 활성화 중에도 CurrentCameraState는 유지
- 종료 후 마지막 Camera State로 복귀

### Stage View 시작

```
Stage View ON
→ 기존 Camera Blend 중지
→ Camera를 StageCameraAnchor에 Attach
→ 현재 Camera Location / Rotation / FOV 저장
```

```
→ StageViewActive = true
→ CurrentCameraState 유지
→ Timeline Blend
```

- `KeepWorld`로 Attach해서 Parent가 바뀌는 순간 Camera Transform이 튀지 않도록 했다
- 이후 Relative Location / Rotation / FOV를 Timeline으로 Stage Camera 기준값까지 보간

## Stage View 종료

```
Stage View OFF
→ Stage View 시작 Timeline 중지
→ Camera를 SpringArm에 다시 Attach
→ StageViewActive = false
→ CurrentCameraState 다시 적용
→ RelativeLocation을 SpringArm 기준 0으로 보간
```

- Camera State의 Rotation과 FOV는 `UpdateCameraState`가 다시 적용
- Stage View용 Transition과 일반 Camera State Transition이 동시에 Camera 값을 수정하지 않도록 기존 Timeline을 먼저 중지

이건 발표에서 트러블슈팅으로도 쓸 만함

## Lighting State

### Lighting DataTable 기반 적용

- Lighting State를 변경할 때 각 Light의 값을 Blueprint에 직접 하드코딩하지 않고 `DT_LightingData`에서 가져옴
- 동일 State 값을 Character Light, Ceiling Light, Moving Head, Stage Emissive에 전달

```
CurrentLightingState
→ DT_LightingData

├─ Character Light
├─ Ceiling Light
├─ Tracking Moving Head
├─ Stage Moving Head
└─ Stage Emissive
```

- Lighting State 하나로 무대 전체의 연출을 같이 변경하도록 구성

### Character Lighting

- `CharacterLightIntensity1`
  - `CharacterLightIntensity2`
  - `CharacterLightColor`
- `BP_BountyHunter.ApplyCharacterLighting`

### Ceiling Light

- Character Light Color를 Ceiling Light에 전달

### Tracking Moving Head

- `TrackingBeamVisible`
- `TrackingDimmer`
- `TrackingBeamColor`

### Stage Moving Head

- `StageLightVisible`
- `StageLightColor`
- `StageLightMove`

### Stage

- `BP_Stage.ChangeStageLight`
- Stage Material의 Emissive Color도 현재 Lighting State에 맞게 변경
- Character Light만 따로 바뀌는 것이 아니라 Stage 전체 Lighting Theme이 함께 변경되도록 연결

### 같은 Lighting State 중복 적용 방지

- `NewLightingState == CurrentLightingState`이면 별도 처리를 하지 않도록 했다
- 같은 State 버튼을 반복해서 눌렀을 때 불필요하게 Light Parameter와 Animation을 다시 적용하지 않도록 한 처리

## Appearance Preset

### DataTable 기반 Appearance 적용

- Appearance Preset도 각 버튼에서 Material 값을 직접 넣지 않고 `DT_AppearancePresetData` 에서 관리
- Preset Enum을 RowName으로 변환해 Data를 조회
- Hair, Clothes, Eye, Lip, Skin 데이터를 Character의 각 Set Function에 전달

```
AppearancePreset
→ DT_AppearancePresetData
→ Hair
→ Clothes
→ Eye
→ Lip
→ Skin
```

- UI와 실제 Material Parameter 값을 분리해서 Appearance 조합을 변경할 때 Blueprint Logic을 수정하지 않고 DataTable 값만 변경할 수 있도록 했다

## Appearance 적용 책임 분리

- `BP_StudioManager`
  - 어떤 Appearance Preset을 사용할지 결정
  - DataTable Row 조회
- `BP_BountyHunter`
  - 실제 Dynamic Material Instance Parameter 적용

```
StudioManager
→ Appearance Data 선택

BountyHunter
→ Material Parameter 적용
```

- State 결정과 실제 Rendering 처리를 분리

## Look Tracking State

- `UpdateLookTrackingState` 에서 현재 Look Tracking 설정값을 StudioManager에 저장
- 실제 Mouse Tracking 계산은 PlayerController에서 처리
- StudioManager가 PlayerController의 `LookTrackingActive` 값을 갱신

```
Operator UI
→ StudioManager
→ CurrentLookTrackingActive
→ PlayerController.LookTrackingActive
→ AimOffsetYaw / Pitch 계산
→ AnimBP
```

- UI가 PlayerController를 직접 찾고 값을 바꾸는 대신 StudioManager를 통해 전달

## Character Mesh 위치 보정

이 부분은 코드만 보면 뜬금없어 보여서 설명을 꼭 넣는 게 좋음

- Animation마다 Root / Mesh 기준 위치 차이 때문에 CharacterState나 MovementPhase가 바뀔 때 Character Mesh 높이나 위치가 미세하게 달라 보이는 문제가 있었다
- 이를 보정하기 위해 State별 `Mesh.RelativeLocation` 기준값을 따로 저장
- CharacterState와 MovementPhase에 따라 목표 Mesh Relative Location을 선택
- Timeline과 `Lerp` 를 사용해 현재 위치에서 목표 위치로 부드럽게 보정

```
CharacterState / MovementPhase
→ 목표 Mesh RelativeLocation 선택
→ 현재 Mesh 위치 저장
→ Timeline
→ Lerp
→ Mesh RelativeLocation 보정
```

## Idle 상태

MovementPhase에 따라 서로 다른 보정값 사용

```
Idle
├─ Idle
├─ TurningToPath
├─ Moving
├─ TurningToCamera
└─ Landing
```

## Talk / Dance

- Talk용 Mesh Relative Location

- Dance용 Mesh Relative Location
- Animation Asset마다 Root 위치가 달라 Character 발이 Stage에서 뜨거나 박히는 현상을 보정하기 위한 처리

이건 예전에 Mesh 위치, 회전 Pivot 문제 때문에 여러 번 수정했던 부분이라 기록할 가치 있음

## Event Dispatcher 사용

StudioManager와 각 System이 매 Frame 서로 상태를 계속 확인하지 않도록 Event Dispatcher를 일부 사용했다

### Character 초기화 완료

```
BountyHunter.OnRuntimeInitialized
→ StudioManager.InitializeStudio
```

### CharacterState 변경

```
OnCharacterStateChanged
→ Mesh Location 보정
→ 필요 시 Music 상태 처리
```

### MovementPhase 변경

```
BountyHunter.OnChangeMovePhase
→ Mesh Location 보정
```

### Gesture 상태 변경

```
OnGestureEnd
→ UI / Runtime 상태 갱신
```

- 상태 변화가 발생했을 때 필요한 System만 반응하도록 구성

다만 `CurrentCharacterState != PreCharacterState` 비교는 현재 Tick에서 직접 감지하고 있으므로 **전부 Event 기반이라고 표현하면 안 됨**

## Dance Music 관리

- Dance Animation과 Sound를 연결하기 위해 `PlayMusic`, `StopMusic`을 StudioManager에서 관리
- 실제 재생 시점은 Anim Notify에서 호출
- Dance가 정상 종료되지 않고 다른 CharacterState로 변경될 경우에도 음악이 남지 않도록 State 변경 시 `StopMusic` 처리

```
Anim Notify
→ PlayMusic

Dance 종료 / Idle 전환
→ StopMusic
→ FadeOut 1초
```

- `FadeOut`을 사용해 음악이 순간적으로 끊기지 않도록 했다

## StudioManager 전체 구조

```
BP_StudioManager
├── Runtime Initialization
│   ├── Character
│   ├── Operator UI
│   ├── PlayerController
│   ├── Stage
│   └── Lighting Actors
├── Runtime State
│   ├── Preset
│   ├── Character
│   ├── Position
│   ├── Camera
│   ├── Lighting
│   ├── Gesture
│   ├── Appearance
│   └── Look Tracking
├── Preset
│   └── Actual State Matching → Built-in / Custom
├── Character
│   ├── CharacterState
│   ├── Movement Request
│   └── Mesh Location Correction
├── Camera
│   ├── CameraState
│   └── Stage View Override
└──
```

```

├─ Lighting
│  └─ Character Light
│  └─ Ceiling Light
│  └─ Moving Head
│  └─ Stage Emissive
├─ Appearance
│  └─ DataTable → BountyHunter Material
├─ Gesture
│  └─ Montage / IsGesturing
└─ Sound
   └─ Dance Music

```

## 구현 의도 정리

**BP\_StudioManager**의 핵심은 기능을 직접 전부 수행하는 Actor를 만드는 것이 아니라 각 **Runtime System**의 현재 **State**를 관리하고 서로의 동작을 연결하는 **Command / State Hub**를 만드는 것

```

Operator 입력
→ StudioManager가 State 결정
→ 필요한 Runtime System에 명령
→ 실제 System이 처리
→ 변경된 State를 다시 UI에 반영

```

이 구조 덕분에

- UI가 Runtime Asset을 직접 제어하지 않음
- Preset과 개별 State Override를 함께 사용할 수 있음
- 현재 실제 상태를 기준으로 **CurrentPreset**을 다시 판정 가능
- Stage View처럼 일시적인 Override가 기존 Camera State를 파괴하지 않음
- Appearance / Position / Gesture처럼 Preset과 독립적인 기능을 별도로 운용 가능

## BP\_StudioManager

### EventGraph

```

BP_StudioManager / EventGraph

BeginPlay:
→ Sequence

Then0:
BountyHunterRef =
    Cast<BP_BountyHunter>(
        GetPlayerPawn(PlayerIndex = 0)
    )

StageCameraAnchorRef =
    GetActorOfClass(BP_StageCamera)

OperatorRef =
    CreateWidget(
        Class = WBP_Operator
    )

→ AddToViewport(OperatorRef)

PlayerControllerRef =
    Cast<BP_PlayerController>(
        GetPlayerController(PlayerIndex = 0)
    )

TrackingMovingHeadRefs =
    GetAllActorsOfClass(BP_StudioMovingHead)

MovingHeadRefs =
    GetAllActorsOfClass(BP_StudioMovingHead)

CeilingLightRef =
    GetActorOfClass(BP_CeilingLight)

StageRef =
    GetActorOfClass(BP_Stage)

Then1:
Branch(
    IsValid(BountyHunterRef)
)

True:
BindEventToOnRuntimeInitialized(
    Target = BountyHunterRef,
    Event = InitializeStudio
)

Branch(
    BountyHunterRef.IsRuntimeInitialized
)

True:

```

```

InitializeStudio()

False:
PrintString("BountyHunterRef Is Not Valid")

InitializeStudio:
Branch(
    IsValid(OperatorRef)
)

True:
OperatorRef.ResetAllState()

Branch(
    IsValid(PlayerControllerRef)
)

True:
SetInputModeGameAndUI(
    PlayerController = PlayerControllerRef,
    InWidgetToFocus = OperatorRef,
    MouseLockMode = DoNotLock
)

PlayerControllerRef.ShowMouseCursor = true

False:
PrintString("PlayerControllerRef Is Not Valid")

Then2:
BindEventToOnCharacterStateChanged(
    Target = OperatorRef,
    Event = EditCharacterMeshLocation
)

BindEventToOnChangeMovePhase(
    Target = BountyHunterRef,
    Event = EditCharacterMeshLocation
)

EditCharacterMeshLocation:
GoalCharacterStateLocation =
    Select(CurrentCharacterState)
    Idle = Select(MovementPhase)
        Idle = BountyHunterRef.IdleMeshRelativeLocation
        TurningToPath = BountyHunterRef.TurningMeshRelativeLocation
        Moving = BountyHunterRef.MovingMeshRelativeLocation
        TurningToCamera = BountyHunterRef.TurningMeshRelativeLocation
        Landing = BountyHunterRef.LandingMeshRelativeLocation

    Talk = BountyHunterRef.TalkMeshRelativeLocation
    Dance = BountyHunterRef.DanceMeshRelativeLocation

StartCharacterStateLocation =
    BountyHunterRef.Mesh.RelativeLocation

TL_SetLocation.PlayFromStart()

TL_SetLocation.Update:
NewLocation = Lerp(
    A = StartCharacterStateLocation,
    B = GoalCharacterStateLocation,
    Alpha = TL_SetLocation.Alpha
)

BountyHunterRef.Mesh.SetRelativeLocation(NewLocation)

Then3:
BindEventToOnCharacterStateChanged(
    Target = self,
    Event = CustomEvent
)

CustomEvent(
    CurrentCharacterState,
    PreCharacterState
):

Switch(CurrentCharacterState)

Idle:
    Switch(PreCharacterState)
        Talk → StopMusic()
        Dance → StopMusic()

Talk:
    no action

Dance:
    no action

Tick:
Branch(
    CurrentCharacterState != PreCharacterState
)

True:
OnCharacterStateChanged(
    CurrentCharacterState = CurrentCharacterState,

```

```

        PreCharacterState = PreCharacterState
    )

    PreCharacterState = CurrentCharacterState

```

## GestureGraph

```

BP_StudioManager / GestureGraph

PlayGesture(CurrentGestureState):

Branch(
    CurrentGestureState == None
    || IsGesturing
)

False:
GestureMontage = Select(CurrentGestureState)
    None         = None
    Wave         = AM_Wave
    Point        = AM_Point
    CrossHands   = AM_CrossHand
    Pose         = AM_Pose
    Clap         = AM_Clap
    Laugh        = AM_Laugh

IsValid(GestureMontage)
→ IsGesturing = true

→ OnGestureEnd(IsGesturing)

→ PlayMontage(
    InSkeletalMeshComponent = BountyHunterRef.Mesh,
    MontageToPlay = GestureMontage,
    PlayRate = 1.0,
    StartingPosition = 0.0,
    StartingSection = Default,
    ShouldStopAllMontages = true
)

OnCompleted
|| OnInterrupted
→ IsGesturing = false

→ OnGestureEnd(IsGesturing)

→ CurrentGestureState = None

```

## CameraGraph

```

BP_StudioManager / CameraGraph

SetStageView(Active):

Branch(Active)

True:
Sequence

Then0:
StopTLEndStageView()

Then1:
IsValid(BountyHunterRef)

Not Valid:
BountyHunterRef = GetActorOfClass(BP_BountyHunter)

→ BountyHunterRef.StopCameraBlend()

Then2:
IsValid(StageCameraAnchorRef)

→ AttachComponentToComponent(
    Target = BountyHunterRef.Camera,
    Parent = StageCameraAnchorRef.RootComponent,
    LocationRule = KeepWorld,
    RotationRule = KeepWorld,
    ScaleRule = KeepWorld
)

StartStageFieldOfView = BountyHunterRef.Camera.FieldOfView
StartStageLocation = BountyHunterRef.Camera.RelativeLocation
StartStageRotation = BountyHunterRef.Camera.RelativeRotation

StageViewActive = true

UpdateCameraState(
    NewCameraState = CurrentCameraState,
    StageViewActive = true
)

Then3:
TL_StartStageView.PlayFromStart()

TL_StartStageView.Update:
BountyHunterRef.Camera.RelativeLocation =
    Lerp(
        A = StartStageLocation,

```

```

        B = (0.0, 0.0, 0.0),
        Alpha = TimelineAlpha
    )

    BountyHunterRef.Camera.RelativeRotation =
    Lerp(
        A = StartStageRotation,
        B = (0.0, 0.0, 0.0),
        Alpha = TimelineAlpha
    )

    BountyHunterRef.Camera.FieldOfView =
    Lerp(
        A = StartStageFieldOfView,
        B = StageFieldOfView,
        Alpha = TimelineAlpha
    )

    StopTLStartStageView:
    → TL_StartStageView.Stop()

False:
Sequence

Then0:
StopTLStartStageView()

Then1:
IsValid(BountyHunterRef)

Not Valid:
BountyHunterRef = GetActorOfClass(BP_BountyHunter)

→ BountyHunterRef.StopCameraBlend()

Then2:
AttachComponentToComponent(
    Target = BountyHunterRef.Camera,
    Parent = BountyHunterRef.SpringArm,
    LocationRule = KeepWorld,
    RotationRule = KeepWorld,
    ScaleRule = KeepWorld
)

StartStageLocation = BountyHunterRef.Camera.RelativeLocation
StageViewActive = false

UpdateCameraState(
    NewCameraState = CurrentCameraState,
    StageViewActive = false
)

→ TL_EndStageView.PlayFromStart()

TL_EndStageView.Update:
BountyHunterRef.Camera.RelativeLocation =
    Lerp(
        A = StartStageLocation,
        B = (0.0, 0.0, 0.0),
        Alpha = TimelineAlpha
    )

// SpringArm 기준 RelativeLocation만 (0,0,0)까지 보간
// Rotation / FOV는 UpdateCameraState에서 현재 Camera State 기준으로 복귀

StopTLEndStageView:
→ TL_EndStageView.Stop()

```

## UpdateStateFunction

```

BP_StudioManager / Update State Functions

UpdatePresetState(NewPresetState):
CurrentPresetState = NewPresetState

UpdateCharacterState(NewCharacterState):
CurrentCharacterState = NewCharacterState

UpdatePositionState(NewPositionState):
IsValid(PlayerControllerRef)

→ CurrentPositionState = NewPositionState

GoalPositionLocation = Select(CurrentPositionState)
Center = StagePositionCenterRef.GetActorLocation()
Left = StagePositionLeftRef.GetActorLocation()
Right = StagePositionRightRef.GetActorLocation()
Custom = PlayerControllerRef.GoalPositionLocation
Back = StagePositionBackRef.GetActorLocation()
Front = StagePositionFrontRef.GetActorLocation()

RequestMove(
    GoalPositionLocation = GoalPositionLocation
)

UpdateCameraState(
    StageViewActive,
    NewCameraState
)

```

```

):

CurrentCameraState = NewCameraState

Branch(StageViewActive)

False:
IsValid(BountyHunterRef)
→ BountyHunterRef.ApplyCameraState(
    CurrentCameraState
)

True:
→ Camera State 값만 갱신
→ Stage View 카메라에는 즉시 적용하지 않음

UpdateLightingState(NewLightingState):
Branch(
    NewLightingState == CurrentLightingState
)

True:
→ 변경 없음

False:
CurrentLightingState = NewLightingState

LightingData = DT_LightingData[
    EnumToString(CurrentLightingState)
]

BountyHunterRef.ApplyCharacterLighting(
    Intensity1 = LightingData.CharacterLightIntensity1,
    Intensity2 = LightingData.CharacterLightIntensity2,
    Color      = LightingData.CharacterLightColor
)

CeilingLightRef.ApplyCeilingLighting(
    Color = LightingData.CharacterLightColor
)

ForEach(TrackingMovingHeadRefs):
    ArrayElement.ApplyTrackingLightState(
        BeamVisible = LightingData.TrackingBeamVisible,
        Dimmer      = LightingData.TrackingDimmer,
        BeamColor   = LightingData.TrackingBeamColor
    )

ForEach(MovingHeadRefs):
    ArrayElement.ApplyStageLightState(
        BeamVisible = LightingData.StageLightVisible,
        BeamColor   = LightingData.StageLightColor,
        LightColor  = LightingData.StageLightColor
    )

    ArrayElement.DancingLight(
        Move = LightingData.StageLightMove
    )

StageRef.ChangeStageLight(
    NewParam = CurrentLightingState
)

UpdateGestureState(NewGestureState):
CurrentGestureState = NewGestureState

UpdateAppearancePresetState(NewAppearancePresetState):
CurrentAppearancePresetState = NewAppearancePresetState

ApplyAppearancePreset(
    Preset = CurrentAppearancePresetState
)

UpdateLookTrackingState(LookTrackingActive):
CurrentLookTrackingActive = LookTrackingActive

PlayerControllerRef.LookTrackingActive =
    LookTrackingActive

```

## MatchPresetState

```

BP_StudioManager / MatchPresetState

MatchPresetState:
RowNames = GetDataTableRowNames(DT_PresetData)

ForEach(RowNames):
    PresetData = GetDataTableRow(
        DataTable = DT_PresetData,
        RowName = ArrayElement
    )

    OutRowPresetState = PresetData.PresetState

    Branch(
        PresetData.CharacterState == CurrentCharacterState
        && PresetData.CameraState == CurrentCameraState
        && PresetData.LightingState == CurrentLightingState
    )

```

```

    )

    True:
        CurrentPresetState = OutRowPresetState

        OnPresetChanged(
            CurrentPresetState = CurrentPresetState
        )

        → Return

ForEach Completed:
OnPresetChanged(
    CurrentPresetState = Custom
)

→ Return

```

## ApplyAppearancePreset

```

BP_StudioManager / ApplyAppearancePreset

ApplyAppearancePreset(Preset):
IsValid(BountyHunterRef)

→ CurrentAppearancePresetState = Preset

RowName = EnumToString(Preset)

AppearanceData = GetDataTableRow(
    DataTable = DT_AppearancePresetData,
    RowName = RowName
)

→ BountyHunterRef.SetHairColor(
    HairColor = AppearanceData.HairColor,
    EyebrowColor = AppearanceData.EyebrowColor
)

→ BountyHunterRef.SetClothesColor(
    ClothesParameters = AppearanceData.ClothesParameters
)

→ BountyHunterRef.SetEyeColor(
    EyeColor = AppearanceData.EyeColor
)

→ BountyHunterRef.SetLipColor(
    LipParameters = AppearanceData.LipColors
)

→ BountyHunterRef.SetSkinColor(
    SkinParameters = AppearanceData.SkinParameters
)

```

## RequestMove

```

BP_StudioManager / RequestMove

RequestMove(GoalPositionLocation):
BountyHunterRef.GoalPositionLocation =
    GoalPositionLocation + (0.0, 0.0, 2.0)

Branch(
    BountyHunterRef.MovementPhase == Idle
)

True:
→ BountyHunterRef.TurningToMove()

False:
→ BountyHunterRef.MoveToGoal()

```

## GetMovingState

```

BP_StudioManager / GetMovingState

GetMovingState:
Return =
    BountyHunterRef.IsMoving

```

## PlayMusic

```

BP_StudioManager / PlayMusic

PlayMusic:
DanceMusic.Play(
    StartTime = 0.0
)

```

## StopMusic

```
BP_StudioManager / StopMusic
```

```
StopMusic:  
DanceMusic.FadeOut(  
    FadeOutDuration = 1.0  
)
```

## Lighting

### Lighting System 전체 구조

- Lighting은 Character를 항상 잘 보이게 하는 조명과 Stage 분위기를 만드는 조명을 역할별로 분리해서 구성했다
- 최종 **LightingState** 는 **Neutral**, **Focus**, **Stage** 3종으로 구성했다
- 하나의 Lighting State가 Character Light, Ceiling Light, Tracking Moving Head, Stage Moving Head, Stage Emissive에 함께 적용되도록 했다
- 각 Light를 UI에서 개별적으로 조작하는 Lighting Editor를 만드는 대신 **미리 정의한 Lighting State를 Operator가 선택하면 전체 조명 구성이 함께 변경되는 방식**으로 구현했다
- 각 State의 Color, Intensity, Beam 표시 여부, Moving Head 동작 여부 등은 **DT\_LightingData** 에서 관리하고 **BP\_StudioManager** 가 각 Lighting Actor에 전달한다

```
Operator UI  
→ LightingState  
→ BP_StudioManager  
→ DT_LightingData  
  ├── Character Light  
  ├── Ceiling Light  
  ├── Tracking Moving Head  
  ├── Stage Moving Head  
  └── Stage Emissive
```

- Character와 Stage Lighting을 별도로 구현하되 같은 State를 기준으로 변경해 서로 다른 분위기가 적용되는 것을 줄였다

### Lighting State

#### Neutral

- 기본 대기 상태용 Lighting
- Character가 보일 정도의 기본 조명 유지
- 전방 Tracking Moving Head는 Character를 추적하지만 Beam 표현은 사용하지 않도록 구성
- 후방 Stage Moving Head의 연출은 사용하지 않음
- Stage Emissive는 기본 Color 유지

#### Focus

- Character를 강조하는 Presentation용 Lighting
- Character Light를 Neutral보다 강조
- 전방 Tracking Moving Head가 Character를 계속 추적
- Moving Head Beam을 표시해 Character 쪽으로 향하는 조명 연출을 추가
- Character가 이동해도 Tracking을 유지

#### Stage

- Dance / Performance 상태용 Lighting
- Character Light와 Stage Light를 같은 계열의 Color Theme으로 변경
- 전방 Tracking Moving Head는 Character를 계속 추적
- 후방 Stage Moving Head를 활성화하고 움직이는 조명 연출 추가
- Stage Emissive도 **StageLightColor** 로 변경해 Character 조명뿐 아니라 무대 전체 분위기가 함께 바뀌도록 구성

```
Neutral  
→ 기본 Character Lighting  
→ Tracking 유지  
→ Beam OFF  
→ Stage 연출 OFF  
  
Focus  
→ Character 강조  
→ Tracking 유지  
→ Tracking Beam ON  
  
Stage  
→ Character Lighting 변경  
→ Tracking Beam ON  
→ Stage Moving Head ON  
→ Moving Head Animation  
→ Stage Emissive 변경
```

## Character Lighting

- Character 자체에는 별도의 Spot Light 2개를 사용했다
- Character의 가시성을 Stage Lighting에만 의존하지 않고 기본 Character Light를 별도로 유지했다
- BP\_StudioManager 에서 DT\_LightingData 를 조회한 뒤 BP\_BountyHunter.ApplyCharacterLighting 으로 Intensity와 Color를 전달한다

```
LightingState
→ DT_LightingData
→ CharacterLightIntensity1
→ CharacterLightIntensity2
→ CharacterLightColor
→ BP_BountyHunter.ApplyCharacterLighting
```

- 두 Spot Light의 Intensity와 Color를 한 함수에서 동시에 변경하도록 구성했다
- Character Light와 Stage Moving Head를 분리해 Stage 연출이 바뀌어도 Character 자체의 가시성을 별도로 조정할 수 있도록 했다

## Character Light Bone Tracking

- Character Light의 Target은 고정된 Actor 위치가 아니라 Character의 현재 Bone 위치를 따라가도록 했다
- BP\_BountyHunter 의 LightTrackingPivot 이 head Socket을 기준으로 Animation에 따른 Character 위치 변화를 추적한다
- 초기 LightTrackingPivot 과 Head Socket 사이 Offset을 Component Space에서 저장하고 Runtime에서 현재 Bone 위치에 다시 적용한다
- World Space로 변환한 뒤 VInterpTo 로 보간해 Animation의 작은 움직임 때문에 Light Target이 급격하게 흔들리지 않도록 했다

```
Head Socket
→ 초기 Offset 적용
→ Component Space
→ World Space
→ VInterpTo
→ LightTrackingPivot
```

- Character가 이동하거나 Animation을 재생해도 Character Lighting이 실제 머리 위치를 기준으로 따라갈 수 있도록 구성했다

## BP\_CeilingLight

### 역할

- Stage 전체를 위에서 보조하는 Rect Light
- 복잡한 Tracking이나 Animation을 담당하지 않고 현재 Lighting State의 Color를 적용하는 보조 Light 역할
- BP\_StudioManager 에서 전달받은 Color를 ApplyCeilingLighting 으로 적용한다

## ApplyCeilingLighting

```
Color
→ RectLightComponent.SetLightColor
```

- Lighting State 변경 시 Character와 Stage 주변의 전체적인 Color Tone도 같이 바뀌도록 연결

## Tracking Moving Head

### BP\_StudioMovingHead

- Character를 따라가는 전방 Moving Head
- 단순히 Beam을 Character 방향으로 돌리는 것뿐 아니라 Character와 Light 사이 거리까지 고려해 Zoom과 Intensity를 Runtime에서 자동 조절하도록 구성했다
- Character Tracking은 Lighting State와 별도로 계속 계산하고, 실제 Beam 표시나 Color는 State에 따라 제어한다

```
BP_StudioMovingHead
├─ Character Tracking
├─ Yoke / Head Rotation
├─ Distance
│ └─ Zoom
│   └─ Intensity
└─ Lighting State
    ├── Beam Visibility
    ├── Dimmer
    └─ Beam Color
```

### 초기화

- BeginPlay 에서 현재 Player Pawn을 가져와 BP\_BountyHunter Reference 저장
- Yoke와 Head의 초기 Relative Rotation을 저장
- Runtime에서 Tracking을 수행할 준비

```
BeginPlay
→ BP_BountyHunter Reference
```

```
→ DefaultYokeRotation 저장
→ DefaultHeadRotation 저장
```

## Character Tracking

- 매 Frame `UpdateTracking` 을 호출해 Moving Head가 Character를 따라가도록 했다
- Tracking Target은 Character Actor Origin이 아니라 `spine_04` Socket을 사용했다
- Character의 상체를 기준으로 조명이 향하도록 해서 Character 중심부가 안정적으로 조명 안에 들어오도록 구성했다

```
BountyHunter
→ spine_04 Socket Location
→ Tracking Target
```

## Look Rotation 계산

- Moving Head Lens 위치와 Character의 `spine_04` 위치를 이용해 `FindLookAtRotation` 계산

```
LightLocation
→ Character spine_04

FindLookAtRotation
→ TargetYaw
→ TargetPitch
```

- 계산된 Rotation을 Moving Head 구조에 맞게 Yoke와 Head에 나누어 적용

## Yoke

- 좌우 Pan 방향 담당
- `TargetYaw` 를 사용해 World Rotation 적용

## Head

- 위아래 Tilt 방향 담당
- `TargetPitch` 를 Moving Head Mesh의 축 방향에 맞게 보정해서 적용
- Mesh 자체의 기본 Rotation이 Unreal World Rotation과 일치하지 않기 때문에 `89°` Offset을 사용했다

```
FindLookAtRotation
├─ Yaw → Yoke
└─ Pitch - 89 → Head
```

- 하나의 Mesh를 통째로 회전시키는 대신 실제 Moving Head 구조처럼 Yoke와 Head의 회전 역할을 분리

## 거리 기반 Zoom

Character가 Moving Head에 가까울 때와 멀리 있을 때 같은 Beam Angle을 사용하면 Character에게 비치는 Beam 범위가 크게 달라짐

이를 보정하기 위해 Character와 Light 사이 거리를 계산해서 Zoom 값을 자동 변경했다

```
Distance
380 ~ 850

→ MapRangeClamped

Zoom
5.5 ~ 2.2
```

- 가까울 때와 멀리 있을 때 Zoom Component 값을 자동 조정
- Distance 범위를 벗어나도 값이 계속 증가하거나 감소하지 않도록 `MapRangeClamped` 사용
- Operator가 Character Position을 변경해도 별도의 Zoom 값을 다시 입력하지 않아도 현재 거리에 맞게 자동으로 변경되도록 했다

## 거리 기반 Intensity

- Character가 Moving Head에서 멀어지면 같은 Intensity에서도 상대적으로 어둡게 보이는 문제를 줄이기 위해 거리와 Intensity를 연결했다

```
Distance
380 ~ 850

→ MapRangeClamped

Intensity
30000 ~ 200000
```

- Character가 Light에서 멀어질수록 Intensity 증가
- 가까운 상태에서는 과도하게 밝아지는 것을 줄임
- Character Position 변화에 따라 Tracking 방향뿐 아니라 조명의 밝기도 함께 보정

이 부분은 Lighting에서 발효 가치가 꽤 높음

Tracking만 한 것이 아니라 Character와 Light 사이의 거리 변화까지 고려해서 Zoom과 Intensity를 같이 조절했다는 점

## Tracking Beam State

### ApplyTrackingLightState

- Moving Head가 Character를 추적하는 계산과 실제 Beam을 화면에 표시하는 상태를 분리했다
- Tracking 자체는 유지하면서 Lighting State에 따라 Beam을 보이거나 숨길 수 있도록 구성했다

```
Lighting State
→ BeamVisible
→ Dimmer
→ BeamColor
```

### Beam Visibility

- `StaticMeshBeam.SetVisibility` 를 사용해 Beam Mesh 표시 여부 변경
- Neutral처럼 Beam 연출이 필요 없는 State에서는 Beam을 숨길 수 있음

### Dimmer

- Beam이 활성화된 경우 `DimmerComponent` 에 현재 State의 Dimmer 값 적용

### Beam Color

- `BeamColor` 의 RGB 값을 분리해서 Beam Material의 `DMX Color` Parameter에 전달
- 동일 Moving Head Actor를 유지하면서 Material Parameter 변경만으로 State별 Beam Color를 적용

```
BeamColor
→ RGB
→ "DMX Color"
→ Beam Material
```

- Lighting State마다 Moving Head Actor나 Material을 교체하지 않고 Parameter만 변경

## Stage Moving Head

### BP\_StudioMovingHead\_NoTracking

- Character를 직접 추적하지 않고 Stage 전체 분위기를 만드는 후방 Moving Head
- Tracking Moving Head와 역할을 분리했다

```
Tracking Moving Head
→ Character 중심
→ Character 위치에 따라 Pan / Tilt 변경
→ Distance 기반 Zoom / Intensity

Stage Moving Head
→ Stage 연출 중심
→ Character Tracking X
→ Timeline 기반 움직임
```

- 모든 Moving Head가 Character만 따라가면 Performance 상태에서도 조명이 한곳에 집중되기 때문에 Character Tracking Light와 Stage Effect Light를 분리했다

### Stage Light State

#### ApplyStageLightState

- 현재 Lighting State에서 전달받은 값으로 Stage Moving Head의 Beam 표시 여부와 Color를 변경

```
BeamVisible
→ Beam Mesh Visibility

BeamColor
→ Beam Material "DMX Color"

LightColor
→ SpotLight Color
```

- Beam Material Color와 실제 Spot Light Color를 같은 State Data에서 받아 시각적인 Beam과 실제 Lighting Color가 서로 다르게 보이지 않도록 했다

## Dancing Light

### Stage 상태의 Moving Head Animation

- `Stage` Lighting에서는 후방 Moving Head가 고정된 방향만 비추지 않고 움직이는 조명 연출을 하도록 했다
- `DancingLight` 함수에서 현재 State가 Moving Head Animation을 사용할지 결정

```

Move = true
→ Stage Light 활성화
→ TL_DancingLight 실행

Move = false
→ Timeline 중지
→ Head 기본 방향 복귀
→ SpotLight OFF

```

## Timeline 기반 움직임

- Head Rotation을 Timeline으로 보간해 Stage Moving Head가 반복적으로 움직이도록 구성
- `LerpRotator` 와 `ShortestPath = true` 를 사용해 불필요하게 반대 방향으로 크게 회전하지 않도록 했다

```

DefaultHeadRotation
→ TargetHeadRotation

TL_DancingLight
→ Head Rotation

```

- `IsDownFirst` 값에 따라 Timeline을 정방향 또는 역방향으로 실행해 Moving Head마다 움직임의 시작 방향을 다르게 구성할 수 있도록 했다

이 방식으로 여러 Moving Head가 모두 완전히 같은 방향으로 동시에 움직이는 느낌을 줄이고 Stage 연출을 만들 수 있음

## Stage Light 비활성화

- Stage Lighting이 아닌 상태로 바뀌면 진행 중인 `TL_DancingLight` 를 정지
- Head Rotation을 기본값으로 복귀
- Spot Light Intensity를 `0` 으로 변경

```

Stage OFF
→ StopDancingLight
→ Head Rotation Reset
→ SpotLight OFF

```

- 이전 Stage State에서 실행되던 Timeline이나 Light가 다른 Lighting State에서도 남아 있는 문제를 방지

## Tracking Light와 Stage Light 역할 분리

Moving Head를 하나의 Blueprint에서 모든 역할을 처리하지 않고 용도에 따라 분리했다

| 구분                 | Tracking Moving Head  | Stage Moving Head |
|--------------------|-----------------------|-------------------|
| 목적                 | Character 강조          | Stage 분위기 연출      |
| Character Tracking | O                     | X                 |
| Target             | <code>spine_04</code> | 고정 Stage 영역       |
| Pan / Tilt         | Character 위치 기반       | Timeline 기반       |
| Distance 계산        | O                     | X                 |
| Zoom 자동 조정         | O                     | X                 |
| Intensity 거리 보정    | O                     | X                 |
| Beam Color         | State 기반              | State 기반          |
| Stage Animation    | X                     | O                 |

- 같은 Moving Head Asset을 사용하더라도 역할이 다르기 때문에 Tracking Logic과 Stage Animation Logic을 분리
- Character Tracking에 필요 없는 Stage Animation 계산과 Stage Light에 필요 없는 Character Distance 계산이 섞이지 않도록 했다

## Stage Emissive와 Lighting 연동

- Lighting State 변경은 실제 Light Actor만 바꾸는 것으로 끝내지 않고 `BP_Stage` 의 Emissive Material에도 전달했다
- `Neutral` , `Focus` 에서는 기본 Stage Color 유지
- `Stage` 에서는 Stage용 Emissive Color 적용

```

LightingState
|
├─ Character Light
├─ Ceiling Light
├─ Tracking Moving Head
├─ Stage Moving Head
└─ Stage Emissive

```

- 실제 Light Color와 Stage Material의 Emissive Accent가 같은 상태에서 변경되기 때문에 하나의 Lighting Theme처럼 보이도록 구성

## DT\_LightingData 기반 통합 제어

Lighting Actor마다 State 조건문을 각각 따로 작성하지 않고 `BP_StudioManager` 가 현재 `LightingState` 에 해당하는 Data Row를 읽어 필요한 값을 각 Actor에 전달한다

```

DT_LightingData
├── CharacterLightIntensity1
├── CharacterLightIntensity2
├── CharacterLightColor
├── TrackingBeamVisible
├── TrackingDimmer
├── TrackingBeamColor
├── StageLightVisible
├── StageLightColor
├── StageLightMove
└── 기타 Lighting Parameter

```

```

CurrentLightingState
→ DT_LightingData
→ BP_StudioManager

├── BountyHunter.ApplyCharacterLighting
├── CeilingLight.ApplyCeilingLighting
├── TrackingMovingHead.ApplyTrackingLightState
├── StageMovingHead.ApplyStageLightState
├── StageMovingHead.DancingLight
└── Stage.ChangeStageLight

```

- Lighting State가 추가되거나 기존 State의 Color / Intensity를 수정할 때 각 Blueprint의 Logic을 수정하는 대신 DataTable 값을 변경하는 방식으로 관리 가능
- Runtime Logic과 Lighting Parameter를 분리

## Lighting 구현 과정에서의 설계 변경

### 단순 Character Spot Light에서 Stage Lighting System으로 확장

초기에는 Character가 보이도록 하는 기본 조명 위주로 시작했지만 최종적으로는 역할을 다음과 같이 분리

```

Character Light
→ 기본 Character 가시성

Tracking Moving Head
→ Character 추적 / Focus 연출

Stage Moving Head
→ Performance 공간 연출

Ceiling Light
→ 전체 Tone 보조

Stage Emissive
→ Stage Material 분위기

```

- 모든 역할을 하나의 Light에 맡기지 않고 목적에 따라 분리해서 각 Lighting State에서 필요한 요소만 활성화

### Character Tracking과 Stage 연출 분리

- 처음부터 모든 Moving Head가 같은 방식으로 움직이게 만들지 않고 최종적으로 Character Tracking용과 Stage 연출용으로 역할을 나눴다
- Tracking Light는 Character 이동에 반응해야 하므로 Tick에서 위치를 갱신
- Stage Light는 Character 위치보다 Performance 연출이 중요하므로 Timeline 기반 Animation 사용

⇒ 같은 Moving Head라는 이유로 같은 Logic을 사용하는 것이 아니라 역할에 따라 제어 방식을 다르게 선택

### Tracking 기준

- Character의 Actor Location 자체를 목표로 하면 Capsule 중심이나 발 위치 때문에 Light가 상체보다 아래쪽을 향할 수 있음
- 실제 Character 연출에서 강조하고 싶은 상체 위치인 `spine_04` Socket을 Tracking Target으로 사용
- Animation 중에도 현재 Bone 위치가 갱신되기 때문에 Character Pose 변화까지 Tracking에 반영

## Lighting System 전체 구조

```

Lighting System
├── E_LightingState
│   ├── Neutral
│   ├── Focus
│   └── Stage
├── DT_LightingData
├── Character Lighting
│   ├── SpotLight 1
│   ├── SpotLight 2
│   └── Head Bone Tracking
├── Ceiling Lighting
│   └── Rect Light
└── Tracking Moving Head

```

```

|   | spine_04 Tracking
|   | Yoke Pan
|   | Head Tilt
|   | Distance-based Zoom
|   | Distance-based Intensity
|   | Beam Visibility
|   | Dimmer
|   | Beam Color
|
|   | Stage Moving Head
|   | Beam Visibility
|   | Beam / Light Color
|   | Timeline Dancing Light
|
|   | Stage
|   | Emissive Color

```

## 구현 의도 정리

Lighting에서 가장 중요하게 잡은 기준은 **Light 개별 조작 기능을 많이 만드는 것보다 Operator가 원하는 무대 상태를 하나의 명령으로 일관되게 만드는 것**

```

Operator
→ Neutral / Focus / Stage 선택
→ StudioManager
→ Lighting Data
→ 전체 Lighting System 적용

```

- Character의 가사성
- Character Tracking
- Stage Beam
- Stage Moving Head Animation
- Ceiling Color
- Stage Emissive

를 하나의 **LightingState** 아래에서 함께 관리하도록 했다

## 구현하면서 확인한 개선점

- Tracking Moving Head는 매 Frame Character Bone 위치, 거리, Rotation, Zoom, Intensity를 계산하기 때문에 개수가 크게 늘어난다면 Tick 비용을 확인할 필요가 있음
- 현재 프로젝트에서는 소수의 Moving Head를 사용하는 Stage이기 때문에 실시간 Tracking을 우선해서 Tick 기반으로 구현
- Stage Moving Head는 Character Tracking이 필요하지 않으므로 동일한 Tick Tracking Logic을 넣지 않고 Timeline 기반으로 분리

## BP\_CeilingLight

### ApplyCeilingLighting

```

BP_CeilingLight / ApplyCeilingLighting

ApplyCeilingLighting(
    Color
)

RectLightComponent.SetLightColor(
    NewLightColor = Color,
    SRGB = true
)

```

## BP\_StudioMovingHead

### EventGraph

```

BP_StudioMovingHead / EventGraph

BeginPlay:
GetPlayerPawn(
    PlayerIndex = 0
)
→ CastTo BP_BountyHunter
→ BountyHunterRef = As BP_BountyHunter

DefaultYokeRotation = Yoke.GetRelativeRotation()
DefaultHeadRotation = Head.GetRelativeRotation()

Tick:
EventTick
→ UpdateTracking(self)

```

### UpdateTracking

```

BP_StudioMovingHead / UpdateTracking

UpdateTracking:

TargetLocation = BountyHunterRef.Mesh.GetSocketLocation("spine_04")
LightLocation = StaticMeshLens.GetWorldLocation()

Distance = Distance(
    V1 = LightLocation,
    V2 = TargetLocation
)

LookRotation = FindLookAtRotation(
    Start = LightLocation,
    Target = TargetLocation
)

TargetYaw = LookRotation.Yaw
TargetPitch = LookRotation.Pitch

Yoke.SetWorldRotation(
    Roll = 180.0,
    Pitch = 0.0,
    Yaw = TargetYaw
)

Head.SetWorldRotation(
    Roll = 0.0,
    Pitch = TargetPitch - 89.0,
    Yaw = TargetYaw
)

ZoomValue = MapRangeClamped(
    Value = Distance,
    InRangeA = 380.0,
    InRangeB = 850.0,
    OutRangeA = 5.5,
    OutRangeB = 2.2
)

ZoomComponent.SetValueNoInterp(
    NewValue = ZoomValue
)

Intensity = MapRangeClamped(
    Value = Distance,
    InRangeA = 380.0,
    InRangeB = 850.0,
    OutRangeA = 30000.0,
    OutRangeB = 200000.0
)

SpotLight.SetIntensity(
    NewIntensity = Intensity
)

```

## ApplyTrackingLightState

```

BP_StudioMovingHead / ApplyTrackingLightState

ApplyTrackingLightState(
    BeamVisible,
    Dimmer,
    BeamColor
)

StaticMeshBeam.SetVisibility(
    NewVisibility = BeamVisible
)

Branch(BeamVisible)

True:
DimmerComponent.SetValueNoInterp(
    NewValue = Dimmer
)

BeamColorRGB = BreakColor(BeamColor)

StaticMeshBeam.SetVectorParameterValueOnMaterials(
    ParameterName = "DMX Color",
    ParameterValueX = BeamColorRGB.R,
    ParameterValueY = BeamColorRGB.G,
    ParameterValueZ = BeamColorRGB.B
)

False:
→ 아무 처리 없음

```

## BP\_StudioMovingHead\_NoTracking

### EventGraph

```

BP_StudioMovingHead_NoTracking / EventGraph

DancingLight(Move):

```

```

Branch(Move)

True:
SpotLight.SetIntensity(0.5)

StartHeadRotation =
    Rotator(
        Roll = 0.0,
        Pitch = -30.0,
        Yaw = 0.0
    )

TargetHeadRotation =
    Rotator(
        Roll = 0.0,
        Pitch = -90.0,
        Yaw = 0.0
    )

Branch(IsDownFirst)

True:
TL_DancingLight.PlayFromStart()

False:
TL_DancingLight.ReverseFromEnd()

TL_DancingLight.Update:
Head.SetRelativeRotation(
    Lerp(
        A = DefaultHeadRotation,
        B = TargetHeadRotation,
        Alpha = TL_DancingLight.Alpha,
        ShortestPath = true
    )
)

StopDancingLight:
TL_DancingLight.Stop()

False:
StopDancingLight()
StaticMeshHead.SetRelativeRotation(
    Rotator(
        Roll = 0.0,
        Pitch = 0.0,
        Yaw = 0.0
    )
)

SpotLight.SetIntensity(0.0)

```

## ApplyStageLightState

```

BP_StudioMovingHead_NoTracking / ApplyStageLightState

ApplyStageLightState(
    BeamVisible,
    BeamColor,
    LightColor
)

StaticMeshBeam.SetVisibility(BeamVisible)

Branch(BeamVisible)

True:
StaticMeshBeam.SetVectorParameterValueOnMaterials(
    ParameterName = "DMX Color",
    Value = BeamColor
)

SpotLight.SetLightColor(LightColor, SRGB = true)

False:
별도 처리 없음

```

## UI

### WBP\_Operator 역할

- **WBP\_Operator** : Virtual Performer의 Runtime 기능을 실시간으로 조작하기 위한 Operator UI
- 단순히 Button Click으로 Blueprint 함수를 직접 실행하는 UI가 아니라 **현재 Runtime State**를 표시하고, 실행 가능한 기능을 제한하며, **Operator의 명령을 BP\_StudioManager에 전달하는 인터페이스**로 구성했다
- 주요 조작 영역
  - Preset
  - Character State
  - Gesture

- Appearance Preset
- Position
- Camera
- Stage View
- Lighting
- Look Tracking
- Reset
- 각 버튼은 단순 On / Off 표시가 아니라 Runtime 상태에 따라 **Selected**, **Default**, **Disabled** 상태를 구분하도록 했다
- 실제 Character, Camera, Lighting 등을 Widget에서 직접 수정하지 않고 대부분의 State 변경을 **BP\_StudioManager**를 통해 전달했다

```
Operator Input
→ WBP_Operator
→ BP_StudioManager
→ Runtime State 갱신
→ Character / Camera / Lighting / Appearance System
→ 변경된 State를 UI에 다시 반영
```

## UI 초기화와 Reference 연결

- **Construct**에서 Operator UI가 사용할 주요 Runtime Actor Reference를 가져왔다
  - **BP\_StudioManager**
  - **BP\_PlayerController**
  - **BP\_BountyHunter**
- StudioManager와 PlayerController, Character의 Event Dispatcher에 필요한 UI Event를 바인딩했다
- UI가 매 Frame 모든 Runtime 상태를 직접 확인하는 방식보다 **상태가 실제로 변경되는 시점에 필요한 UI만 갱신**하도록 일부 Event Dispatcher를 사용했다

```
Event Construct
→ StudioManagerRef
→ PlayerControllerRef
→ BountyHunterRef
→ Runtime Event Bind
```

현재 코드에서는 **OnPresetChanged**, **OnGestureEnd**, **OnMoveToClicked**, **OnChangeMovePhase**, **OnCameraRotateStateChange** 등을 UI에 연결하고 있다 붙여넣은 텍스트(1) 붙여넣은 텍스트(1)

## Preset UI

### Built-in Preset

- Preset은 **Standby**, **Presentation**, **Performance** 3종으로 구성
- Mouse Button뿐 아니라 Keyboard **1**, **2**, **3**으로도 실행 가능
- Preset은 단순 UI 표시값이 아니라 **CharacterState**, **CameraState**, **LightingState** 조합을 한 번에 적용하는 상위 명령

```
[1] Standby
[2] Presentation
[3] Performance
```

각 Button과 Keyboard 입력은 동일한 **ClickedPreset** 흐름으로 연결했다 붙여넣은 텍스트(1)

### Preset 실행 조건

- 이동 중이나 Gesture 실행 중에 Character State까지 포함된 Preset을 변경하면 현재 Animation / Movement와 충돌할 수 있으므로 실행 가능 조건을 검사했다

```
CanExecutePreset
=
!IsGesturing
&& !IsMoving
```

- 조건을 통과한 경우에만 새로운 Preset을 적용
- Preset 적용 후 현재 CharacterState를 기준으로 Gesture Button 사용 가능 여부도 함께 갱신
- Preset 실행 시 Stage View는 해제하고 일반 Camera State로 복귀하도록 구성

```
Preset Click
→ 실행 가능 상태 확인
→ Preset State 적용
→ Character / Camera / Lighting 갱신
→ Gesture Button 상태 갱신
→ Stage View OFF
```

현재 **ClickedPreset**에서도 Gesture와 이동 상태를 먼저 검사한 뒤 Preset을 실행하도록 구성되어 있다 붙여넣은 텍스트(1)

## Preset Button과 실제 Runtime State 동기화

- Preset Button은 마지막으로 Click한 버튼을 계속 Selected로 유지하는 방식이 아님
- `BP_StudioManager.MatchPresetState`에서 실제 `CharacterState + CameraState + LightingState` 조합을 다시 판정
- 판정 결과는 `OnPresetChanged`를 통해 UI로 전달
- UI는 전달받은 실제 `CurrentPresetState`를 기준으로 Button Visual과 `Current Preset` Text를 다시 갱신

```
Runtime State 변경
→ StudioManager.MatchPresetState
→ Built-in Preset / Custom 판정
→ OnPresetChanged
→ WBP_Operator
→ Preset Button Visual 갱신
→ CurrentPreset Text 갱신
```

따라서

```
Presentation
→ Camera를 Close로 수동 변경
→ CurrentPreset = Custom

Camera를 다시 Upper로 변경
→ 실제 State 조합이 Presentation과 다시 일치
→ CurrentPreset = Presentation
```

처럼 동작할 수 있다

UI에서 `OnPresetChanged`를 받아 `SetPresetButtonState`와 현재 Preset Text를 갱신하도록 되어 있다 불여넣은 텍스트(1)

이 부분은 발표에서도 중요한 UI 설계 포인트

## Preset Button Visual

- Preset Button은 선택 상태를 명확히 보여주기 위해 일반 Common Button보다 별도의 Visual 처리 함수를 사용했다
- 선택 상태에서는
  - Index Color 변경
  - Preset Name Color 변경
  - Preset 구성 Text Color 변경
  - Selected Button Style 적용
  - Glow 표시
- 선택되지 않은 Preset은 Default Visual로 복귀

```
SetPresetButtonState
→ SetPresetButtonVisual
  └─ Selected
    └─ SetSelectedPresetButton
  └─ Unselected
    └─ SetUnselectedPresetButton
```

같은 Selected / Unselected Node를 세 Preset Button마다 반복해서 작성하지 않고 공통 함수로 묶었다 불여넣은 텍스트(1) 불여넣은 텍스트(1)

## Character State UI

- Character State는 `Idle`, `Talk`, `Dance`로 구성
- Button Click은 공통 `ClickedCharacterState` 함수로 연결
- CharacterState를 바로 변경하기 전에 현재 Gesture와 Movement 상태 확인

```
CanChangeCharacterState
=
!IsGesturing
&& !IsMoving
```

- 이동 중이나 Gesture 실행 중에는 CharacterState 변경을 허용하지 않아 Animation State 충돌을 줄였다

```
Character State Click
→ 현재 이동 / Gesture 확인
→ State 변경
→ Character State Button Visual 갱신
→ Gesture Button 가능 여부 갱신
→ CurrentPreset 재판정
```

현재 코드에서도 State 변경 후 `MatchPresetState`를 호출해 실제 State 조합에 맞게 Preset 표시를 다시 계산한다 불여넣은 텍스트(1)

## Gesture UI

- Gesture는 Wave, Point, CrossHands, Pose, Clap, Laugh 6종으로 구성
- Gesture마다 같은 실행 구조를 사용하고 Button별 Hover Text Color만 개별 처리
- Gesture Button을 선택하면 StudioManager 에 GestureState 를 전달하고 Montage 실행을 요청

```
Gesture Button
→ SetGestureStateButtonState
→ UpdateGestureState
→ PlayGesture
→ Gesture Button 전체 Disabled
```

Gesture Button 입력은 모두 동일한 ClickedGesture 흐름으로 연결되어 있다 불여넣은 텍스트(1) 불여넣은 텍스트(1)

## Gesture 중 중복 입력 방지

- Gesture가 시작되면 다른 Gesture Button을 모두 Disabled
- Gesture Montage가 완료되거나 Interrupted되면 StudioManager.OnGestureEnd Event를 받음
- IsGesturing = false 가 되면 Gesture Button을 다시 활성화

```
Gesture 시작
→ IsGesturing = true
→ Gesture Buttons Disabled

Montage 완료 / 중단
→ IsGesturing = false
→ OnGestureEnd
→ Gesture Buttons Enabled
```

- 일회성 Animation이 중첩 재생되면서 Pose가 꼬이는 것을 방지하기 위한 처리

UI에서는 OnGestureEnd 를 받아 전체 Gesture Button 활성 상태를 다시 변경한다 불여넣은 텍스트(1)

## CharacterState에 따른 Gesture 제한

- Gesture는 모든 Character State에서 허용하지 않도록 했다
- Talk, Dance 상태에서는 Gesture Button을 비활성화
- Idle 상태에서만 일반 Gesture 사용 가능하도록 구성

```
Idle
→ Gesture 0

Talk
→ Gesture X

Dance
→ Gesture X
```

- 지속 Animation과 Gesture Montage가 동시에 복잡하게 섞이는 것을 제한하고 Runtime 규칙을 단순화하기 위한 선택

## 공통 Button Visual System

UI Button 종류가 많아지면서 각각의 Button에 Selected / Default / Disabled Style Logic을 반복해서 만드는 대신 공통 함수를 만들었다

```
SetCommonButtonVisual
→ IsSelected?
├─ true
│   → SetSelectedCommonButton
└─ false
    → SetUnselectedCommonButton
```

공통 대상으로

- CharacterState
- Position
- Camera
- Lighting

를 처리한다 불여넣은 텍스트(1)

## Button Type별 Disabled 규칙

같은 Common Button이라도 기능마다 실행 제한 조건이 다르기 때문에 E\_CommonButton 으로 Button Type을 구분했다

## Character State

```
Disabled
=
IsMoving
|| IsGesturing
```

## Position

```
Disabled
=
CharacterState == Talk
|| CharacterState == Dance
|| IsGesturing
```

## Camera

```
StageViewActive
→ 일반 Camera Button Disabled Visual
```

## Lighting

- Character Movement나 Gesture와 독립적으로 변경 가능
- 별도의 Disabled 조건 없이 Selected / Default 상태 표시

이 규칙은 단순히 Button 색만 바꾸는 게 아니라 **현재 Runtime에서 어떤 명령을 실행할 수 있는지를 Operator에게 UI로 알려주는 역할**

실제 `SetSelectedCommonButton`, `SetUnselectedCommonButton` 에서도 Button Type별로 서로 다른 Disabled 조건을 검사하고 있다 불여넣은 텍스트(1) 불여넣은 텍스트(1)

## Position UI

### 고정 Position

최종 Position은 다음 위치를 사용

- `Left`
- `Center`
- `Right`
- `Back`
- `Front`
- `Custom`

UI Button으로 직접 선택하는 것은

```
Left
Center
Right
Back
Front
```

이며 `Custom`은 Click-to-Move로 생성되는 위치

현재 UI에도 다섯 개의 고정 Position Button이 연결되어 있다 불여넣은 텍스트(1)

### Position 실행 조건

- Position 변경 전 `CanMove()`로 현재 Character가 이동 가능한 상태인지 확인
- `Talk`, `Dance`, Gesture 상태에서는 이동 제한

```
CanMove
=
CharacterState != Talk
&& CharacterState != Dance
&& !IsGesturing
```

불여넣은 텍스트(1)

### Position 변경과 Look Tracking

- Character 이동 중 Mouse Look까지 동시에 적용하면 Animation Blend와 Character 방향이 복잡해지기 때문에 Position 이동을 시작할 때 `SetLookTrackingOff()` 호출
- Look Tracking Toggle도 UI에서 OFF 상태로 동기화
- 이동 중 CharacterState 자체는 유지하지만 실제 Look Tracking은 끄고 Movement에 집중하도록 했다

```
Position Click
→ CanMove
→ Look Tracking OFF
→ 이전 Position 저장
→ 새 Position 적용
→ Character 이동
```

### 같은 Position 재선택 처리

- 이전 Position과 새로운 Position을 비교

- Position이 실제로 변경된 경우 이동 중 Gesture Button을 Disabled
- `Custom` Position은 같은 Enum 값을 사용해도 실제 World 위치가 매번 다를 수 있기 때문에 항상 새로운 이동으로 처리

```
PositionChanged
=
PrePositionState != CurrentPositionState

또는

CurrentPositionState == Custom
```

→ 이동 중 Gesture Button 비활성화

현재 `ClickedPosition` 이 이 흐름으로 구성되어 있다 붙여넣은 텍스트(1)

이 부분은 `Custom` 을 단순 Enum 비교만으로 판단하지 않았다는 점에서 기록해둘 만함

## Click-to-Move와 UI 영역 분리

- Operator UI가 화면 대부분을 차지하기 때문에 일반 Mouse Click을 모두 World 이동 입력으로 처리하면 UI Button을 누를 때도 Character가 이동할 수 있음
- 이를 막기 위해 중앙 Stage Preview에서 실제로 이동 입력을 허용할 영역을 `BorderViewportInputArea` 로 따로 지정했다
- UI Tick에서 해당 영역이 Hover 상태인지 확인하고 PlayerController의 `CanClickMove` 를 갱신

```
BorderViewportInputArea.IsHovered()
→ PlayerController.CanClickMove
```

현재 코드에서는 이 값이 Tick에서 계속 PlayerController로 전달된다 붙여넣은 텍스트(1)

```
UI Panel 위
→ CanClickMove = false

Stage Preview 입력 영역
→ CanClickMove = true
```

- UI 입력과 World Click 이동이 같은 Mouse를 사용하기 때문에 **화면 영역을 기준으로 두 입력을 분리한 것**

이건 Click-to-Move 작업 중 실제로 문제가 있었던 부분이라 남기는 게 좋음

## Click-to-Move와 Position UI 동기화

- PlayerController에서 정상적인 Click-to-Move Goal이 생성되면 `OnMoveToClicked` Event Dispatcher 호출
- UI는 해당 Event를 받아 `PositionState = Custom` 으로 변경

```
World Click 성공
→ PlayerController.OnMoveToClicked
→ WBP_Operator.SetCustomPosition
→ ClickedPosition(Custom)
```

따라서 Character가 자유 위치로 이동했는데 UI에는 이전 `Left / Center / Right` Button이 선택되어 있는 상태가 남지 않도록 했다

현재 이 Event 연결은 Construct에서 설정되어 있다 붙여넣은 텍스트(1)

## Camera UI

- Camera State는 `Full`, `Upper`, `Close`
- Button을 누르면 `SetCameraButtonState` 에서 StudioManager에 Camera State 변경 요청
- 이후 UI Button Visual도 같은 State 기준으로 변경

```
Full
Upper
Close
→ UpdateCameraState
→ Camera Runtime 적용
→ Button Visual 갱신
```

Camera 버튼 변경 후 Preset 조합도 다시 판단한다 붙여넣은 텍스트(1)

## Stage View Toggle

- Stage View는 일반 Camera State가 아니라 Camera Override이므로 별도의 Toggle로 분리했다
- Toggle ON / OFF가 즉시 튀지 않도록 Toggle 자체에도 Animation을 추가했다

### ON

```
Toggle ON
→ Toggle Animation Forward
→ StageViewActive = true
```

```
→ StudioManager.SetStageView(true)
→ Camera Button 상태 갱신
```

## OFF

```
Toggle OFF
→ Toggle Animation Reverse
→ StageViewActive = false
→ StudioManager.SetStageView(false)
→ 마지막 CurrentCameraState 복귀
```

현재 Stage View Toggle에서 Forward / Reverse Animation과 `SetStageView` 호출을 함께 처리한다 붙여넣은 텍스트(1)

## Stage View 중 Camera Button 표시

- Stage View는 `Full / Upper / Close` 중 하나가 아니기 때문에 기존 Camera State 자체는 계속 유지
- Stage View 중에는 일반 Camera Button을 Disabled Visual로 표시
- Stage View를 끄면 마지막으로 유지되던 `CurrentCameraState` 기준 Button을 다시 복구

⇒ Camera State와 Stage View Override를 UI에서도 서로 다른 개념으로 표현

## Camera Rotate와 UI 연동

- Mouse Drag로 Camera Rotate를 시작하면 PlayerController의 `OnCameraRotateStateChange` Event를 UI가 받는다
- Camera Rotate 중에는 Look Tracking과 Camera Drag가 동시에 Mouse 방향을 사용하지 않도록 `SetLookTrackingOff()` 호출

```
Camera Rotate Start
→ CanRotate = true
→ Look Tracking OFF
```

현재 `OnCameraRotateStateChange` 에 이 처리가 연결되어 있다 붙여넣은 텍스트(1)

## Lighting UI

- Lighting State는
  - `Neutral`
  - `Focus`
  - `Stage`
- 각 Button은 `SetLightingButtonState` 를 통해 StudioManager에 State 변경을 전달
- 변경 후 `MatchPresetState()` 를 호출해 Character / Camera / Lighting 조합이 Built-in Preset과 일치하는지 다시 판단

```
Lighting Button
→ Lighting State 변경
→ Lighting System 적용
→ CurrentPreset 재판정
```

현재 각 Lighting Button에서 State 변경 후 `MatchPresetState` 를 호출한다 붙여넣은 텍스트(1)

- Lighting은 Character 이동이나 Gesture와 직접 충돌하지 않기 때문에 Runtime 중에도 변경 가능하도록 구성

## Look Tracking Toggle

- Mouse 기반 Aim Offset을 켜고 끄는 기능
- 단순 Button이 아니라 ON / OFF 상태를 시각적으로 보여주는 Toggle로 구성
- Toggle Animation을 Forward / Reverse로 사용해 상태 전환 표현

## 변경 가능 조건

```
CanChangeLookTracking
=
!IsMoving
&& !CanRotate
```

- Character 이동 중에는 Look Tracking 변경 제한
- Camera Rotate 중에도 같은 Mouse 입력을 사용하므로 변경 제한

현재 Look Tracking Toggle에서 이 조건을 검사한다 붙여넣은 텍스트(1)

## Look Tracking ON

```
Toggle ON
→ Toggle Animation Forward
→ LookTrackingActive = true
→ StudioManager.UpdateLookTrackingState(true)
```

```
→ PlayerController
→ AimOffset 계산
```

## Look Tracking OFF

```
Toggle OFF
→ Toggle Animation Reverse
→ LookTrackingActive = false
→ StudioManager.UpdateLookTrackingState(false)
→ AimOffset Neutral 복귀
```

- Character Movement나 Camera Rotate 같은 충돌 상황에서는 UI도 실제 Runtime 상태에 맞춰 OFF로 변경하도록 `SetLookTrackingOff` 공통 함수를 만들었다

## Appearance Preset UI

- Character Appearance는 Color Picker를 직접 조작하는 방식 대신 미리 구성한 5개의 Appearance Preset으로 제공
- `Preset_01 ~ Preset_05`
- Appearance Button을 선택하면 StudioManager의 `CurrentAppearancePresetState`를 갱신하고 해당 DataTable Preset을 Character Material에 적용

```
Appearance Button
→ AppearancePresetState
→ StudioManager
→ DT_AppearancePresetData
→ BountyHunter Material
```

- UI에는 선택된 Appearance Preset을 BackFrame으로 표시
- Preset마다 Hair, Clothes, Eye, Lip, Skin 조합을 한 번에 적용

현재 5개 Appearance Preset Button과 선택 표시 함수가 분리되어 있다    붙여넣은 텍스트(1)    붙여넣은 텍스트(1)

- Appearance는 CharacterState / CameraState / LightingState와 독립적인 Runtime Control이므로 변경해도 `CurrentPreset` 판정에는 영향을 주지 않음

## Reset

- Reset은 Standby Preset 버튼을 누르는 것과 같지 않고 **Operator System** 전체를 기준 상태로 복구하는 명령
- Mouse Button뿐 아니라 Keyboard `R`로도 실행 가능

현재 Reset Button과 `R` 입력이 같은 `ResetAllState`로 연결된다    붙여넣은 텍스트(1)

### Reset 대상

```
Preset
→ Standby

CharacterState
→ Idle

Position
→ Center

Camera
→ Full

Lighting
→ Neutral

Appearance
→ Preset_01

Stage View
→ OFF

Look Tracking
→ OFF
```

`ResetAllState`에서 실제로 이 순서로 State를 초기화한다    붙여넣은 텍스트(1)

- 각 값을 내부 변수만 바꾸는 것이 아니라 기존 State 변경 Function을 재사용해서 Runtime System과 UI Visual이 함께 초기화되도록 했다
- Reset 전용으로 같은 Logic을 다시 구현하지 않고 기존 Update / Set Function을 재사용

이게 중요한 부분

## Runtime State와 UI Visual 동기화

UI 구현에서 가장 중요하게 잡은 기준은 **Button의 표시 상태와 실제 Runtime State가 서로 달라지지 않도록 하는 것**

```
Runtime State
→ Selected / Default / Disabled Visual

X
Button을 눌렀다는 이유만으로 Selected 유지
```

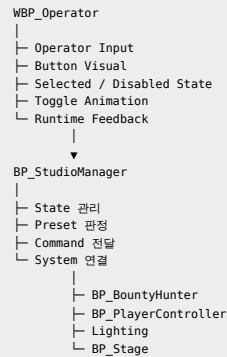
예를 들어

- Preset을 누른 뒤 Camera를 변경하면 CurrentPreset이 Custom
- Click-to-Move를 하면 Position은 Custom
- Stage View를 켜면 Camera Button은 기존 State를 유지하지만 Disabled 표시
- Gesture 실행 중에는 Gesture Button Disabled
- Character 이동 중에는 CharacterState Button Disabled
- Look Tracking이 강제로 꺼지는 상황에서는 Toggle Visual도 OFF로 변경

즉 UI를 단순 입력 장치라 아니라 **현재 Runtime 상태를 확인하는 Operator Feedback Interface**로 사용했다

## UI와 Runtime Logic 역할 분리

전체 흐름은 다음처럼 정리 가능



- UI Widget 내부에 Character Animation Asset이나 Material Parameter, Light 값을 직접 넣지 않음
- UI는 어떤 상태를 요청했는지 전달하고 실제 Runtime 적용은 각 System이 담당
- UI Visual은 실제 Runtime State를 기준으로 갱신

## Operator UI의 입력 충돌 규칙

여러 기능을 동시에 조작할 수 있는 UI라서 기능별 실행 가능 조건을 따로 정했다

| Runtime 상태      | Character State | Gesture | Position          | Camera                      | Lighting | Look Tracking |
|-----------------|-----------------|---------|-------------------|-----------------------------|----------|---------------|
| 일반 상태           | O               | O       | O                 | O                           | O        | O             |
| 이동 중            | X               | X       | 이동 중 Target 변경 가능 | O                           | O        | X             |
| Gesture 중       | X               | X       | X                 | O                           | O        | 제한            |
| Camera Rotate 중 | O               | O       | Click 이동 제한       | Rotate                      | O        | X             |
| Stage View      | O               | 조건에 따름  | 조건에 따름            | Camera State 유지 / Visual 제한 | O        | X             |

다만 이 표는 **최종 코드 기준으로 실제 구현된 제약만 설명할 때 사용하는 게 좋음**

초기 기획 단계에서 생각했던 규칙과 지금 최종 UI 코드가 일부 달라졌기 때문에 발표자료에서는 현재 Build가 실제로 어떻게 동작하는지를 우선하면 됨

## 구현 과정에서 중요했던 부분

### 1. UI Button 상태와 실제 Runtime State 분리

초기에는 Button을 누르면 해당 Button만 선택 표시하면 되는 것처럼 보였지만, Preset 이후 Camera / Lighting / CharacterState를 개별 변경할 수 있기 때문에 **마지막으로 누른 Button과 실제 Runtime State가 달라질 수 있다는 문제**가 생김

최종적으로

```

User Click
→ Runtime State 변경
→ State 다시 편집
→ UI Visual 갱신
  
```

방식으로 정리

특히 CurrentPreset을 실제 State 조합으로 다시 계산하도록 바꾼 이유가 이것

### 2. UI와 World Click 충돌

Click-to-Move를 추가하면서 UI도 Mouse Click을 사용하기 때문에 Button Click이 World 이동으로 전달될 수 있는 문제가 발생

최종적으로

```

BorderViewportInputArea
→ Hover 중일 때만 CanClickMove = true
  
```

방식으로 중앙 Stage 영역만 Click-to-Move 입력을 허용하도록 했다

### 3. Camera Rotate와 Click / Look Tracking 충돌

Mouse 입력 하나를

- Click-to-Move
- Camera Rotate
- Look Tracking

세 기능에서 사용하기 때문에 동시에 실행되지 않도록 상태값을 분리했다

```
Camera Rotate 중
→ Click-to-Move X
→ Look Tracking OFF

Stage View
→ Camera Rotate X
→ Look Tracking X
```

- Input Action 자체보다 **현재 Operator Mode**를 기준으로 같은 **Mouse** 입력의 의미를 바꾼 것

### 4. Selected와 Disabled를 별개 상태로 처리

- Button이 현재 State를 나타내는 Selected 상태여도 현재 Runtime 조건에서는 조작할 수 없을 수 있음
- 예를 들어 현재 **Idle** 이 Selected지만 Character가 이동 중이면 CharacterState Button은 Selected이면서 동시에 Disabled 성격을 가짐
- 이를 처리하기 위해 Button의 선택 여부뿐 아니라 **ButtonType** 별 Runtime 조건까지 같이 확인해서 Visual을 결정

이게 **SetCommonButtonVisual**을 단순 Selected / Unselected 함수보다 복잡하게 만든 이유

## UI 구조 정리

```
WBP_Operator
├── PRESET
│   ├── Standby
│   ├── Presentation
│   ├── Performance
│   └── CurrentPreset
├── ANIMATION
│   ├── CharacterState
│   │   ├── Idle
│   │   ├── Talk
│   │   └── Dance
│   └── Gesture
│       ├── Wave
│       ├── Point
│       ├── CrossHands
│       ├── Pose
│       ├── Clap
│       └── Laugh
├── APPEARANCE
│   └── Preset 01 ~ 05
├── POSITION
│   ├── Left
│   ├── Center
│   ├── Right
│   ├── Back
│   ├── Front
│   └── Custom / Click-to-Move
├── CAMERA
│   ├── Full
│   ├── Upper
│   ├── Close
│   └── Stage View
├── LIGHTING
│   ├── Neutral
│   ├── Focus
│   └── Stage
├── LOOK_TRACKING
│   └── ON / OFF
└── SYSTEM
    └── Reset
```

## 구현 의도 정리

**WBP\_Operator**에서 가장 중요하게 잡은 것은 기능 개수를 많이 배치하는 것이 아니라 **현재 Virtual Performer** 상태를 **Operator**가 한 화면에서 이해하고 안전하게 변경할 수 있도록 하는 것

```
Operator가 명령
→ 실행 가능 여부 확인
→ Runtime State 변경
→ 실제 System 처리
→ UI가 결과 State 반영
```

즉

- Button = 단순 함수 실행기 X
- **Button = Runtime State를 변경하고 현재 State를 다시 표시하는 Operator Control**

이라는 방향으로 구성했다

## WBP\_Operator

### EventGraph

```
WBP_Operator / EventGraph

Construct:
EventConstruct
→ Sequence

Then0:
StudioManagerRef =
    GetActorOfClass(
        ActorClass = BP_StudioManager
    )

PlayerControllerRef =
    CastToBP_PlayerController(
        GetPlayerController(PlayerIndex = 0)
    )

BountyHunterRef =
    CastToBP_BountyHunter(
        GetPlayerPawn(PlayerIndex = 0)
    )

Then1:
Bind StudioManagerRef.OnPresetChanged
→ HandlePresetChanged(CurrentPresetState)

HandlePresetChanged:
SetPresetButtonState(
    PresetState = CurrentPresetState
)

TxtCurrentPresetValue.SetText(
    ToText(CurrentPresetState)
)

Bind StudioManagerRef.OnGestureEnd
→ CE_GestureEnd(IsGesturing)

CE_GestureEnd:
SetEnableAllGestureButton(
    IsEnabled = !IsGesturing
)

SetPositionButtonState(
    PositionState = StudioManagerRef.CurrentPositionState
)

ClickedCharacterState(
    CharacterState = StudioManagerRef.CurrentCharacterState
)

Then2:
Bind PlayerControllerRef.OnMoveToClicked
→ SetCustomPosition

SetCustomPosition:
ClickedPosition(
    PositionState = Custom
)

Bind BountyHunterRef.OnChangeMovePhase
→ EndMoving

EndMoving:
Switch(BountyHunterRef.MovementPhase)

Idle:
SetCharacterStateButtonState(
    CharacterState = Idle
)
→ SetEnableAllGestureButton(
    IsEnabled = true
)

TurningToPath:
(no connection visible)

Moving:
(no connection visible)

TurningToCamera:
(no connection visible)

Landing:
(no connection visible)

CameraRotateState:
```

```

Bind_PlayerControllerRef.OnCameraRotateStateChange
→ CustomEvent(CanRotate)

CustomEvent(CanRotate):
CanRotate = InputCanRotate
→ Branch(CanRotate)

True:
SetLookTrackingOff()

False:
(no connection visible)

Reset:

Btn_Reset.OnHovered
→ TxtReset.SetColorAndOpacity(White)

Btn_Reset.OnReleased
→ TxtReset.SetColorAndOpacity(White)

Btn_Reset.OnUnhovered
→ TxtReset.SetColorAndOpacity(Pink)

Btn_Reset.OnPressed
→ TxtReset.SetColorAndOpacity(Pink)

Btn_Reset.OnClicked
→ ResetAllState()

Keyboard R Pressed
→ ResetAllState()

Tick:
EventTick
→ PlayerControllerRef.CanClickMove =
    BorderViewportInputArea.IsHovered()

```

## AppearanceGraph

```

WBP_Operator / AppearanceGraph

AppearancePresetButtons:

Btn_Preset_1.OnClicked
→ SetAppearancePresetButtonState(
    AppearancePresetState = Preset_01
)

Btn_Preset_2.OnClicked
→ SetAppearancePresetButtonState(
    AppearancePresetState = Preset_02
)

Btn_Preset_3.OnClicked
→ SetAppearancePresetButtonState(
    AppearancePresetState = Preset_03
)

Btn_Preset_4.OnClicked
→ SetAppearancePresetButtonState(
    AppearancePresetState = Preset_04
)

Btn_Preset_5.OnClicked
→ SetAppearancePresetButtonState(
    AppearancePresetState = Preset_05
)

```

## PresetGraph

```

WBP_Operator / PresetGraph

PresetButtons:

Btn_Preset_Standby.OnClicked
OR
Keyboard 1.Pressed
→ ClickedPreset(
    PresetState = Standby
)

Btn_Preset_Presentation.OnClicked
OR
Keyboard 2.Pressed
→ ClickedPreset(
    PresetState = Presentation
)

Btn_Preset_Performance.OnClicked
OR
Keyboard 3.Pressed
→ ClickedPreset(
    PresetState = Performance
)

```

## LightingGraph

```
WBP_Operator / LightingGraph

LightingButtons:

Btn_Lighting_Neutral.OnClicked
→ SetLightingButtonState(
    LightingState = Neutral
)
→ StudioManagerRef.MatchPresetState()

Btn_Lighting_Focus.OnClicked
→ SetLightingButtonState(
    LightingState = Focus
)
→ StudioManagerRef.MatchPresetState()

Btn_Lighting_Stage.OnClicked
→ SetLightingButtonState(
    LightingState = Stage
)
→ StudioManagerRef.MatchPresetState()
```

## CharacterStateGraph

```
WBP_Operator / CharacterStateGraph

CharacterStateButtons:

Btn_CharacterState_Idle.OnClicked
→ ClickedCharacterState(
    CharacterState = Idle
)

Btn_CharacterState_Talk.OnClicked
→ ClickedCharacterState(
    CharacterState = Talk
)

Btn_CharacterState_Dance.OnClicked
→ ClickedCharacterState(
    CharacterState = Dance
)
```

## PositionGraph

```
WBP_Operator / PositionGraph

PositionButtons:

Btn_Position_Left.OnClicked
→ ClickedPosition(
    PositionState = Left
)

Btn_Position_Center.OnClicked
→ ClickedPosition(
    PositionState = Center
)

Btn_Position_Right.OnClicked
→ ClickedPosition(
    PositionState = Right
)

Btn_Position_Back.OnClicked
→ ClickedPosition(
    PositionState = Back
)

Btn_Position_Front.OnClicked
→ ClickedPosition(
    PositionState = Front
)
```

## CameraGraph

```
WBP_Operator / CameraGraph

CameraButtons:

Btn_Camera_Full.OnClicked
→ SetCameraButtonState(
    CameraState = Full
)
→ StudioManagerRef.MatchPresetState()

Btn_Camera_Upper.OnClicked
→ SetCameraButtonState(
    CameraState = Upper
)
→ StudioManagerRef.MatchPresetState()

Btn_Camera_Close.OnClicked
```

```

→ SetCameraButtonState(
    CameraState = Close
)
→ StudioManagerRef.MatchPresetState()

StageViewToggle:

CB_StageView.OnCheckStateChanged(IsChecked)
→ Branch(IsChecked)

True:
PlayAnimation(
    Animation = StageViewToggleOffToOn,
    StartAtTime = 0.0,
    NumLoopsToPlay = 1,
    PlayMode = Forward,
    PlaybackSpeed = 1.0
)
→ StageViewActive = true
→ StudioManagerRef.SetStageView(
    Active = StageViewActive
)
→ SetCameraButtonState(
    CameraState = StudioManagerRef.CurrentCameraState
)

False:
PlayAnimation(
    Animation = StageViewToggleOffToOn,
    StartAtTime = 0.0,
    NumLoopsToPlay = 1,
    PlayMode = Reverse,
    PlaybackSpeed = 1.0
)
→ StageViewActive = false
→ StudioManagerRef.SetStageView(
    Active = StageViewActive
)
→ SetCameraButtonState(
    CameraState = StudioManagerRef.CurrentCameraState
)

```

## GestureGraph

```

WBP_Operator / GestureGraph

Wave:

Btn_Gesture_Wave.OnClicked
→ ClickedGesture(
    GestureState = Wave
)

Btn_Gesture_Wave.OnHovered
→ TxtGestureWave.SetColorAndOpacity(
    HoveredGestureTextColor
)

Btn_Gesture_Wave.OnUnhovered
→ TxtGestureWave.SetColorAndOpacity(
    UnHoveredGestureTextColor
)

Point:

Btn_Gesture_Point.OnClicked
→ ClickedGesture(
    GestureState = Point
)

Btn_Gesture_Point.OnHovered
→ TxtGesturePoint.SetColorAndOpacity(
    HoveredGestureTextColor
)

Btn_Gesture_Point.OnUnhovered
→ TxtGesturePoint.SetColorAndOpacity(
    UnHoveredGestureTextColor
)

CrossHands:

Btn_Gesture_CrossHands.OnClicked
→ ClickedGesture(
    GestureState = CrossHands
)

Btn_Gesture_CrossHands.OnHovered
→ TxtGestureCrossHands.SetColorAndOpacity(
    HoveredGestureTextColor
)

Btn_Gesture_CrossHands.OnUnhovered
→ TxtGestureCrossHands.SetColorAndOpacity(
    UnHoveredGestureTextColor
)

Pose:

```

```

Btn_Gesture_Pose.OnClicked
→ ClickedGesture(
    GestureState = Pose
)

Btn_Gesture_Pose.OnHovered
→ TxtGesturePose.SetColorAndOpacity(
    HoveredGestureTextColor
)

Btn_Gesture_Pose.OnUnhovered
→ TxtGesturePose.SetColorAndOpacity(
    UnHoveredGestureTextColor
)

Clap:

Btn_Gesture_Clap.OnClicked
→ ClickedGesture(
    GestureState = Clap
)

Btn_Gesture_Clap.OnHovered
→ TxtGestureClap.SetColorAndOpacity(
    HoveredGestureTextColor
)

Btn_Gesture_Clap.OnUnhovered
→ TxtGestureClap.SetColorAndOpacity(
    UnHoveredGestureTextColor
)

Laugh:

Btn_Gesture_Laugh.OnClicked
→ ClickedGesture(
    GestureState = Laugh
)

Btn_Gesture_Laugh.OnHovered
→ TxtGestureLaugh.SetColorAndOpacity(
    HoveredGestureTextColor
)

Btn_Gesture_Laugh.OnUnhovered
→ TxtGestureLaugh.SetColorAndOpacity(
    UnHoveredGestureTextColor
)

```

## LookTrackingGraph

```

WBP_Operator / LookTrackingGraph

LookTrackingToggle:

CB_LookTracking.OnCheckStateChanged(IsChecked)

CanChangeLookTracking =
    !BountyHunterRef.IsMoving
    && !CanRotate

→ Branch(CanChangeLookTracking)

False:
CB_LookTracking.SetIsChecked(false)

True:
→ Branch(IsChecked)

    True:
    PlayAnimation(
        Animation = MouseLookToggleOffToOn,
        StartAtTime = 0.0,
        NumLoopsToPlay = 1,
        PlayMode = Forward,
        PlaybackSpeed = 1.0
    )
    → LookTrackingActive = true
    → StudioManagerRef.UpdateLookTrackingState(
        LookTrackingActive = true
    )

    False:
    PlayAnimation(
        Animation = MouseLookToggleOffToOn,
        StartAtTime = 0.0,
        NumLoopsToPlay = 1,
        PlayMode = Reverse,
        PlaybackSpeed = 1.0
    )
    → LookTrackingActive = false
    → StudioManagerRef.UpdateLookTrackingState(
        LookTrackingActive = false
    )

```

## SetPresetButtonState

```

WBP_Operator / SetPresetButtonState

Input:
PresetState

UpdatePresetState:
StudioManagerRef.UpdatePresetState(
    NewPresetState = PresetState
)

→ Sequence

Then0:
SetPresetButtonVisual(
    PresetButton = BtnPresetStandby,
    PresetIndex = TxtStandbyIndex,
    PresetName = TxtStandby,
    PresetComponent = TxtStandbyComponent,
    PresetButtonGlow = ImageGlowStandby,
    IsSelected = (PresetState == Standby)
)

Then1:
SetPresetButtonVisual(
    PresetButton = BtnPresetPresentation,
    PresetIndex = TxtPresentationIndex,
    PresetName = TxtPresentation,
    PresetComponent = TxtPresentationComponent,
    PresetButtonGlow = ImageGlowPresentation,
    IsSelected = (PresetState == Presentation)
)

Then2:
SetPresetButtonVisual(
    PresetButton = BtnPresetPerformance,
    PresetIndex = TxtPerformanceIndex,
    PresetName = TxtPerformance,
    PresetComponent = TxtPerformanceComponent,
    PresetButtonGlow = ImageGlowPerformance,
    IsSelected = (PresetState == Performance)
)

Then3:
TxtCurrentPresetValue.SetText(
    ToText(PresetState)
)

RowName =
    EnumToString(PresetState)

GetDataTableRow(
    DataTable = DT_PresetData,
    RowName = RowName
)

RowFound:
SetCharacterStateButtonState(
    CharacterState = OutRow.CharacterState
)
→ SetCameraButtonState(
    CameraState = OutRow.CameraState
)
→ SetLightingButtonState(
    LightingState = OutRow.LightingState
)

RowNotFound:
(no connection visible)

```

## SetSelectedPresetButton

```

WBP_Operator / SetSelectedPresetButton

Input:
PresetButton
PresetIndex
PresetName
PresetComponent
PresetButtonGlow

→ Sequence

Then0:
PresetIndex.SetColorAndOpacity(
    SpecifiedColor
)

→ PresetName.SetColorAndOpacity(
    SpecifiedColor
)

Then1:
PresetComponent.SetColorAndOpacity(
    SpecifiedColor
)

```

```

)

→ PresetButton.SetStyle(
    PresetButtonStyleSelected
)

→ PresetButtonGlow.SetVisibility(
    Visible
)

```

## SetUnselectedPresetButton

```

WBP_Operator / SetUnselectedPresetButton

Input:
PresetButton
PresetIndex
PresetName
PresetComponent
PresetButtonGlow

→ Sequence

Then0:
PresetIndex.SetColorAndOpacity(
    DefaultColor
)

→ PresetName.SetColorAndOpacity(
    DefaultColor
)

Then1:
PresetComponent.SetColorAndOpacity(
    DefaultColor
)

→ PresetButton.SetStyle(
    PresetButtonStyleDefault
)

→ PresetButtonGlow.SetVisibility(
    Hidden
)

```

## SetPresetButtonVisual

```

WBP_Operator / SetPresetButtonVisual

Input:
PresetButton
PresetIndex
PresetName
PresetComponent
PresetButtonGlow
IsSelected

→ Branch(IsSelected)

True:
SetSelectedPresetButton(
    PresetButton = PresetButton,
    PresetIndex = PresetIndex,
    PresetName = PresetName,
    PresetComponent = PresetComponent,
    PresetButtonGlow = PresetButtonGlow
)

False:
SetUnselectedPresetButton(
    PresetButton = PresetButton,
    PresetIndex = PresetIndex,
    PresetName = PresetName,
    PresetComponent = PresetComponent,
    PresetButtonGlow = PresetButtonGlow
)

```

## ClickedPreset

```

WBP_Operator / ClickedPreset

Input:
PresetState

→ Sequence

Then0:

CanExecutePreset =
    !StudioManagerRef.IsGesturing
    && !BountyHunterRef.IsMoving

→ Branch(CanExecutePreset)

```

```

True:
SetPresetButtonState(
    PresetState = PresetState
)

GestureButtonsEnabled =
    !(
        StudioManagerRef.CurrentCharacterState == Talk
        || StudioManagerRef.CurrentCharacterState == Dance
    )

→ SetEnableAllGestureButton(
    IsEnabled = GestureButtonsEnabled
)

→ SetPositionButtonState(
    PositionState = StudioManagerRef.CurrentPositionState
)

→ CB_StageView.SetIsChecked(
    InIsChecked = false
)

False:
(no connection visible)

Then1:

PlayAnimation(
    Animation = StageViewToggleOffToOn,
    StartAtTime = 0.0,
    NumLoopsToPlay = 1,
    PlayMode = Reverse,
    PlaybackSpeed = 1.0
)

→ StageViewActive = false

→ StudioManagerRef.SetStageView(
    Active = StageViewActive
)

→ SetCameraButtonState(
    CameraState = StudioManagerRef.CurrentCameraState
)

```

## SetCharacterStateButtonState

```

WBP_Operator / SetCharacterStateButtonState

Input:
CharacterState

StudioManagerRef.UpdateCharacterState(
    NewCharacterState = CharacterState
)

→ CallOnCharacterStateChanged()
→ Sequence

Then0:
SetCommonButtonVisual(
    Button = BtnCharacterStateIdle,
    Name = TxtCharacterStateIdle,
    Glow = BtnGlowIdle,
    IsSelected = (CharacterState == Idle),
    ButtonType = CharacterState
)

Then1:
SetCommonButtonVisual(
    Button = BtnCharacterStateTalk,
    Name = TxtCharacterStateTalk,
    Glow = BtnGlowTalk,
    IsSelected = (CharacterState == Talk),
    ButtonType = CharacterState
)

Then2:
SetCommonButtonVisual(
    Button = BtnCharacterStateDance,
    Name = TxtCharacterStateDance,
    Glow = BtnGlowDance,
    IsSelected = (CharacterState == Dance),
    ButtonType = CharacterState
)

```

## ClickedCharacterState

```

WBP_Operator / ClickedCharacterState

Input:
CharacterState

```

```

CanChangeCharacterState =
    !StudioManagerRef.IsGesturing
    && !BountyHunterRef.IsMoving

→ Branch(CanChangeCharacterState)

True:
SetCharacterStateButtonState(
    CharacterState = CharacterState
)

GestureButtonsEnabled =
    !(
        CharacterState == Talk
        || CharacterState == Dance
    )

→ SetEnableAllGestureButton(
    IsEnabled = GestureButtonsEnabled
)

→ SetPositionButtonState(
    PositionState = StudioManagerRef.CurrentPositionState
)

→ StudioManagerRef.MatchPresetState()

False:
(no connection visible)

```

## SetPositionButtonState

```

WBP_Operator / SetPositionButtonState

Input:
PositionState

CurrentPositionState = PositionState

→ StudioManagerRef.UpdatePositionState(
    NewPositionState = CurrentPositionState
)

→ Sequence

Then0:
SetCommonButtonVisual(
    Button = BtnPositionCenter,
    Name = TxtPositionCenter,
    Glow = BtnGlowCenter,
    IsSelected = (PositionState == Center),
    ButtonType = Position
)

Then1:
SetCommonButtonVisual(
    Button = BtnPositionLeft,
    Name = TxtPositionLeft,
    Glow = BtnGlowLeft,
    IsSelected = (PositionState == Left),
    ButtonType = Position
)

Then2:
SetCommonButtonVisual(
    Button = BtnPositionRight,
    Name = TxtPositionRight,
    Glow = BtnGlowRight,
    IsSelected = (PositionState == Right),
    ButtonType = Position
)

Then3:
SetCommonButtonVisual(
    Button = BtnPositionBack,
    Name = TxtPositionBack,
    Glow = BtnGlowBack,
    IsSelected = (PositionState == Back),
    ButtonType = Position
)

Then4:
SetCommonButtonVisual(
    Button = BtnPositionFront,
    Name = TxtPositionFront,
    Glow = BtnGlowFront,
    IsSelected = (PositionState == Front),
    ButtonType = Position
)

```

## CanMove

```

WBP_Operator / CanMove (pure)

```

```

Return:
CanMove =
    (StudioManagerRef.CurrentCharacterState != Talk)
    && (StudioManagerRef.CurrentCharacterState != Dance)
    && !StudioManagerRef.IsGesturing

```

## ClickedPosition

```

WBP_Operator / ClickedPosition

Input:
PositionState

→ Sequence

Then0:
→ Branch(
    CanMove()
)

True:
SetLookTrackingOff()

→ PrePositionState =
    StudioManagerRef.CurrentPositionState

→ SetPositionButtonState(
    PositionState = PositionState
)

False:
→ Return

Then1:
SetCharacterStateButtonState(
    CharacterState = StudioManagerRef.CurrentCharacterState
)

PositionChanged =
    !(PrePositionState == StudioManagerRef.CurrentPositionState)

IsCustomPosition =
    (StudioManagerRef.CurrentPositionState == Custom)

→ Branch(
    PositionChanged
    || IsCustomPosition
)

True:
SetEnableAllGestureButton(
    IsEnabled = false
)

False:
(no connection visible)

```

## SetCameraButtonState

```

WBP_Operator / SetCameraButtonState

Input:
CameraState

StudioManagerRef.UpdateCameraState(
    StageViewActive = StageViewActive,
    NewCameraState = CameraState
)

→ Sequence

Then0:
SetCommonButtonVisual(
    Button = BtnCameraFull,
    Name = TxtCameraFull,
    Glow = BtnGlowFull,
    IsSelected = (CameraState == Full),
    ButtonType = Camera
)

Then1:
SetCommonButtonVisual(
    Button = BtnCameraUpper,
    Name = TxtCameraUpper,
    Glow = BtnGlowUpper,
    IsSelected = (CameraState == Upper),
    ButtonType = Camera
)

Then2:
SetCommonButtonVisual(
    Button = BtnCameraClose,
    Name = TxtCameraClose,
    Glow = BtnGlowClose,

```

```

        IsSelected = (CameraState == Close),
        ButtonType = Camera
    )

```

## SetStageViewOff

```

WBP_Operator / SetStageViewOff

CB_StageView.SetCheckedState(
    InCheckedState = Unchecked
)

→ Branch(
    StageViewActive
)

True:
PlayAnimation(
    Animation = StageViewToggleOffToOn,
    StartAtTime = 0.0,
    NumLoopsToPlay = 1,
    PlayMode = Reverse,
    PlaybackSpeed = 1.0
)

→ StageViewActive = false

→ StudioManagerRef.SetStageView(
    Active = StageViewActive
)

→ SetCameraButtonState(
    CameraState = StudioManagerRef.CurrentCameraState
)

False:
(no connection visible)

```

## SetLightingButtonState

```

WBP_Operator / SetLightingButtonState

Input:
LightingState

StudioManagerRef.UpdateLightingState(
    NewLightingState = LightingState
)

→ Sequence

Then0:
SetCommonButtonVisual(
    Button = BtnLightingNeutral,
    Name = TxtLightingNeutral,
    Glow = BtnGlowNeutral,
    IsSelected = (LightingState == Neutral),
    ButtonType = Lighting
)

Then1:
SetCommonButtonVisual(
    Button = BtnLightingFocus,
    Name = TxtLightingFocus,
    Glow = BtnGlowFocus,
    IsSelected = (LightingState == Focus),
    ButtonType = Lighting
)

Then2:
SetCommonButtonVisual(
    Button = BtnLightingStage,
    Name = TxtLightingStage,
    Glow = BtnGlowStage,
    IsSelected = (LightingState == Stage),
    ButtonType = Lighting
)

```

## SetGestureStateButtonState

```

WBP_Operator / SetGestureStateButtonState

Input:
GestureState

StudioManagerRef.UpdateGestureState(
    NewGestureState = GestureState
)

→ StudioManagerRef.PlayGesture(
    CurrentGestureState = GestureState
)

```

```

→ SetEnableAllGestureButton(
    IsEnabled = false
)

```

## SetEnableGestureButton

```

WBP_Operator / SetEnableGestureButton

Input:
Button
Text
IsEnabled

→ Branch(IsEnabled)

True:
Button.SetIsEnabled(
    InIsEnabled = true
)

→ Text.SetColorAndOpacity(
    EnabledTextColor
)

→ Button.SetStyle(
    ButtonStyle
)

False:
Button.SetIsEnabled(
    InIsEnabled = false
)

→ Text.SetColorAndOpacity(
    DisabledTextColor
)

```

## SetEnableAllGestureButton

```

WBP_Operator / SetEnableAllGestureButton

Input:
IsEnabled

→ Sequence

Then0:
SetEnableGestureButton(
    Button = BtnGestureWave,
    Text = TxtGestureWave,
    IsEnabled = IsEnabled
)

→ SetEnableGestureButton(
    Button = BtnGesturePose,
    Text = TxtGesturePose,
    IsEnabled = IsEnabled
)

→ SetEnableGestureButton(
    Button = BtnGesturePoint,
    Text = TxtGesturePoint,
    IsEnabled = IsEnabled
)

Then1:
SetEnableGestureButton(
    Button = BtnGestureCrossHands,
    Text = TxtGestureCrossHands,
    IsEnabled = IsEnabled
)

→ SetEnableGestureButton(
    Button = BtnGestureLaugh,
    Text = TxtGestureLaugh,
    IsEnabled = IsEnabled
)

→ SetEnableGestureButton(
    Button = BtnGestureClap,
    Text = TxtGestureClap,
    IsEnabled = IsEnabled
)

```

## ClickedGesture

```

WBP_Operator / ClickedGesture

Input:
GestureState

SetGestureStateButtonState(

```

```

        GestureState = GestureState
    )

    → SetCharacterStateButtonState(
        CharacterState = StudioManagerRef.CurrentCharacterState
    )

```

## SetCommonButtonVisual

```

WBP_Operator / SetCommonButtonVisual

Input:
Button
Name
Glow
IsSelected
ButtonType

→ Branch(IsSelected)

True:
SetSelectedCommonButton(
    Button = Button,
    Name = Name,
    Glow = Glow,
    ButtonType = ButtonType
)

False:
SetUnselectedCommonButton(
    Button = Button,
    Name = Name,
    Glow = Glow,
    ButtonType = ButtonType
)

```

## SetSelectedCommonButton

```

WBP_Operator / SetSelectedCommonButton

Input:
Button
Name
Glow
ButtonType

→ SwitchOnE_CommonButton(
    Selection = ButtonType
)

CharacterState:

IsDisabled =
    BountyHunterRef.IsMoving
    || StudioManagerRef.IsGesturing

→ Branch(IsDisabled)

True:
Name.SetColorAndOpacity(
    DisabledColor
)
→ Button.SetStyle(
    ButtonStyleDisabled
)
→ Glow.Visibility = Hidden

False:
Name.SetColorAndOpacity(
    SelectedColor
)
→ Button.SetStyle(
    ButtonStyleSelected
)
→ Glow.Visibility = Visible

Position:

IsDisabled =
    (StudioManagerRef.CurrentCharacterState == Talk)
    || (StudioManagerRef.CurrentCharacterState == Dance)
    || StudioManagerRef.IsGesturing

→ Branch(IsDisabled)

True:
Name.SetColorAndOpacity(
    DisabledColor
)
→ Button.SetStyle(
    ButtonStyleDisabled
)
→ Glow.Visibility = Hidden

False:
Name.SetColorAndOpacity(
    SelectedColor
)

```

```

)
→ Button.SetStyle(
    ButtonStyleSelected
)
→ Glow.Visibility = Visible

Camera:

→ Branch(StageViewActive)

True:
Name.SetColorAndOpacity(
    DisabledColor
)
→ Button.SetStyle(
    ButtonStyleDisabled
)
→ Glow.Visibility = Hidden

False:
Name.SetColorAndOpacity(
    SelectedColor
)
→ Button.SetStyle(
    ButtonStyleSelected
)
→ Glow.Visibility = Visible

Lighting:

Name.SetColorAndOpacity(
    SelectedColor
)
→ Button.SetStyle(
    ButtonStyleSelected
)
→ Glow.Visibility = Visible

```

## SetUnselectedCommonButton

```

WBP_Operator / SetUnselectedCommonButton

Input:
Button
Name
Glow
ButtonType

→ SwitchOnE_CommonButton(
    Selection = ButtonType
)

CharacterState:

IsDisabled =
    BountyHunterRef.IsMoving
    || StudioManagerRef.IsGesturing

→ Branch(IsDisabled)

True:
Name.SetColorAndOpacity(
    DisabledColor
)
→ Button.SetStyle(
    ButtonStyleDisabled
)
→ Glow.Visibility = Hidden

False:
Name.SetColorAndOpacity(
    DefaultColor
)
→ Button.SetStyle(
    ButtonStyleDefault
)
→ Glow.Visibility = Hidden

Position:

IsDisabled =
    (StudioManagerRef.CurrentCharacterState == Talk)
    || (StudioManagerRef.CurrentCharacterState == Dance)
    || StudioManagerRef.IsGesturing

→ Branch(IsDisabled)

True:
Name.SetColorAndOpacity(
    DisabledColor
)
→ Button.SetStyle(
    ButtonStyleDisabled
)
→ Glow.Visibility = Hidden

False:
Name.SetColorAndOpacity(

```

```

        DefaultColor
    )
    → Button.SetStyle(
        ButtonStyleDefault
    )
    → Glow.Visibility = Hidden

Camera:

Name.SetColorAndOpacity(
    DefaultColor
)
    → Button.SetStyle(
        ButtonStyleDefault
    )
    → Glow.Visibility = Hidden

Lighting:

Name.SetColorAndOpacity(
    DefaultColor
)
    → Button.SetStyle(
        ButtonStyleDefault
    )
    → Glow.Visibility = Hidden

```

## ResetAllState

```

WBP_Operator / ResetAllState

→ Sequence

Then0:
SetPresetButtonState(
    PresetState = Standby
)

→ SetCharacterStateButtonState(
    CharacterState = Idle
)

→ SetPositionButtonState(
    PositionState = Center
)

→ SetCameraButtonState(
    CameraState = Full
)

→ StudioManagerRef.UpdatePositionState(
    NewPositionState = Center
)

Then1:
SetLightingButtonState(
    LightingState = Neutral
)

→ SetAppearancePresetButtonState(
    AppearancePresetState = Preset_01
)

→ SetStageViewOff()

→ SetLookTrackingOff()

```

## SetLookTrackingOff

```

WBP_Operator / SetLookTrackingOff

CB_LookTracking.SetCheckedState(
    InCheckedState = Unchecked
)

→ Branch(
    LookTrackingActive
)

True:
PlayAnimation(
    Animation = MouseLookToggleOffToOn,
    StartAtTime = 0.0,
    NumLoopsToPlay = 1,
    PlayMode = Reverse,
    PlaybackSpeed = 1.0
)

→ LookTrackingActive = false

→ StudioManagerRef.UpdateLookTrackingState(
    LookTrackingActive = false
)

```

```
False:  
(no connection visible)
```

## SetAppearanceButtonVisual

```
WBP_Operator / SetAppearanceButtonVisual  
  
Input:  
Button  
BackFrame  
IsSelected  
  
→ Branch(IsSelected)  
  
True:  
SetSelectedAppearanceButton(  
    Button = Button,  
    BackFrame = BackFrame  
)  
  
False:  
SetUnselectedAppearanceButton(  
    Button = Button,  
    BackFrame = BackFrame  
)
```

## SetSelectedAppearanceButton

```
WBP_Operator / SetSelectedAppearanceButton  
  
Input:  
Button  
BackFrame  
  
Button.SetStyle(  
    ButtonStyleSelected  
)  
  
→ BackFrame.Visibility = Visible
```

## SetUnselectedAppearanceButton

```
WBP_Operator / SetUnselectedAppearanceButton  
  
Input:  
Button  
BackFrame  
  
Button.SetStyle(  
    ButtonStyleSelected  
)  
  
→ BackFrame.Visibility = Hidden
```

## SetAppearancePresetButtonState

```
WBP_Operator / SetAppearancePresetButtonState  
  
Input:  
AppearancePresetState  
  
StudioManagerRef.UpdateAppearancePresetState(  
    NewAppearancePresetState = AppearancePresetState  
)  
  
→ Sequence  
  
Then0:  
SetAppearanceButtonVisual(  
    Button = BtnPreset1,  
    BackFrame = ImagePreset1,  
    IsSelected = (AppearancePresetState == Preset_01)  
)  
  
Then1:  
SetAppearanceButtonVisual(  
    Button = BtnPreset2,  
    BackFrame = ImagePreset2,  
    IsSelected = (AppearancePresetState == Preset_02)  
)  
  
Then2:  
SetAppearanceButtonVisual(  
    Button = BtnPreset3,  
    BackFrame = ImagePreset3,  
    IsSelected = (AppearancePresetState == Preset_03)  
)  
  
Then3:
```

```

SetAppearanceButtonVisual(
    Button = BtnPreset4,
    BackFrame = ImagePreset4,
    IsSelected = (AppearancePresetState == Preset_04)
)

Then4:
SetAppearanceButtonVisual(
    Button = BtnPreset5,
    BackFrame = ImagePreset5,
    IsSelected = (AppearancePresetState == Preset_05)
)

```

## DataTable

### Preset

#### ST\_PresetData

```

ST_PresetData / Struct

Fields:
PresetState : E_PresetState
CharacterState : E_CharacterState
CameraState : E_CameraState
LightingState : E_LightingState

```

#### DT\_PresetData

| Row Name     | PresetState  | CharacterState | CameraState | LightingState |
|--------------|--------------|----------------|-------------|---------------|
| Standby      | Standby      | Idle           | Full        | Neutral       |
| Presentation | Presentation | Talk           | Upper       | Focus         |
| Performance  | Performance  | Dance          | Full        | Stage         |

### AppearancePreset

#### ST\_AppearancePresetData

```

ST_AppearancePresetData / Struct

Fields:
HairColor : LinearColor
EyebrowColor : LinearColor
ClothesParameters : ST_ClothesAppearanceData
EyeColor : LinearColor
LipColors : ST_LipAppearanceData
SkinParameters : ST_SkinAppearanceData

```

#### ST\_ClothesAppearanceData

```

ST_ClothesAppearanceData / Struct

Fields:
GogglesHue : Float
ShoulderArmorHue : Float
DressHue : Float
DressDepth : Float
ShoelacesHue : Float
CoatHue : Float
CoatDepth : Float

```

#### ST\_LipAppearanceData

```

ST_LipAppearanceData / Struct

Fields:
Lip_Color01 : Linear Color
Lip_Color02 : Linear Color

```

#### ST\_SkinAppearanceData

```

ST_SkinAppearanceData / Struct

Feilds:
SkinHue : Float
SkinDepth : Float

```

#### DT\_AppearancePresetData

| Row Name  | HairColor                                     | EyebrowColor                                  | ClothesParameters                  |
|-----------|-----------------------------------------------|-----------------------------------------------|------------------------------------|
| Preset_01 | (R=0.679543,G=0.072272,B=0.000000,A=1.000000) | (R=0.278894,G=0.132868,B=0.039546,A=1.000000) | (GogglesHue_6_78ABB89E43AB31904495 |
| Preset_02 | (R=0.452571,G=0.669565,B=0.000000,A=1.000000) | (R=0.165132,G=0.215861,B=0.102242,A=1.000000) | (GogglesHue_6_78ABB89E43AB31904495 |
| Preset_03 | (R=1.000000,G=0.182609,B=0.466446,A=1.000000) | (R=0.332452,G=0.144128,B=0.226966,A=1.000000) | (GogglesHue_6_78ABB89E43AB31904495 |
| Preset_04 | (R=0.143229,G=0.258873,B=0.916667,A=1.000000) | (R=0.138432,G=0.187821,B=0.323143,A=1.000000) | (GogglesHue_6_78ABB89E43AB31904495 |
| Preset_05 | (R=1.000000,G=0.519731,B=0.286458,A=1.000000) | (R=0.327778,G=0.219526,B=0.088656,A=1.000000) | (GogglesHue_6_78ABB89E43AB31904495 |

## Camera

### ST\_CameraData

|                                |
|--------------------------------|
| ST_CameraData / Struct         |
| Fields:                        |
| SpringArmLength : Float        |
| SpringArmSocketOffsetZ : Float |
| CameraFieldOfView : Float      |
| CameraPitch : Float            |
| SpringArmMinClamp : Float      |
| SpringArmMaxClamp : Float      |
| FocusSocketName : Name         |

### DT\_CameraData

| Row Name | SpringArmLength | SpringArmSocketOffsetZ | CameraFieldOfView | CameraPitch | SpringArmMinClamp | SpringArmMaxClamp | FocusSocketName |
|----------|-----------------|------------------------|-------------------|-------------|-------------------|-------------------|-----------------|
| Full     | 421.465550      | 63.497726              | 55.000000         | -8.000000   | -77.000000        | 16.000000         | pelvis          |
| Upper    | 186.131323      | 68.152200              | 40.000000         | -7.000000   | -28.000000        | 23.000000         | spine_04        |
| Close    | 121.561730      | 60.800000              | 30.000000         | -3.000000   | -12.000000        | 22.000000         | head            |

## Lighting

### ST\_LightingData

|                                   |
|-----------------------------------|
| ST_LightingData / Struct          |
| Fields:                           |
| CharacterLightIntensity_1 : Float |
| CharacterLightIntensity_2 : Float |
| CharacterLightColor : LinearColor |
| bTrackingBeamVisible : Boolean    |
| TrackingDimmer : Float            |
| TrackingBeamColor : LinearColor   |
| bStageLightVisible : Boolean      |
| StageLightColor : LinearColor     |
| bStageLightMove : Boolean         |

### DT\_LightingData

| Row Name | CharacterLightIntensity_1 | CharacterLightIntensity_2 | CharacterLightColor                           | bTrackingBeamVisible | TrackingDimmer |
|----------|---------------------------|---------------------------|-----------------------------------------------|----------------------|----------------|
| Neutral  | 30.000000                 | 20.000000                 | (R=1.000000,G=1.000000,B=1.000000,A=1.000000) | False                | 0.000000       |
| Focus    | 40.000000                 | 40.000000                 | (R=1.000000,G=0.998057,B=0.911458,A=1.000000) | True                 | 0.015000       |
| Stage    | 50.000000                 | 50.000000                 | (R=1.000000,G=0.626087,B=0.826087,A=1.000000) | True                 | 0.010000       |